



PDF Download
3548606.3560676.pdf
13 January 2026
Total Citations: 21
Total Downloads: 2232

 Latest updates: <https://dl.acm.org/doi/10.1145/3548606.3560676>

RESEARCH-ARTICLE

Batching, Aggregation, and Zero-Knowledge Proofs in Bilinear Accumulators

SHRAVAN SRINIVASAN, University of Maryland, College Park, College Park, MD, United States

IOANNA KARANTAIDOU, George Mason University, Fairfax, VA, United States

FOTEINI BALDIMTSI, George Mason University, Fairfax, VA, United States

CHARALAMPOS PAPAMANTHOU, Yale University, New Haven, CT, United States

Open Access Support provided by:

George Mason University

University of Maryland, College Park

Yale University

Published: 07 November 2022

[Citation in BibTeX format](#)

CCS '22: 2022 ACM SIGSAC Conference on Computer and Communications Security

November 7 - 11, 2022
CA, Los Angeles, USA

Conference Sponsors:
SIGSAC



PDF下载
3548606.3560676.pdf
2026年1月13日 总引用
次数: 21
下载总数: 2232

发布日期: 2022年11月7日

BibTeX格式引用

中国化学会 '22:2022 ACM SIGSAC

会议 计算机与通信 安全

2022年11月7日至11日

美国洛杉矶加利福尼亚州

会议赞助商:
SIGSAC

DL 最新更新: <https://dl.acm.org/doi/10.1145/3548606.3560676>

RESEARCH-ARTICLE

双线性累加器中的批量处理、聚合与零知识证明

Shravan Srinivasan, 马里兰大学帕克分校, 帕克分校, 马里兰州, 美国

IOANNA karantaidou, 美国弗吉尼亚州费尔法克斯乔治梅森大学 foteini baldimtsi, 美国

弗吉尼亚州费尔法克斯乔治梅森大学 charalampos papamantou, 美国康涅狄格州纽黑文
耶鲁大学

开放获取支持由以下机构提供:

乔治梅森大学

马里兰大学帕克分校

耶鲁大学

Batching, Aggregation, and Zero-Knowledge Proofs in Bilinear Accumulators

Shravan Srinivasan
University of Maryland
College Park, USA

Foteini Baldimtsi
George Mason University
Fairfax, USA

Ioanna Karantaidou
George Mason University
Fairfax, USA

Charalampos Papamanthou
Yale University
New Haven, USA

ABSTRACT

An accumulator is a cryptographic primitive that allows a prover to succinctly commit to a set of values while being able to provide proofs of (non-)membership. A batch proof is an accumulator proof that can be used to prove (non-)membership of multiple values simultaneously.

In this work, we present a zero-knowledge batch proof with constant proof size and constant verification in the Bilinear Pairings (BP) setting. Our scheme is $16\times$ to $42\times$ faster than state-of-the-art SNARK-based zero-knowledge batch proofs in the RSA setting. Additionally, we propose protocols that allow a prover to aggregate multiple individual non-membership proofs, in the BP setting, into a single batch proof of constant size. Our construction for aggregation satisfies a strong soundness definition—one where the accumulator value can be chosen arbitrarily.

We evaluate our techniques and systematically compare them with RSA-based alternatives. Our evaluation results showcase several scenarios for which BP accumulators are clearly preferable and can serve as a guideline when choosing between the two types of accumulators.

CCS CONCEPTS

• Security and privacy → Cryptography; • Theory of computation → Cryptographic primitives; Cryptographic protocols.

KEYWORDS

Accumulators, bilinear pairings, proof aggregation, zero-knowledge

ACM Reference Format:

Shravan Srinivasan, Ioanna Karantaidou, Foteini Baldimtsi, and Charalampos Papamanthou. 2022. Batching, Aggregation, and Zero-Knowledge Proofs in Bilinear Accumulators. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*, November 7–11, 2022, Los Angeles, CA, USA. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3548606.3560676>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CCS '22, November 7–11, 2022, Los Angeles, CA, USA

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-9450-5/22/11...\$15.00

<https://doi.org/10.1145/3548606.3560676>

1 INTRODUCTION

An *accumulator* is an authenticated data structure for a set of elements. It allows a *prover* to provide a succinct binding digest to a set of elements and to generate a short proof of membership or non-membership for any element in the accumulator domain. A *verifier* can efficiently check the proof of (non-)membership using the digest without requiring access to the entire set. Accumulators have found numerous applications including timestamping [8], fail-stop signature schemes [6], anonymous credentials [2, 4, 15, 16], cloud storage [48, 57] and more recently, stateless and privacy-preserving cryptocurrencies [10, 21, 35].

Batching and aggregation. In traditional applications, accumulators have been used in a setting where the prover had to provide (non-)membership proofs for a single element at a time. However, in emerging applications, such as cryptocurrencies, a prover must simultaneously prove (non-)membership of multiple elements. Naively, the prover could include individual proofs for each element, but this imposes high bandwidth usage and computational cost on the verifier. A better approach is a *batch proof*, that is, a succinct proof for multiple elements, which can be used to efficiently and simultaneously prove (non-)membership of multiple elements. For example, in UTXO-based stateless blockchains [10, 21], all transactions are accompanied by a proof of membership in the UTXO set. If a block proposer naively includes all individual proofs for validation (instead of a single batch proof), the size of the blocks transmitted across the network increases along with the computational overhead on the verifiers.

Let X be the set of elements in an accumulator. A batch membership proof for a set of elements $I \subseteq X$ can be computed: (1) by using the *trapdoor* (e.g., factors of the modulus in the RSA setting), or (2) *from scratch* using *all* the elements in X , or (3) by *aggregating* previously computed individual proofs of elements in I . Unfortunately, computing the batch proof using the trapdoor is impractical as it requires a trusted accumulator manager to hold the trapdoor. Furthermore, if the trapdoor is compromised, an adversary can forge proofs at-will. Computing the batch proof using the entire set is also impractical in a *distributed setting*, as nodes may not have access to the entire set or the trapdoor or a trusted accumulator manager. Moreover, updates to the accumulator arrive in batches (e.g., batch of transactions in a block). In comparison with other approaches, computing the batch proof by *aggregating* individual proofs is useful and relevant in the distributed setting as it does not require the trapdoor or the entire set. This brings us to the first question: *Is it possible to efficiently aggregate individual (non-)membership proofs?*

双线性累加器中的批量处理、聚合与零知识证明

斯拉万·斯里尼瓦桑

美国马里兰大学帕克分校

Foteini Baldimtsi

乔治梅森大学

美国费尔法克斯

Ioanna Karantaidou 美国

乔治梅森大学费尔法克斯分校

查拉曼波斯

耶鲁大学

美国纽黑文

摘要

累加器是一种密码学原语，它允许证明者简洁地承诺一组值，同时能够提供（非）成员资格的证明。批量证明是一种累加器证明，可用于同时证明多个值的（非）成员资格。

在本研究中，我们提出了一种双线性配对（BP）场景下的零知识批量证明方案，其证明规模和验证时间均保持常数。相较于当前RSA场景下基于snark的零知识批量证明，我们的方案速度提升了16至42倍。此外，我们还设计了协议，使证明者能够在BP场景中将多个独立的非成员证明聚合为单一的常规模批量证明。该聚合机制满足强健性定义——即允许任意选择累加器的初始值。

我们评估了所提出的技术，并系统地将其与基于RSA的替代方案进行比较。评估结果展示了多种场景，其中BP累加器明显更优，可作为选择两种类型累加器时的指导依据。

中国化学会概念

• 安全性和隐私性—密码学；计算理论—密码学原语；密码学协议。

关键词

累加器、双线性配对、证明聚合、零知识

ACM参考格式：

Shravan Srinivasan、Ioanna Karantaidou、Foteini Baldimtsi 和 Charalampos Papamanthou。2022年。双线性累加器中的批量处理、聚合与零知识证明。收录于2022年ACM计算机与通信安全SIGSAC会议论文集（中国化学会'22），2022年11月7-11日，美国加利福尼亚州洛杉矶。ACM，美国纽约州纽约市，15页。<https://doi.org/10.1145/3548606.3560676>

本作品允许个人或课堂教学用途免费制作数字或纸质副本，但需满足以下条件：副本不得用于营利或商业用途，且首页必须标注本声明及完整引用信息。对于作者以外第三方持有的作品组成部分，必须遵守版权规定。允许在注明出处的前提下进行摘要引用。如需以其他方式复制、转载、上传至服务器或分发至邮件列表，须事先获得特别授权或支付费用。授权申请请发送邮件至permissions@acm.org。

中国化学会 22,2022年11月7日至11日，美国加利福尼亚州洛杉矶

© 2022 版权所有/作者保留。出版授权给ACM。ACM ISBN 978-1-4503-9450-5/22/11... \$15.00 <https://doi.org/10.1145/3548606.3560676>

1 介绍

累加器是一种用于一组元素的认证数据结构。它允许证明者为元素提供简洁的绑定摘要，并为累加器域中的任意元素生成简短的成员或非成员证明。验证者可以高效地使用该摘要验证（非）成员证明，而无需访问整个集合。累加器已应用于多种场景，包括时间戳[8]、故障停止签名方案[6]、匿名凭证[2, 4, 15, 16]、云存储[48, 57]，以及最近的无状态和隐私保护型加密货币[10, 21, 35]。

批处理与聚合。在传统应用场景中，累加器通常用于需要证明者逐个提供（非）成员证明的情况。然而在加密货币等新兴应用中，证明者必须同时验证多个元素的（非）成员关系。若采用传统方式，证明者需为每个元素单独生成证明，这将导致验证方面临高带宽消耗和计算成本压力。更优的解决方案是采用**批量证明**——即针对多个元素生成简洁的证明，从而高效实现多元素（非）成员关系的同步验证。例如在基于UTXO的无状态区块链[10、21]中，所有交易都需附带UTXO集合成员证明。若区块提议者仍采用传统方式逐个提交验证证明（而非批量处理），网络传输的区块数据量将大幅增加，验证方的计算开销也会随之攀升。

设 x 为累加器中的元素集合。对于元素集合 I 中 x 的批量成员证明可通过以下方式计算：(1) 使用**陷阱门**（例如RSA场景中的模数因子），或(2) **从头开始**使用所有 x 中的元素，或(3) **聚合**先前计算的 I 中元素的个体证明。遗憾的是，使用陷阱门计算批量证明是不切实际的，因为它需要可信的累加器管理器持有陷阱门。此外，如果陷阱门被攻破，攻击者可以随意伪造证明。在**分布式场景**中，使用整个集合计算批量证明也不切实际，因为节点可能无法访问整个集合、陷阱门或可信的累加器管理器。此外，累加器的更新以批次形式到达（例如区块中的交易批次）。与其他方法相比，通过**聚合**单个证明来计算批证明在分布式环境中是有用和相关的，因为它不需要陷阱门或整个集合。这就引出了第一个问题：**是否有可能有效地聚合单个（非）成员证明？**

Zero-knowledge proofs. Proofs of (non-)membership for accumulators are often required in applications where privacy is critical. As an example, consider the application of anonymous credentials for authentication where valid (i.e., non-revoked) credentials are stored in an accumulator. When users wish to prove something about their credential embedded attributes, they also need to prove that their credential is valid via a proof of membership. Such proofs are usually done in a zero-knowledge (ZK) fashion in order to guarantee unlinkability between proving sessions and specific user credentials. More concretely, let x be a user credential accumulated in A . The user will compute a commitment $c = \text{Commit}(x)$, and prove, in zero-knowledge, membership of the committed value x . However, being able to simultaneously prove (non-)membership of a set of elements, I , is an important property in scenarios where a user/organization controls multiple credentials. Additionally, a prover may also need to argue that this set I is of at least some size d while hiding I . For instance an organization holding $|I| \geq d$ valid credentials might want to prove in ZK that it can cast up to d votes. Such questions become even more relevant in recently developed decentralized identity systems [1, 30]. This brings us to a second question: *Is it possible to efficiently prove knowledge of a set I that is a subset of/disjoint from X , without revealing I ? And reveal a lower bound of $|I|$, if needed?*

Batching, aggregation, and ZK in RSA accumulators. One of the most popular accumulator instantiations is the *RSA accumulator* [8, 16, 32]. Given a set X of prime numbers $(x_1, \dots, x_n)$, one can define the accumulator A_X as the RSA group element $g^{\prod x_i}$, and a membership proof w of $y \in X$ as the accumulator of $X \setminus \{y\}$, which can be verified by checking whether w^y equals A_X . Boneh et al. [10] defined batch proofs for the RSA accumulator and also provided aggregation algorithms for both membership and non-membership RSA proofs. The resulting batch proofs are non-interactive and of constant size. They also present a Proof-of-Exponentiation protocol (PoE) to concretely speed up batch verification by reducing the number of group operations from $O(|I|)$ to a constant. Without PoE, the verifier would need to perform a large exponentiation that grows with the number of elements in the proof. On the ZK front, Ozdemir et al., introduced *improved* SNARK-friendly techniques to batch prove (non-)membership in the RSA setting [39]. Subsequently, Campanelli et al. adopted a “hybrid” approach where they prove the batch membership without SNARKs and prove that elements of the batch are from the prime domain using SNARKs [18]. However, their accumulator does not support both membership and non-membership simultaneously.

While RSA accumulators have been used in many applications, they do present some crucial limitations:

- First, RSA group elements are large and this affects verification time and proof sizes.
- Second, they only support accumulation of elements that reside in a prime domain. Thus, they cannot be directly used for the accumulation of arbitrary elements.¹

These problems are fundamental and limit the use of RSA accumulators in the batch setting regardless of the privacy concerns.

The case for bilinear accumulators. As opposed to RSA accumulators, *bilinear-pairing accumulators* (BP) [38, 57] have much smaller proofs and faster exponentiations. Moreover, BP accumulators can support the accumulation of arbitrary elements, making them directly applicable to a broader set of applications. Given a set X of arbitrary numbers $(x_1, \dots, x_n)$, the accumulator A is: $g^{\prod (s+x_i)}$, where g is a prime order group generator and s is a secret *trapdoor*. The membership proof w of $y \in X$ is the accumulator of $X \setminus \{y\}$, which can be verified by using the A_X and the pairing operator. As highlighted above, we are interested in accumulators for the distributed setting, i.e., where nobody can have access to the secret trapdoor s or the accumulated set X . Existing batching approaches in bilinear setting are only secure under a weak soundness definition [19, 26, 44] (with [44] additionally missing security proof), or require large public parameters [57]. Moreover, to the best of our knowledge, no prior work allows for efficient aggregation of non-membership proofs or efficient zero-knowledge (non-)membership proofs in the batch setting for *trapdoorless* accumulators (no accumulator operation except the setup requires the trapdoor, and the trapdoor is destroyed after setup if used). This is the main focus of our work.

Contributions. In the distributed setting, we make the following contributions to trapdoorless BP accumulators:

- (1) We formally prove soundness of batch membership and non-membership proofs for the Nguyen [38] accumulator under the Uber assumption [7] (§4). Our proof holds for a *stronger* definition of soundness than those considered in prior works, since we *do not* assume that the accumulator is well-formed and we allow the adversary to pick the accumulator value (Definition 3.2). We remark that this strong definition is *crucial* for many modern applications of accumulators, especially in the distributed/blockchain setting, as it is not always realistic to assume that the accumulator value is well-formed.
- (2) We design the first efficient ZK scheme that can prove batch (non-)membership of BP accumulator using the knowledge-of-exponent assumption (§7). Moreover, we show how to additionally reveal a lower bound on the size of the batch witness without revealing the elements of the batch proof. Asymptotically, our proof size is constant, the verification cost is constant, and the prover is $O(d)$, where d is the size of the batched subset.
- (3) We propose a new algorithm to aggregate individual non-membership proofs into a single constant sized proof (§5) and provide a constant-time algorithm to update the extended Euclidean based individual non-membership proofs (§6).
- (4) We perform an experimental evaluation and comparison of the RSA and BP accumulators in both ZK and non-ZK setting (§9). Concretely, we observe that in the ZK setting, both the prover and the verifier is at least $16\times$ and $7\times$ faster than RSA based baseline, respectively.

In the non-ZK setting, we benchmark the batching and aggregation of accumulator proofs. For the first time, we explicitly take the mapping cost (from arbitrary domain to accumulator domain) into account. We observe that membership and non-membership proofs in BP accumulators are $2.5\times$ to $5\times$

¹While there exist techniques to map arbitrary elements to primes (i.e., hash to primes [10]), such mappings can harm the soundness of the accumulator if not carefully implemented during verification, especially if the mappings are not 1-1.

零知识证明。在隐私至关重要的应用场景中, 通常需要对累加器进行(非)成员资格证明。例如, 在使用匿名凭证进行身份验证时, 有效(即未被撤销)的凭证会被存储在累加器中。当用户希望证明其凭证嵌入属性的某些信息时, 还需通过成员资格证明来验证其凭证的有效性。此类证明通常采用零知识(ZK)方式完成, 以确保证明会话与特定用户凭证之间的不可关联性。具体而言, 设 x 是存储在 A 中的用户凭证。用户将计算承诺 $c = \text{Commit}(x)$, 并通过零知识方式证明承诺值 x 的成员资格。然而, 在用户/组织控制多个凭证的场景中, 能够同时证明一组元素 I 的(非)成员资格, 是一个重要特性。此外, 证明者可能还需要论证这个集合 I 在隐藏 I 的同时至少具有某个大小 d 。例如, 一个持有 $|I| \geq d$ 个有效凭证的组织可能希望在ZK中证明其最多能投出 d 票。这类问题在最近开发的去中心化身份系统中变得更为相关[1, 30]。这引出了第二个问题: 是否可以在不泄露集合 I 的情况下, 高效证明对 X 的子集/不交集 I 的认知? 以及在必要时揭示 I 的下限?

RSA累加器中的分批、聚合与ZK。最流行的累加器实例之一是RSA累加器[8, 16, 32]。给定一组素数 $X(x_1, \dots, x_n)$, 可以将累加器 A_X 定义为RSA群元素 $g^{\prod x_i}$, 并将 $y \in X$ 的成员证明 w 定义为 $X \setminus \{y\}$ 的累加器, 可通过检查 $w \cdot y$ 是否等于 A_X 来验证。Boneh等人[10]为RSA累加器定义了批量证明, 并为成员与非成员RSA证明提供了聚合算法。由此产生的批量证明是非交互式的且规模恒定。他们还提出了一种指数运算证明协议(PoE), 通过将群操作次数从 $O(|I|)$ 减少到常数, 具体加速了批量验证。若不采用PoE, 验证者需要执行随证明中元素数量增长的大型指数运算。在ZK方面, Ozdemir等人在RSA场景中引入了改进的SNARK友好技术来批量证明(非)成员性[39]。随后, Campanelli等人采用了一种“混合”方法: 他们通过SNARK证明批量成员性, 同时使用SNARK证明批量元素来自素数域[18]。然而, 他们的累加器无法同时支持成员与非成员性证明。

尽管RSA累加器已被广泛应用于多种场景, 但其仍存在若干关键局限性:

- 首先, RSA群元素较大, 这会影响验证时间和证明规模。
- 其次, 它们仅支持位于主域中的元素的积累。因此, 它们不能直接用于任意元素的积累。¹

这些问题具有根本性, 无论隐私问题如何, 都限制了RSA累加器在批量设置中的应用。

¹ 虽然存在将任意元素映射到素数的技术(即, 将哈希映射到素数[10]), 但如果在验证过程中没有仔细实现这些映射, 特别是如果这些映射不是一一对应的, 那么这些映射可能会损害累加器的可靠性。

双线性累加器的优势。与RSA累加器不同, 双线性配对累加器(BP)[38, 57]具有更小的证明量和更快的指数运算。此外, BP累加器可支持任意元素的累加, 使其直接适用于更广泛的应用场景。给定一组任意数 $X(x_1, \dots, x_n)$, 累加器 A 为: $g^{n \cdot (s + \sum x_i)}$, 其中 g 是素阶群生成元, S 是秘密陷阱门。 $y \in X$ 的成员证明 w 是 $X \setminus \{y\}$ 的累加器, 可通过 A_X 和配对算子进行验证。如上所述, 我们关注的是分布式场景下的累加器, 即无人能访问秘密陷阱门 S 或累加集 X 。现有的双线性设置中的批量处理方法仅在弱健全性定义下具有安全性[19、26、44](其中[44]还缺少安全证明), 或者需要较大的公共参数[57]。此外, 据我们所知, 此前没有任何工作允许在批量设置中高效聚合非成员证明或高效零知识(非)成员证明, 特别是在无后门累加器的场景下(除设置外, 累加器操作不需要后门, 且使用后门时会在设置后销毁)。这正是我们工作的主要关注点。

贡献。在分布式场景中, 我们针对无后门BP累加器做出了以下贡献:

- (1) 我们正式证明了在Uber假设[7](§4)下, Nguyen[38]累加器的批量成员与非成员证明的正确性。我们的证明适用于比先前研究中考虑的更强的正确性定义, 因为我们不假设累加器是良好形成的, 并且允许攻击者选择累加器值(定义3.2)。我们指出, 这种强定义对于许多现代累加器应用至关重要, 尤其是在分布式/区块链场景中, 因为假设累加器值是良好形成的并不总是现实的。
- (2) 我们设计了首个高效ZK方案, 该方案能够利用指数知识假设(§7)证明BP累加器的批量(非)成员关系。此外, 我们展示了如何在不泄露批量证明元素的情况下, 额外揭示批量见证的大小下限。渐近而言, 我们的证明规模为常数, 验证成本为常数, 且证明者的时间复杂度为 $O(d)$, 其中 d 表示批量子集的大小。
- (3) 我们提出一种新算法, 将个体非成员证明聚合为单一常规模证明(§5), 并提供一种常时算法来更新基于扩展欧几里得的个体非成员证明(§6)。
- (4) 我们对RSA和BP累加器在ZK和非ZK场景下的实验评估与比较(§9)进行了研究。具体而言, 在ZK场景中, 证明者和验证者的速度分别比基于RSA的基准方法快至少16倍和7倍。在非ZK场景中, 我们对累加器证明的分批处理和聚合进行了基准测试。首次明确考虑了映射成本(从任意域到累加器域)。我们观察到, BP累加器中的成员与非成员证明成本是2.5倍至5倍。

smaller and $3.5\times$ smaller than the RSA accumulators, respectively. Moreover, verification of aggregated BP accumulator proofs is on an average $4\times$ faster than the RSA accumulators.

- (5) Finally, in §8, we propose a PoE protocol in Elliptic Curve (EC) groups that help us speed up batch verification. Our PoE can prove exponentiation of an arbitrary group element and it concretely saves one exponentiation for the prover when compared to concurrent works [23, 44]. We use our PoE to verify exponentiation of a group element by ℓ -degree polynomial without having to perform $O(\ell)$ group exponentiations, only $O(\ell)$ operations in the field.

Implications of our results and evaluations. As indicated above, batch *membership* proofs in BP accumulators, clearly outperform their RSA counterparts in the size of the proofs ($2.5\times$ to $5\times$). This makes our protocols very appealing to applications where communication cost is critical.

An example of such an application is the extension of ℓ -bit Byzantine agreement (BA) and broadcast protocols (BB) [37] where bilinear-accumulators are used to obtain state-of-the-art communication costs for ℓ -bit BA/BB. Our batching techniques can improve their so called: “distribute phase” by batching the proofs sent between the participants which would result in a constant size proof and significantly reduce their communication costs.

In the ZK setting, the advantages of BP accumulators are apparent in both prover and verification costs. In decentralized identity systems with privacy, issuers sign the users attributes/identity and these signatures (aka the credential) are kept in an accumulator. Later, users should be able to perform batch proofs of (non-)membership for their stored credentials. Typically, the underlying signature schemes output integers (dlog, RSA-based schemes) or group elements that can be trivially mapped to integers (ECC schemes). Thus, one can immediately utilize a BP accumulator. If an RSA accumulator is used to hold the credentials, a H_{prime} function (maps to prime) needs to be applied to each element individually and a ZK proof of primality is required. Ozdemir et al. [39] show how the use of Pocklington certificates reduce the cost of proving primality in ZK, but it only reduces the number of Miller-Rabin primality tests. This still results in non-constant verification time. This hashing operation, along with a primality test, needs to also be included in the ZK proof. This makes the computation of the ZK proof much more complex (e.g., if one uses zk-SNARKs or other specialized ZK proofs for non-algebraic statements [3]).

Limitations. While both BP and RSA accumulators require a trusted setup phase to compute the public parameters, the size of BP parameters is significant: 18 MiB for 2^{17} elements (although not necessarily needed for verification). These parameters grow even more ($3\times$) in our ZK setting due to the use of the knowledge-of-exponent assumption. Finally, it should be noted that in BP accumulators (as opposed to RSA), it is unknown how to add or generate proofs of (non-)membership without the knowledge of the entire accumulated set X .

2 RELATED WORK

Based on the use of *trapdoors* for accumulator operations, accumulators can be: *trapdoor-based* or *trapdoorless*. In a *trapdoor-based*

accumulator, a trusted entity, called the *accumulator manager*, holds some secret trapdoor information, which allows the entity to efficiently perform accumulator operations. A *trapdoorless* accumulator on the other hand, operates without the trapdoor and if a trapdoor is used during setup, it is later destroyed.

We classify prior works into three broad categories: (1) accumulators based on hash functions, (2) accumulators in hidden-order groups, and (3) accumulators in known-order groups.

Hash-based. Accumulators built based on Merkle tree [5, 34] or Bloom-filters [56] do not support batching, aggregation, and ZK proof of batch (non-)membership without general purpose tools such as SNARKs, unlike the techniques proposed in this work.

Hidden-order groups. In the single-proof setting, Camenisch and Lysyanskaya [16] proposed the first *dynamic* accumulator (supports both set difference and set union without requiring to fully recompute the accumulator from scratch) secure under *strong RSA* assumption based on prior accumulator constructions [6, 8]. However, their construction is not in the trapdoorless setting as the trapdoor is used to delete elements from the accumulators. Li et al. [32] proposed the first *universal* accumulator (supports both membership and non-membership proofs) by generalizing the Camenisch and Lysyanskaya accumulators [16] to support non-membership proofs under the strong RSA assumption. They also provided efficient algorithms to update non-membership proofs on changes to the accumulated set.

Boneh et al. [10] support batching and aggregation of membership and non-membership proofs in the distributed and trapdoorless setting. They also leverage their contributions to realize a stateless blockchain [10, 21] in the UTXO setting. However, RSA based constructions have larger proof size and verification cost when compared to the BP-based constructions.

Known-order groups. In the single-proof setting, Nguyen proposed the first accumulator [38] using bilinear-maps and based on the t -strong bilinear Diffie-Hellman assumption. In a later work, Damgård et al. [24] and Au et al. [4] extended the accumulator construction by Nguyen to support constant-sized non-membership proof for a *single* element. This non-membership proof scheme relies on Polynomial Remainder Theorem (PRT) to prove non-membership [55]. However, this construction does not extend to a constant-sized batch non-membership proofs or consider efficient aggregation of non-membership proofs. In our work, we study constant-sized batch proofs and present aggregation techniques for the greatest common divisor (GCD) based non-membership construction, rather than the PRT-based.

Thakur [44] propose batching techniques for BP accumulators (for weak soundness and without rigorous analysis). In their latest version (Sept, 2021), they also propose two PoE protocols. However, one of the PoE approaches assumes that it is possible to exponentiate an arbitrary base to the trapdoor (which is only feasible in the trapdoor-based setting). Connolly et al. [23] also propose a PoE protocol based on [44], under a different assumption (q-co-GSDH). Compared to [44] and [23], our PoE is concretely one exponentiation fewer under the adaptive variant of q-SDH. Finally, neither [44] nor [23] consider aggregation or ZK batch proofs.

其体积更小, 且分别比RSA累加器小3.5倍。此外, 聚合BP累加器证明的验证速度平均比RSA累加器快4倍。

- (5) 最后, 在第8节中, 我们提出了一种椭圆曲线(EC)群中的PoE协议, 有助于加速批量验证。我们的PoE能够证明任意群元素的指数运算, 与并发研究[23、44]相比, 具体节省了证明者一次指数运算。我们利用PoE验证 ℓ 次多项式的群元素指数运算, 无需执行 $O(\ell)$ 次群指数运算, 仅需域中 $O(\ell)$ 次操作。

我们的研究结果与评估意义。如上所述, BP累加器中的批处理成员证明在证明规模上明显优于RSA的对应方案(2.5倍至5倍)。这使得我们的协议在通信成本至关重要的应用场景中极具吸引力。

此类应用的一个实例是扩展 ℓ 位拜占庭协议(BA)和广播协议(BB)[37], 其中使用双线性累加器来获得 ℓ 位BA/BB的先进通信成本。我们的分批技术可以通过分批处理参与者之间发送的证明来改进它们所谓的“分布式阶段”, 从而产生恒定大小的证明, 并显著降低其通信成本。

在ZK场景中, BP累加器的优势在证明和验证成本方面都显而易见。在具有隐私保护的去中心化身份系统中, 发行方会对用户属性/身份进行签名, 这些签名(即凭证)会被存储在累加器中。之后, 用户应能对其存储的凭证进行批量(非)成员资格证明。通常, 底层签名方案会输出整数(如dlog、基于RSA的方案)或可轻松映射为整数的群元素(如ECC方案)。因此, 可以直接使用BP累加器。若使用RSA累加器存储凭证, 则需对每个元素单独应用H素数函数(映射至素数), 并需要提供素性ZK证明。Ozdemir等人[39]展示如何通过使用Pocklington证书降低ZK证明素性所需的成本, 但这仅能减少Miller-Rabin素性测试的次数。这仍然会导致验证时间非常数级。这种哈希操作连同素性测试也需包含在ZK证明中。这使得ZK证明的计算复杂度显著增加(例如, 若使用zk-SNARK或其他针对非代数陈述的专用ZK证明[3])。

局限性。虽然BP和RSA累加器都需要经过可信设置阶段来计算公钥参数, 但BP参数的体积相当可观: 2¹⁷个元素需要18 MiB(尽管验证时未必需要)。在我们的ZK场景中, 由于采用了指数知识假设, 参数量甚至会成倍增长(达到3倍)。最后需要指出的是, 与RSA不同, BP累加器在缺乏完整累积集合 X 信息的情况下, 无法实现(非)成员资格的添加或证明生成。

2 相关工作

基于使用陷阱门进行累加器操作, 累加器可以是: 基于陷阱门或无陷阱门。在基于陷阱门

累加器是一个可信实体, 称为累加器管理器, 它持有某些秘密的后门信息, 这使得该实体能够高效地执行累加器操作。另一方面, 一个无后门累加器在没有后门的情况下运行, 如果在设置过程中使用了后门, 它将被销毁。

我们将现有研究分为三大类: (1) 基于哈希函数的累加器, (2) 隐藏顺序组的累加器, 以及 (3) 已知顺序组的累加器。

基于哈希。基于Merkle树[5, 34]或Bloom过滤器[56]构建的累加器不支持批处理、聚合以及批处理(非)成员资格的ZK证明, 除非使用SNARKs等通用工具, 这与本研究提出的技术不同。

隐藏阶群。在单证明场景中, Camenisch和Lysyanskaya[16]基于先前的累加器构造[6, 8], 提出了首个在强RSA假设下安全的动态累加器(支持集合差和集合并运算, 且无需从头重新计算累加器)。然而, 他们的构造并不适用于无陷阱门场景, 因为陷阱门被用于从累加器中删除元素。Li等人[32]他们提出了第一个通用累加器(支持成员和非成员证明), 通过将Camenisch和Lysyanskaya累加器[16]推广到支持强RSA假设下的非成员证明。他们还提供了在累积集合发生变化时更新非成员证明的高效算法。

Boneh等[10]支持在分布式且无后门的环境中对成员资格与非成员资格证明进行批量处理和聚合。他们还利用其贡献在UTXO环境中实现了一个无状态区块链[10, 21]。然而, 与基于BP的构造相比, 基于RSA的构造具有更大的证明规模和验证成本。

已知顺序群。在单证明场景中, Nguyen提出了首个基于双线性映射的累加器[38], 该累加器基于 t 强双线性Diffie-Hellman假设。在后续研究中, Damgard等人[24]以及Au等[4] Nguyen扩展了累加器构造方法, 支持对单个元素进行常规非成员证明。该非成员证明方案依赖多项式余数定理(PRT)来验证非成员关系[55]。然而, 该构造方法既无法扩展到常规规模批量非成员证明, 也未考虑非成员证明的高效聚合问题。在本研究中, 我们重点探讨常规规模批量证明, 并针对基于最大公约数(GCD)的非成员构造提出聚合技术, 而非基于PRT的方案。

Thakur[44]提出了针对BP累加器的批量处理技术(针对弱健壮性且未进行严格分析)。在其最新版本(Sept, 2021)中, 他们还提出了两种PoE协议。然而, 其中一种PoE方法假设可以对任意基数进行指数运算以实现陷阱门(这仅在基于陷阱门的设置中可行)。Connolly等人[23]同时提出一种基于[44]的PoE协议, 该协议采用不同的假设(q-co-GSDH)。与[44]和[23]相比, 我们的PoE在自适应变体q-SDH下具体少了一个指数运算。最后, [44]和[23]均未考虑聚合或ZK批证明。

Prior works [19, 23, 26, 41] define a batch proof for multiple elements. However, these works either: (1) assume in their soundness definition that the accumulator is well-formed and honestly computed or (2) does not consider aggregation [26] or (3) rely on an accumulator manager [49]. On the other hand, our batch proofs are sound even for an adversarially chosen accumulator value and our techniques support aggregation in the trapdoorless setting.

Camenisch et al. [15] and Zhang et al. [57] proposed BP accumulators that are algebraically quite different from [38]. These schemes have parameters that are equal to the size of the accumulator domain or more.

Vector Commitment (VC). A VC is a primitive closely related to accumulators, that provides a succinct commitment and positional binding to an *ordered* set of values [20]. Catalano and Fiore proposed a technique to transform a VC into an accumulator [20]. However, this approach results in an accumulator scheme with public parameters that are equal to the size of the accumulator domain [20, 21, 27, 31, 47].

Tomescu et al. proposed aggregatable sub-vector commitments in the bilinear-map setting based on Lagrange polynomials and KZG commitments [29, 47]. Their work uses the partial fraction decomposition technique [53] to aggregate VC proofs. We adopt techniques to aggregate proofs in BP accumulators. Campanelli et al. [17] defined incremental aggregation, i.e., aggregation of aggregated proofs for a RSA-based VC. Although inspired by [10], their work cannot be efficiently directly applied to accumulators.

Zero-knowledge (ZK). There are known techniques to efficiently achieve ZK for a single element in the BP [4] and the RSA [9, 16] setting. Naively, a batch ZK RSA proof corresponds to proving that the exponent is the product of multiple distinct elements, which results in a linear size proof. Recent work by Campanelli et al. [18] constructs ZK proofs of batch membership for RSA using SNARKs and get a constant size proof. Their construction can be transformed to prove non-membership (it corresponds to membership of intervals) but it cannot support both membership and non-membership at the same time without having an accumulator manager holding the trapdoor.

3 PRELIMINARIES

Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ denote groups of prime order p and let g_1, g_2 be the generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively. Also, let $\mathbb{Z}_p$ be a field of prime order and $\mathbb{Z}_p[x]$ be a polynomial ring. We denote the degree of polynomial $I(x) \in \mathbb{Z}_p[x]$ as $\deg(I)$.

Bilinear pairing. A bilinear pairing is an *efficiently* computable map, $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, satisfying the following properties:

- **bilinearity:** $\forall (P, Q, a, b) \in (\mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{Z}_p \times \mathbb{Z}_p) : e(P^a, Q^b) = e(P^a, Q)^b = e(P, Q^b)^a = e(P, Q)^{ab}$
- **non-degeneracy:** $e(g_1, g_2) \neq 1$

We denote a pairing instance $\text{bp} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2) \leftarrow \text{BilGen}(1^\lambda)$. When $\mathbb{G}_1 = \mathbb{G}_2 = \langle g \rangle$, the pairing is called *symmetric* and is denoted as $(p, \mathbb{G}, \mathbb{G}_T, e, g)$.

Partial fraction decomposition (PFD). A rational polynomial can be decomposed into simpler fractions [53]. Concretely, let $A(x) = \prod_{i \in I} (x + a_i)$ be a polynomial and let $A'(x)$ be the first

derivative of $A(x)$ with respect to x . Then,

$$\frac{1}{A(x)} = \sum_{i \in I} \frac{1}{A'(-a_i)(x + a_i)}.$$

Polynomial remainder theorem (PRT). When a polynomial $A(x)$ is divided by $(x + r)$, the remainder is the evaluation of $A(x)$ at $-r$. Let $q(x)$ denote the quotient polynomial [55]. Then,

$$A(x) = q(x)(x + r) + A(-r).$$

Bézout's theorem (for polynomials). Given $f(x), g(x) \in \mathbb{F}[x]$, there exists $p(x), q(x), h(x) \in \mathbb{F}[x]$ such that [54]:

$$p(x)f(x) + q(x)g(x) = \gcd(f(x), g(x)) = h(x)$$

Moreover, $\deg(p) < \deg(g) - \deg(h)$ and $\deg(q) < \deg(f) - \deg(h)$.

Pedersen vector commitment (PVC). Given a group $\mathbb{G}_1$ or $\mathbb{G}_2$ of prime order p and a vector $\vec{c} = (c_0, \dots, c_t) \in \mathbb{Z}_p^t$. Let $(g_1, g_2, \dots, g_t, h) \in \mathbb{G}_1^{t+1}$ or $\mathbb{G}_2^{t+1}$ generators where $\log_{g_i} g_j, i \neq j$ relationship is unknown. In order to commit to the vector $\vec{c}$, one has to pick $r \leftarrow \mathbb{Z}_p$ and compute $\text{PVC}(\vec{c}, r) = h^r g_0^{c_0} g_1^{c_1} \dots g_t^{c_t}$.

The Pedersen commitment scheme [42] (and its vector generalization) is homomorphic, perfectly hiding and computationally binding under the discrete logarithm assumption.

Zero-knowledge proofs. A ZK proof for a relation $\mathcal{R}(x; w)$, where x is the public statement and w is the witness, is a set of algorithms (Setup, Prove, Verify) with the following syntax:

- **Setup** $(1^\lambda, \mathcal{R}) \rightarrow \text{pp}$: given the security parameter λ and the relation, outputs parameters pp .
- **Prove** $(\text{pp}, x, w) \rightarrow \pi$: given the parameters pp , a statement x and a witness w , it returns a proof π for $\mathcal{R}(x; w)$.
- **Verify** $(\text{pp}, x, \pi) \rightarrow b \in \{0, 1\}$: given the parameters pp , the statement x and the proof π , it accepts or rejects the proof.

Properties. A ZK proof has to be *correct*, *sound* and *zero-knowledge*. Correctness means that if $(x, w) \in \mathcal{R}$, then $\text{Verify}(\text{pp}, x, \pi) = 1$ with overwhelming probability. We support *knowledge soundness* - if a proof passes verification, then there exists a polynomial time algorithm (the extractor) which by interacting with the prover can extract the witness. Finally, zero-knowledge implies that the proof leaks nothing about the witness, i.e., there exists a simulator with access only to the public statement which can output a valid proof.

3.1 Cryptographic accumulators

An accumulator is a cryptographic primitive that supports a succinct binding commitment to an arbitrary set of values. In this work, we consider trapdoorless, dynamic, and universal accumulators. Following the definitions and notation from [5] and [10], our definitions refer to the *batch* setting, where I is a set of elements. The traditional accumulator algorithms can be derived for $I = \{x\}$.

Definition 3.1 (Trapdoorless Accumulator). Let $\mathcal{D}$ be the domain of the accumulated elements, and consider set $X \in \mathcal{D}^{|X|}$ and set $I \in \mathcal{D}^{|I|}$ (where each element from sets X and I is in $\mathcal{D}$). An accumulator consists of the following PPT algorithms:

- (1) $(\text{pp}, \mathcal{D}) \leftarrow \text{Acc.Setup}(1^\lambda)$: Takes security parameter λ , returns the public parameters pp and $\mathcal{D}$ of the accumulated set. A *trusted* entity may use a secret trapdoor to generate pp . The trapdoor

先前的研究[19, 23, 26, 41]定义了针对多个元素的批量证明。然而, 这些研究要么:(1)在其健全性定义中假设累加器是良好形成且诚实计算的, 要么(2)不考虑聚合[26], 要么(3)依赖于累加器管理器[49]。另一方面, 我们的批量证明即使面对对抗性选择的累加器值也是健全的, 且我们的技术在无后门设置中支持聚合。

Camenisch等人[15]以及张等[57]所提出的BP累加器在代数上与[38]存在显著差异。这些方案的参数等于累加器域的大小或更大。

向量承诺(VC)。VC是一种与累加器密切相关的原语, 它为有序值集提供简洁的承诺和位置绑定[20]。Catalano和Fiore提出了一种将VC转换为累加器的技术[20]。然而, 这种方法会导致累加器方案的公共参数等于累加器域的大小[20, 21, 27, 31, 47]。

Tomescu等人在双线性映射场景中提出了基于拉格朗日多项式和KZG承诺的可聚合子向量承诺[29, 47]。他们的工作采用部分分式分解技术[53]来聚合VC证明。我们采用技术在BP累加器中聚合证明。Campanelli等人[17]定义了增量聚合, 即基于RSA的VC的聚合证明的聚合。尽管受到[10]的启发, 但他们的工作无法直接高效地应用于累加器。

零知识(ZK)。已知存在有效实现BP[4]和RSA[9, 16]场景中单个元素ZK的技术。直观上, 批量ZK的RSA证明对应于证明指数是多个不同元素的乘积, 这会导致线性规模证明。Campanelli等人近期的研究[18]构建了使用SNARKs的RSA批量成员证明ZK并获得常规模证明。该构造可转换为证明非成员关系(对应区间成员关系), 但若不通过持有陷阱门的累加器管理器, 则无法同时支持成员与非成员关系。

3 前期研究

设 G_1, G_2, G_T 表示素阶为 p 的群, 令 g_1, g_2 分别为 G_1 和 G_2 的生成元。同时, 设 Z_p 为素阶域, $Z_p[x]$ 为多项式环。我们将多项式 $I(x) \in Z_p[x]$ 的次数记作 $\deg(I)$ 。

双线性配对。双线性配对是一个高效可计算的映射, $e: G_1 \times G_2 \rightarrow G_T$, 满足以下性质:

• **双线性:** $(p, Q, a, b) \in (G_1 \times G_2 \times Z_p \times Z_p) : e(p, a, Q, b) = e(p, a, Q, b) = e(p, Q, b, a) = e(p, Q, a, b)$

• **非简并性:** $e(g_1, g_2) = 1$

我们将配对实例记为 $\text{bp} = (p, G_1, G_2, G_T, e, g_1, g_2) \leftarrow \text{BilGen}(1\varphi)$ 。当 $G_1 = G_2 = \langle g \rangle$ 时, 该配对称为**对称**, 记为 (p, G, G_T, e, g) 。

部分分式分解(PFD)。有理多项式可以分解为更简单的分数[53]。

具体来说, 设 $A(x) = \sum_{i=1}^n a_i x^i$ 为一个多项式, 且 A/x 为第一

A 关于 x 的导数。然后,

$$\frac{1}{A(x)} = \sum_{i \in I} \frac{1}{A/(-a_i)(x+a_i)}.$$

多项式余数定理(PRT)。当多项式 $A(x)$ 除以 $(x+r)$ 时, 余数是 $A(x)$ 在 $-r$ 处的取值。设 $q(x)$ 表示商多项式[55]。那么,

$$A(x) = q(x)(x+r) + A(-r).$$

Bezout定理(关于多项式)。给定 $f(x), g(x) \in F[x]$, 存在 $p(x), q(x), h(x) \in F[x]$ 使得[54]:

$$p(x)f(x) + q(x)g(x) = \gcd(f(x), g(x)) = h(x)$$

此外, $\deg(p) < \deg(g) - \deg(h)$ 且 $\deg(q) < \deg(f) - \deg(h)$ 。

Pedersen向量承诺(聚氯乙炔)。给定一个素数阶 p 的群 G_1 或 G_2 和一个向量 $\vec{c} = (c_0, \dots, c_t) \in \mathbb{Z}_p^t$ 。Let $(g_1, g_2, \dots, g_t, h) \in G_{t+1}^{t+1}$ 或 G_{t+2}^{t+2} 生成元, 其中 $\log_{g_i} g_j, i = j$ 关系未知。为了承诺向量 $\vec{c}$, one has to pick $r \leftarrow \mathbb{Z}_p$ 并计算 $\text{PVC}(\vec{c}, r) = h^r g_0^{c_0} g_1^{c_1} \dots g_t^{c_t}$ 。

Pedersen承诺方案[42] (及其向量推广) 在离散对数假设下具有同态性、完美隐藏性和计算绑定性。

零知识证明。对于关系 $R(x; w)$ 的ZK证明, 其中 x 是公开声明, w 是见证者, 是一组算法(设置、证明、验证)的集合, 具有以下语法:

- 设置 $(1\varphi, R) \rightarrow \text{pp}$: 给定安全参数 φ 和关系, 输出参数 pp 。
- 证明 $(\text{pp}, x, w) \rightarrow \hat{e}$: 给定参数 pp , 一个陈述 x 和一个证人 w , 它返回一个证明 $\hat{e}$ 用于 $R(x; w)$ 。
- 验证 $(\text{pp}, x, \hat{e}) \rightarrow b \in \{0, 1\}$: 给定参数 pp 、陈述 x 和证明 $\hat{e}$, 它接受或拒绝该证明。**属性。**ZK证明必须是**正确、可靠且零知识**的。正确性意味着若 $(x, w) \in R$, 则验证 $(\text{pp}, x, \hat{e}) = 1$ 且概率极高。若证明通过验证, 则存在一个多项式时间算法(提取器), 通过与证明者交互可提取见证者, 我们支持**知识可靠性**。最后, 零知识意味着证明不会泄露任何关于见证者的信息, 即存在一个仅能访问公开陈述的模拟器, 可输出有效证明。

3.1 密码累加器

累加器是一种支持对任意值集进行简洁绑定承诺的密码学原语。在本研究中, 我们考虑无陷阱门、动态且通用的累加器。根据[5]和[10]中的定义与符号, 我们的定义基于**批量**场景, 其中 I 为元素集合。传统累加器算法可推导适用于 $I = \{x\}$ 的情况。

定义3.1 (无陷阱累加器)。设 D 为累积元素的定义域, 考虑集合 $X = D \setminus \{x\}$ 和集合 $I = D \setminus \{i\}$ (其中集合 X 和 I 的每个元素均在 D 中)。累加器由以下ppt算法组成:

- (1) $(\text{pp}, D) \leftarrow \text{Acc.Setup}(1\varphi)$: 接受安全参数 φ , 返回累积集合的公钥 pp 和私钥 D 。**可信**实体可使用秘密后门生成 pp 。后门

is then destroyed by the setup. In the rest of the document, we assume that $\text{pp} = (\text{pp}, \mathcal{D})$.

- (2) $A_X \leftarrow \text{Acc.Commit}_{\text{pp}}(X = \{x_1, \dots, x_{|X|}\})$: Takes set $X \in \mathcal{D}^{|X|}$, outputs the accumulator digest A_X .
- (3) $A'_X \leftarrow \text{Acc.Add}_{\text{pp}}(A_X, X, I)$: Adds set $I \in \mathcal{D}^{|I|}$, $I \cap X = \emptyset$, to the accumulator and returns the new digest A'_X .
- (4) $A'_X \leftarrow \text{Acc.Del}_{\text{pp}}(A_X, X, I)$: Removes set I , $I \subseteq X$, from the accumulator and returns the new accumulator value, A'_X .
- (5) $\pi_I \leftarrow \text{Acc.MemProve}_{\text{pp}}(X, I)$: Generates a membership proof for set I , $I \subseteq X$.
- (6) $\{0, 1\} \leftarrow \text{Acc.MemVerify}_{\text{pp}}(A_X, I, \pi_I)$: Returns 1 if the membership proof π_I , for the set I , $I \subseteq X$, is valid against the accumulator digest, A_X .
- (7) $\bar{\pi}_I \leftarrow \text{Acc.NonMemProve}_{\text{pp}}(X, I)$: Generates non-membership proof for the set I , $I \cap X = \emptyset$, disjoint from the accumulated set.
- (8) $\{0, 1\} \leftarrow \text{Acc.NonMemVerify}_{\text{pp}}(A_X, I, \bar{\pi}_I)$: Returns 1 if the non-membership proof $\bar{\pi}_I$, of the set I , $I \cap X = \emptyset$, is valid against the accumulator digest, A_X .
- (9) $\pi_I \leftarrow \text{Acc.AggMem}_{\text{pp}}(A_X, I, \{\pi_1, \dots, \pi_{|I|}\})$: Combines individual membership proofs $\{\pi_1, \dots, \pi_{|I|}\}$ into a single aggregated proof.
- (10) $\bar{\pi}_I \leftarrow \text{Acc.AggNonMem}_{\text{pp}}(A_X, I, \{\bar{\pi}_1, \dots, \bar{\pi}_{|I|}\})$: Combines individual non-membership proofs $\{\bar{\pi}_1, \dots, \bar{\pi}_{|I|}\}$ into a single aggregated proof.

The following algorithms update a (non-)membership proof of a single element after changes to the accumulated set.

- (11) $\pi'_y \leftarrow \text{Acc.MemProofUpdOnAdd}_{\text{pp}}(A_X, X, y, \pi_y, I)$: Updates the membership proof π_y of element y on addition of set I ($y \notin I$, $X = X \cup I$) to the accumulator.
- (12) $\pi'_y \leftarrow \text{Acc.MemProofUpdOnDel}_{\text{pp}}(A_X, A'_X, X, I, y, \pi_y)$: Updates the membership proof π_y of element y on deletion of set I , $X = X \setminus I$, from the accumulator.
- (13) $\bar{\pi}'_y \leftarrow \text{Acc.NonMemProofUpdOnAdd}_{\text{pp}}(A_X, X, y, \bar{\pi}_y, I)$: Updates the non-membership proof $\bar{\pi}_y$ of element y disjoint from set X on addition of set I ($y \notin I$, $X = X \cup I$) to the accumulator.
- (14) $\bar{\pi}'_y \leftarrow \text{Acc.NonMemProofUpdOnDel}_{\text{pp}}(A_X, A'_X, X, I, y, \bar{\pi}_y)$: Updates the non-membership proof $\bar{\pi}_y$ of an element y disjoint from set X on deletion of set I ($X = X \setminus I$) from the accumulator.

3.2 Correctness and soundness

The basic security property for accumulators is *soundness* (sometimes called *undeniability* [33]) which states that an adversary *cannot* construct an accumulator A and a set I for which both π_I and $\bar{\pi}_I$ are simultaneously valid. Below we state *strong soundness* (also found in [10]), which allows the adversary to create the accumulator *without* revealing the accumulated set X to the challenger.

Definition 3.2 (Soundness). For any PPT adversary $\mathcal{A}$, it holds:

$$\Pr \left[\begin{array}{c} \text{pp} \leftarrow \text{Acc.Setup}(1^\lambda) \\ (A, I, \pi_I, \bar{\pi}_I) \leftarrow \mathcal{A}(\text{pp}) \\ \text{Acc.MemVerify}_{\text{pp}}(A, I, \pi_I) = 1 \wedge \\ \text{Acc.NonMemVerify}_{\text{pp}}(A, I, \bar{\pi}_I) = 1 \end{array} \right] = \text{negl}(\lambda)$$

Since no trapdoor is needed for algorithms $\text{Acc.Add}_{\text{pp}}$, $\text{Acc.Del}_{\text{pp}}$, $\text{Acc.MemProve}_{\text{pp}}$, $\text{Acc.NonMemProve}_{\text{pp}}$, $\mathcal{A}$ can adaptively update the accumulator and construct honest proofs during the game before coming up with the accumulator value A and proofs $\pi_I, \bar{\pi}_I$.

We formally define correctness of (non-)membership proofs, the correctness of aggregation, and present the accumulator soundness proof under a weak definition in the extended version of our paper. This weak soundness definition assumes that the accumulator value is honestly generated for the set X . Our construction is secure under this weak definition of soundness using the t -SBDH assumption. The weak soundness definition is useful if one wants to avoid the Uber assumption [7] (under which our protocols satisfy strong soundness).

3.3 Accumulator based on bilinear-maps

In this subsection, we review the standard accumulator based on bilinear-pairing (BP) introduced by Nguyen [38] and improved by [24, 40]. Here, we present the classic BP construction that operates on *individual* proofs. In §4 and §5 we show how to *batch* and *aggregate* proofs in the BP setting, and in §6 we present a technique to efficiently update non-membership proofs.

Let X be the accumulator set and $X(s) = \prod_{x \in X} (s + x)$ be the accumulator polynomial.

$\text{pp} \leftarrow \text{Acc.Setup}(1^\lambda)$: Let $\text{bp} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$ be the output of $\text{BilGen}(1^\lambda)$. Assume that s is a trapdoor randomly sampled from $\mathbb{Z}_p^*$. The public parameters are $\text{pp} = (\text{bp}, \{g_1^s, g_2^s \mid 0 \leq i \leq t\}, \mathcal{D} = \mathbb{Z}_p)$ where t is the maximum capacity of the accumulator. $A_X \leftarrow \text{Acc.Commit}_{\text{pp}}(X = \{x_1, \dots, x_{|X|}\})$: The accumulator di-

gest of the set $X = \{x_1, \dots, x_n\}$, is $A_X = g_1^{X(s)}$, where the polynomial $X(s) = \prod_{x \in X} (s + x)$.

$A'_X \leftarrow \text{Acc.Add}_{\text{pp}}(A_X, X, I)$: A set of elements I , $I \cap X = \emptyset$, can be added to the accumulator and the digest can be updated as:

$$A'_X = g_1^{\prod_{x \in X \cup I} (s+x)} = A_X^{\prod_{x_i \in I} (s+x_i)}$$

$A'_X \leftarrow \text{Acc.Del}_{\text{pp}}(A_X, X, I)$: A set of elements $I \subseteq X$ can be deleted from the accumulator and the digest can be updated as: $A'_X = g_1^{\prod_{x \in X \setminus I} (s+x)} = A_X^{1/\prod_{x_i \in I} (s+x_i)}$.

$\pi_y \leftarrow \text{Acc.MemProve}_{\text{pp}}(X, y)$: The proof of membership of an element y can be computed as $\pi_y = g_1^{\prod_{x \in X \setminus \{y\}} (s+x)}$, where X is the accumulated set.

$\{0, 1\} \leftarrow \text{Acc.MemVerify}_{\text{pp}}(A_X, y, \pi_y)$: The membership proof of an element $y \in X$ in the accumulator can be verified by performing the following pairing check: $e(\pi_y, g_2^y g_2^s) = e(A_X, g_2)$.

$\bar{\pi}_y = (\alpha, g_1^{\beta(s)}) \leftarrow \text{Acc.NonMemProve}_{\text{pp}}(X, y)$: The non-membership proof of $\{y\} \cap X = \emptyset$ involves computing the Bézout coefficients $\alpha(s)$ and $\beta(s)$ s.t. $\alpha(s) \cdot X(s) + \beta(s) \cdot (s + y) = 1$. As the monomial $(s + y)$ is of degree one, the polynomial $\alpha(s)$ is in fact a *constant*! Thus, the $\bar{\pi}_y = (\alpha, g_1^{\beta(s)}) \in (\mathbb{Z}_p, \mathbb{G}_1)$ is the non-membership proof of element y .

$\{0, 1\} \leftarrow \text{Acc.NonMemVerify}_{\text{pp}}(A_X, y, \bar{\pi}_y)$: The non-membership proof can be verified by checking $e(A_X, g_2^\alpha) \cdot e(g_1^{\beta(s)}, g_2^s g_2^y) = e(g_1, g_2)$, where $\bar{\pi}_y = (\alpha, g_1^{\beta(s)})$.

$\pi'_y \leftarrow \text{Acc.MemProofUpdOnAdd}_{\text{pp}}(A_X, X, y, \pi_y, I)$: The membership proof of an element π_y , can be updated on addition of set I , to the accumulated set, $X = X \cup I$, by computing, $\pi'_y = \pi_y^{\prod_{z \in I} (s+z)}$. When $I = \{z\}$, $\pi'_y = A_X \cdot \pi_y^{s+z}$ is the updated proof.

随后被设置破坏。在文档的其余部分, 我们假设 $\text{pp} = (\text{pp}, \text{D})$ 。

(2) $Ax \leftarrow \text{Acc} \cdot \text{Commit}_{\text{pp}}(X = \{x_1, \dots, x_{|X|}\})$: 接受集合 X $\text{D}[x]$, 输出累加器摘要 Ax 。

(3) $A/x \leftarrow \text{Acc} \cdot \text{Add}_{\text{pp}}(Ax, X, I)$: 将集合 $I \subseteq [1, |X|]$, $I \cap X = \emptyset$ 添加到累加器中, 并返回新的摘要 A/x 。

(4) $A/x \leftarrow \text{Acc} \cdot \text{Del}_{\text{pp}}(Ax, X, I)$: 从累加器中移除集合 I , $I \subseteq [1, |X|]$, 并返回新的累加器值 A/x 。

(5) $\hat{a} \leftarrow \text{Acc} \cdot \text{MemProve}_{\text{pp}}(X, I)$: 为集合 I 生成成员证明, 其中 $I \subseteq X$ 。

(6) $\{0, 1\} \leftarrow \text{Acc} \cdot \text{MemVerify}_{\text{pp}}(Ax, I, \hat{a})$: 若集合 I 的成员证明 $I \subseteq X$ 对应的成员证明 $\hat{a}$ 对累加器摘要 Ax 有效, 则返回1。

(7) $\hat{a} \leftarrow \text{Acc} \cdot \text{NonMemProve}_{\text{pp}}(X, I)$: 为集合 I 生成非成员证明, $I \cap X = \emptyset$, 且与累积集合不相交。

(8) $\{0, 1\} \leftarrow \text{Acc} \cdot \text{NonMemVerify}_{\text{pp}}(Ax, I, \hat{a})$: 若集合 I 的非成员证明 $\hat{a}$, 且 $I \cap X = \emptyset$, 则对累加器摘要 Ax 有效, 返回1。

(9) $\hat{a} \leftarrow \text{Acc} \cdot \text{AggMem}_{\text{pp}}(Ax, I, \{\hat{e}_1, \dots, \hat{e}_{|I|}\})$: 将个体成员资格证明 $\{\hat{e}_1, \dots, \hat{e}_{|I|}\}$ 聚合为单一聚合证明。

(10) $\hat{e} \leftarrow \text{Acc} \cdot \text{AggNonMem}_{\text{pp}}(\overline{Ax}, \overline{I}, \{\hat{e}_1, \dots, \hat{e}_{|I|}\})$: 将个体非成员证明 $\{\hat{e}_1, \dots, \hat{e}_{|I|}\}$ 融合为单一聚合证明。

以下算法在累积集合发生变更后, 对单个元素的 (非) 成员资格证明进行更新。

(11) $\hat{e}/y \leftarrow \text{Acc} \cdot \text{MemProofUpdOnAdd}_{\text{pp}}(Ax, X, y, \hat{e}_y, I)$: 在集合 I ($y \in I, X = X \cup I$) 添加到累加器时, 更新元素 y 的成员证明 $\hat{e}_y$ 。

(12) $\hat{e}/y \leftarrow \text{Acc} \cdot \text{MemProofUpdOnDel}_{\text{pp}}(Ax, A/x, X, I, y, \hat{e}_y)$: 在集合 I , $X = X \setminus I$ 从累加器中删除时, 更新元素 y 的成员证明 $\hat{e}_y$ 。

(13) $\hat{e}/y \leftarrow \text{Acc} \cdot \text{NonMemProofUpdOnAdd}_{\text{pp}}(Ax, \overline{X}, y, \hat{e}_y, I)$: 更新元素 y 的非成员证明 $\hat{e}_y$, 该元素在集合 $\overline{X}$ 上与集合 I ($y \in I, X = X \cup I$) 相交后被添加到累加器中。

(14) $\hat{e}/y \leftarrow \text{Acc} \cdot \text{NonMemProofUpdOnDel}_{\text{pp}}(Ax, \overline{A/x}, X, I, y, \hat{e}_y)$: 在集合 I ($X = X \setminus I$) 从累加器中删除时, 更新元素 y 与集合 X 互不相交的非成员证明 $\hat{e}_y$ 。

3.2 正确性和健全性

累加器的基本安全属性是**健全性** (有时称为**不可否认性** [33]), 其表明攻击者无法构造一个累加器 A 和一个集合 I , 使得 $\hat{a}$ 和 $\hat{e}/I$ 同时有效。下文我们将阐述**强健性** (亦见于 [10]), 该属性允许攻击者创建累加器而不向挑战者泄露累积集合 X 。

定义 3.2 (健全性)。对于任何 ppt 对手 A , 都成立:

$$\Pr_{\text{pp}} \left[\begin{array}{l} \text{pp} \leftarrow \text{Acc} \cdot \text{Setup}(1^\rho)(A, I, \hat{a}), \\ \hat{a} \leftarrow A(\text{pp}) \cdot \text{Acc} \cdot \text{MemVerify}_{\text{pp}}(A, I, \hat{a}) = 1, \\ \text{Acc} \cdot \text{NonMemVerify}_{\text{pp}}(\overline{A}, \overline{I}, \hat{e}/I) = 1 \end{array} \right] \leq \text{negl}(\rho)$$

由于算法 $\text{Acc} \cdot \text{Add}_{\text{pp}}, \text{Acc} \cdot \text{Del}_{\text{pp}}, \text{Acc} \cdot \text{MemProve}_{\text{pp}}, \text{Acc} \cdot$

$\text{NonMemProve}_{\text{pp}}$ 不需要陷阱门, A 可以在游戏过程中自适应地更新累加器并构建诚实证明, 然后才得出累加器值 A 和证明 $\hat{a}, \hat{e}/I$ 。

在本文扩展版本中, 我们正式定义了 (非) 成员证明的正确性、聚合操作的正确性, 并基于弱定义提出了累加器健全性证明。该弱健全性定义假设累加器值是为集合 X 诚实生成的。通过采用 t -SBDH 假设, 我们的构造在该弱健全性定义下保持安全。若需避免Uber假设[7] (该假设要求我们的协议满足强健性), 则弱健全性定义具有重要价值。

3.3 基于双线性映射的累加器

在本小节中, 我们回顾了 Nguyen [38] 提出的基于双线性配对 (BP) 的标准累加器, 并由 [24, 40] 改进。本文提出经典 BP 构造, 其作用于单个证明。在第4节和第5节中, 我们展示了如何在 BP 框架下对证明进行**批量处理**和**聚合**; 在第6节中, 我们提出了一种高效更新非隶属证明的技术。

设 X 为累加器集, $X(s) = \prod_{x \in X} (s+x)$ 为累加器多项式。

$\text{pp} \leftarrow \text{Acc} \cdot \text{Setup}(1^\rho)$: 设 $\text{bp} = (p, G_1, G_2, GT, e, g_1, g_2)$ 为 $\text{BilGen}(1^\rho)$ 的输出。假设 s 是从 $\mathbb{Z}_p$ 中随机采样的陷阱门。公共参数为 $\text{pp} = (\text{bp}, \{g_1^i, g_2^i \mid 0 \leq i \leq t\}, \text{D}/\mathbb{Z}_p)$, 其中 t 是累加器的最大容量。 $Ax \leftarrow \text{Acc} \cdot \text{Exec}_{\text{pp}}(X = \{x_1, \dots, x_{|X|}\})$: 累加器 di -

集合 $X = \{x_1, \dots, x_n\}$ 的生成元是 $Ax = g_1^{X(s)}$, 其中多聚-nomial $X(s) = \prod_{x \in X} (s+x)$ 。

$A/x \leftarrow \text{Acc} \cdot \text{Add}_{\text{pp}}(Ax, X, I)$: 一组元素 I , $I \cap X = \emptyset$, 可添加至累

加器, 且摘要可更新为: $A/x = g_1^{-1 \cdot \sum_{x \in I} (s+x)} = A \cdot g_1^{-\sum_{x \in I} (s+x)}$ 。

$A/x \leftarrow \text{Acc} \cdot \text{Del}_{\text{pp}}(Ax, X, I)$: 从累加器中删除元素集 $I \subseteq X$, 并更新摘要为: $A/x = g_1^{-1 \cdot \sum_{x \in I} (s+x)} = A \cdot g_1^{\sum_{x \in I} (s+x)}$ 。

$\hat{e}_y \leftarrow \text{Acc} \cdot \text{MemProve}_{\text{pp}}(X, y)$: 元素 y 的成员证明可计算为

$\hat{e}_y = g_1^{-1 \cdot \sum_{x \in X} (s+x)}$, 其中 X 是累积集。

$\{0, 1\} \leftarrow \text{Acc} \cdot \text{MemVerify}_{\text{pp}}(Ax, y, \hat{e}_y)$: 可通过执行以下配对检查来验证累加器中元素 $y \in X$ 的成员证明: $e(\hat{e}_y, g_2^{g_2}) = e(Ax, g_2)$ 。

$\hat{e}_y = (\zeta, g_1^{\zeta(s)}) \leftarrow \text{Acc} \cdot \text{NonMemProve}_{\text{pp}}(X, y): \{y\} \cap X = \emptyset$ 的非成员证明涉及计算贝祖系数 $\zeta(s)$ 和 $\zeta(s)$, 使得 $\zeta(s) \cdot X(s) + \zeta(s) \cdot (s+y) = 1$ 。由于单项式 $(s+y)$ 的次数为一, 多项式 $\zeta(s)$ 实际上是一个常数! 因此, $\hat{e}_y = (\zeta, g_1^{\zeta(s)}) \in (\mathbb{Z}_p, \mathbb{G}_1)$ 是元素 y 的非成员证明。

$\{0, 1\} \leftarrow \text{Acc} \cdot \text{NonMemVerify}_{\text{pp}}(Ax, y, \hat{e}_y)$: 非成员证明可通过检查 $e(Ax, g_2^{\zeta(s)}) \cdot e(g_1^{\zeta(s)}, g_2^{g_2}) = e(g_1, g_2)$ 来验证, 其中 $\hat{e}_y = (\zeta, g_1^{\zeta(s)})$ 。

$\hat{e}/y \leftarrow \text{Acc} \cdot \text{MemProofUpdOnAdd}_{\text{pp}}(Ax, X, y, \hat{e}_y, I)$: 成员-元素 $\hat{e}_y$ 的船形证明, 可通过计算 $\hat{e}/y = \hat{e}_y \cdot g_1^{\sum_{x \in I} (s+x)}$, 在集合 I 添加到累积集合 $X = X \cup I$ 时进行更新。当 $I = \{z\}$ 时, $\hat{e}/y = Ax \cdot \hat{e}_y^{-y}$ 即为更新后的证明。

$\pi'_y \leftarrow \text{Acc.MemProofUpdOnDel}_{\text{pp}}(A_X, A'_X, X, I, y, \pi_y)$: The membership proof π_y can be updated on deletion of set I from the accumulated set, $X = X \setminus I$, by computing, $\pi'_y = \pi_y^{\frac{1}{\prod_{z \in I}(s+z)}}$. When $I = \{z\}$, $\pi'_y = (\frac{\pi_y}{\pi_z})^{\frac{1}{z-y}}$ is the updated proof.

Non-membership in BP accumulator. There are two ways to prove non-membership in the BP accumulator: (1) based on the PRT [4, 24] and (2) based on GCD [40]. The two methods are equivalent, in that they both require $O(|X|)$ field operations. In this work, we focus on the GCD-based method because it allows for efficient batching and aggregation as we show in §4.2 and §5.2. Additionally, we present a new algorithm to update individual non-membership proofs §6.

BP accumulator soundness. The soundness of the BP construction relies on the t -sBDH assumption.

4 BATCHING BP ACCUMULATOR PROOFS

4.1 Membership

Recall that the membership proof of a single element $y \in X$ is defined as $\pi_y = g_1^{\prod_{x \in X \setminus y}(s+x)}$ and the membership proof of y can be verified by checking $e(\pi_y, g_2^{(s+y)}) = e(A_X, g_2)$, where A_X is the digest of the accumulator.

Batch proof. A batch membership proof is a single succinct proof for a set of elements in the accumulator. A batch proof contains a polynomial in the exponent that includes every element in the accumulator except the monomials terms that correspond to values in the set I .

$\pi_I \leftarrow \text{Acc.MemProve}_{\text{pp}}(X, I)$: A batch membership proof for set $I \subseteq X$ is computed as follows: $\pi_I = g_1^{\prod_{x_i \in X \setminus I}(s+x_i)}$
 $\{0, 1\} \leftarrow \text{Acc.MemVerify}_{\text{pp}}(A_X, I, \pi_I)$: The batch proof can be verified by checking $e(\pi_I, g_2^{\prod_{x_i \in I}(s+x_i)}) = e(A_X, g_2)$.

Asymptotics. The batch membership proof for I consists only of one element in $\mathbb{G}_1$ (as opposed to $|I| \cdot \mathbb{G}_1$ elements, if proved individually). Verifying a batch proof takes $O(|I| \log^2 |I|)$ field operations to compute the coefficients of polynomial $I(s) = \prod_{x_i \in I}(s + x_i)$ using FFT, one $|I|$ -sized multi-exponentiation in $\mathbb{G}_2$ to compute $g_2^{I(s)}$ and 2 pairings. However, verifying $|I|$ individual proofs requires $|I|$ individual exponentiations (cannot use fast multi-exponentiations), and $2|I|$ pairings.

4.2 Non-membership

A non-membership proof of an element $y \notin X$ leverages the fact that $\gcd(X(s), (s+y)) = 1$. Thus the Bézout coefficients α and $\beta(s)$ satisfy: $\alpha X(s) + \beta(s)(s+y) = 1$. We generalize this observation to prove the non-membership of set $I \cap X = \emptyset$.

Batch proof. We define a batch non-membership proof as a single succinct proof for a set of elements disjoint from the accumulator. Observe that there are no common roots between polynomials $X(s)$ and $I(s) = \prod_{y_i \in I}(s + y_i)$, therefore $\gcd(X(s), I(s)) = 1$. Moreover, the Bézout coefficients $\alpha(s)$ and $\beta(s)$ of $X(s), I(s)$ are non-constant polynomials.

$\bar{\pi}_I \leftarrow \text{Acc.NonMemProve}_{\text{pp}}(X, I)$: A batch non-membership proof for a set I , where $I \cap X = \emptyset$, is $\bar{\pi}_I = (g_2^{\alpha(s)}, g_1^{\beta(s)}) \in (\mathbb{G}_2, \mathbb{G}_1)$ such that $\alpha(s) \cdot X(s) + \beta(s) \cdot \prod_{y_i \in I}(s + y_i) = 1$.
 $\{0, 1\} \leftarrow \text{Acc.NonMemVerify}_{\text{pp}}(A_X, I, \bar{\pi}_I)$: The batch proof can be verified by checking $e(A_X, g_2^{\alpha(s)}) \cdot e(g_1^{\beta(s)}, g_2^{\prod_{y_i \in I}(s+y_i)}) = e(g_1, g_2)$, where $\bar{\pi}_I = (g_2^{\alpha(s)}, g_1^{\beta(s)})$.

Asymptotics. The batch non-membership proof consists of only one element from $\mathbb{G}_1$ and one element from $\mathbb{G}_2$. Verifying a batch proof takes $O(|I| \log^2 |I|)$ field operations to compute the coefficients of polynomial $I(s)$ using FFT, one $|I|$ -sized multi-exponentiation in $\mathbb{G}_2$ to compute $g_2^{I(s)}$ and 2 pairings. However, verifying $|I|$ individual proofs requires $|I|$ individual exponentiations (cannot use fast multi-exponentiations) and $2|I|$ pairings. If $e(g_1, g_2)$ is not precomputed, then batch verification and individual verification requires 3 pairings and $2|I| + 1$ pairings, respectively.

4.3 Soundness of batching

THEOREM 4.1. *Batch membership and non-membership proofs $\pi_I, \bar{\pi}_I$, are sound, by Definition 3.2, under the adaptive Uber assumption in the AGM.*

PROOF. Deferred to the extended version of our paper. $\square$

5 AGGREGATION

5.1 Membership

The BP accumulator based on Nguyen et al. resembles the KZG polynomial commitments [29, 38]. As demonstrated in prior works [14, 25, 47], KZG polynomial commitments can be aggregated using the PFD [53]. We use these techniques to aggregate proofs in the BP accumulator.

$\pi_I \leftarrow \text{Acc.AggMem}_{\text{pp}}(A_X, I, \{\pi_1, \dots, \pi_{|I|}\})$: Let $I \subseteq X$ be the set of elements to be aggregated, $X(s) = \prod_{x \in X}(s + x)$ be the accumulator polynomial of X , $I(s) = \prod_{x_i \in I}(s + x_i)$ be the accumulator polynomial of I , and $X_i(s) = \prod_{x \in X \setminus \{x_i\}}(s + x)$. The goal is to compute the polynomial $Y(s) = \prod_{x_i \in X \setminus I}(s + x_i)$, which excludes all the monomials that correspond to values in the set I . Using PFD [53]:

$$Y(s) = X(s) \cdot \frac{1}{I(s)} = X(s) \sum_{x_i \in I} \frac{1}{I'(-x_i)(s + x_i)} = \sum_{x_i \in I} \frac{1}{I'(-x_i)} \cdot X_i(s)$$

Thus, the aggregated proof, $\pi_I = g_1^{Y(s)} = \prod_{x_i \in I} \pi_i^{c_i}$, where $c_i = \frac{1}{I'(-x_i)}$, I' is the first derivative of I w.r.t s , and π_i is the individual membership proof of element i .

Correctness and soundness. We can see that as long as each individual π_i 's are correct, π_I satisfies *aggregation correctness*. Note that Thm. 4.1, is sufficient to argue the soundness of aggregation (Definition 3.2) since the resulting proof after aggregation is a batch proof and thus can be verified by running `MemVerify`.

Asymptotics. In a field with sufficiently many roots of unity, FFT based polynomial interpolation and multi-point evaluation techniques can be used to compute the polynomial $I(s)$ and evaluate $I'(s)$ at all x_i 's in $O(|I| \log^2 |I|)$ field operations. Asymptotically, it takes $O(|I| \log^2 |I|)$ operations in $\mathbb{Z}_p$ (to compute all the c_i 's) and $|I|$ sized multi-exponentiation in $\mathbb{G}_1$ to compute the aggregated proof.

$\hat{e}_y \leftarrow \text{Acc} \cdot \text{MemProofUpOnDel}_{\text{pp}}(Ax, A/x, x, I, y, \hat{e}_y)$: The mem-
当集合 I 从 ac 中被删除时, $\hat{e}_y$ 的保真度可被更新。
累积集, $x = x \setminus I$, 通过计算, $\hat{e}_y = \hat{e}_{n y \cdot I(s+z)}$ 。当 $I = \{z\}$
时, $\hat{e}_y = (\frac{\pi_y}{\pi_z})^{\frac{1}{z-y}}$ is the updated proof.

BP累加器中的非成员证明。 证明非成员身份有两种方法: (1) 基于PRT[4, 24]; (2) 基于 GCD [40]。这两种方法等价, 均需 $O(|x|)$ 次字段操作。本研究聚焦 GCD 方法, 因其支持高效批量处理与聚合 (详见§4.2和§5.2)。此外, 我们提出新算法用于更新个体非成员证明§6。

BP累加器的稳健性。 BP结构的稳健性依赖于 t -sBDH假设。

4 批次血压累积器校验

4.1 成员资格

回忆一下, 单个元素 $y \in x$ 的成员证明定义为 $\hat{e}_y = g^{n1^{x \cdot y}(s+y)}$, 而 y 的成员证明可以通过检查 $e(\hat{e}_y, g_2(s+y)) = e(Ax, g_2)$ 来验证, 其中 Ax 是累加器的摘要。

批量证明。 批量证明是针对累加器中一组元素的单一简洁证明。批量证明包含一个指数多项式, 该多项式包含累加器中的每个元素, 除了与集合 I 中值相对应的单项式项。

$\hat{e} \leftarrow \text{Acc} \cdot \text{MemProve}_{\text{pp}}(x, I)$: 集合 $I \subseteq x$ 的批量成员证明计算如下:
 $\pi_I = g^{n1^{x \cdot I}(s+xi)}$
 $\{0, 1\} \leftarrow \text{Acc} \cdot \text{MemVerify}_{\text{pp}}(Ax, I, \hat{e})$: 批量证明可通过检查 $e(\hat{e}, g^{n2^{xi \cdot I}(s+xi)}) = e(Ax, g_2)$ 来验证。

渐近性。 I 的批量成员证明仅包含 G_1 中的一个元素 (与单独证明时需要 $|I| \cdot G_1$ 个元素的情况不同)。验证批量证明需要 $O(|I| \log^2 |I|)$ 次域运算来计算多项式 $I(s) = \sum_{xi} I(s+xi)$ 的系数, 其中使用 FFT 进行一次 $|I|$ 规模的 G_2 多次指数运算来计算 $g^{I_2(s)}$ 和 2 次配对。然而, 验证 $|I|$ 个单独证明需要 $|I|$ 次单独指数运算 (无法使用快速多次指数运算), 以及 $2|I|$ 次配对。

4.2 非会员

一个元素 $y \in x$ 的非成员证明利用了 $\gcd(x(s), (s+y)) = 1$ 的事实。因此贝祖系数 $\hat{c}$ 和 $\hat{c}(s)$ 满足: $\hat{c}(s) + \hat{c}(s)(s+y) = 1$ 。我们将这一观察推广以证明集合 $I \cap x = \emptyset$ 的非成员性。

批量证明。 我们将批量非成员证明定义为针对与累加器不相交的元素集合的单一简洁证明。注意到多项式 $x(s)$ 与 $I(s) = \prod_{yi} I(s+yi)$ 之间不存在公根, 因此 $\gcd(x(s), I(s)) = 1$ 。此外, $x(s), I(s)$ 的贝祖系数 $\hat{c}(s)$ 和 $\hat{c}(s)$ 是非常数多项式。

$\hat{e} \leftarrow \text{Acc} \cdot \text{NonMemProve}_{\text{pp}}(x, I)$: 为集合 I (其中 $I \cap x = \emptyset$) 设计的批量非成员证明, 其满足 $\hat{e} = (g_2^{c_2(s)}, g_1^{c_1(s)})$ (G_2, G_1), 且 $\hat{c}(s) \cdot x(s) + \hat{c}(s) \cdot \prod_{yi} I(s+yi) = 1$ 。
 $\{0, 1\} \leftarrow \text{Acc} \cdot \text{NonMemVerify}_{\text{pp}}(Ax, I, \hat{e})$: 批量证明可通过检查 $e(Ax, g_2^{c_2(s)}) \cdot e(g_1^{c_1(s)}, g^{n2^{yi \cdot I(s+yi)}}) = e(g_1, g_2)$ 来验证, 其中 $\hat{e} = (g_2^{c_2(s)}, g_1^{c_1(s)})$ 。

渐近性。 批量非成员证明仅包含来自 G_1 的一个元素和来自 G_2 的一个元素。验证一个批次证明需要 $O(|I| \log^2 |I|)$ 次域运算, 通过 FFT 计算多项式 $I(s)$ 的系数, 一次 $|I|$ 规模的 G_2 中多指数运算来计算 $g^{I_2(s)}$ 以及 2 次配对。然而, 验证 $|I|$ 个单独证明需要 $|I|$ 次单独指数运算 (无法使用快速多指数运算) 和 $2|I|$ 次配对。如果 $e(g_1, g_2)$ 未预先计算, 则批次验证和单独验证分别需要 3 次配对和 $2|I|+1$ 次配对。

4.3 配料的完整性

定理 4.1 $\hat{e}_I, \hat{e}_I$ 的批量成员与非成员证明, 根据定义 3.2, 在 AGM 的自适应 Uber 假设下是有效的。

证明。延后至本文扩展版本。 $\square$

5 聚合

5.1 成员资格

基于 Nguyen 等人的 BP 累加器类似于 KZG 多项式承诺 [29, 38]。如先前工作 [14, 25, 47] 所示, KZG 多项式承诺可通过 PFD [53] 进行聚合。我们使用这些技术在 BP 累加器中聚合证明。

$\hat{e} \leftarrow \text{Acc} \cdot \text{AggMem}_{\text{pp}}(Ax, I, \{\hat{e}_1, \dots, \hat{e}_{|I|}\})$: 设 $I \subseteq x$ 为待聚合元素集, $x(s) = \sum_{xi} x(s+xi)$ 为 x 的累加多项式, $I(s) = \sum_{xi} I(s+xi)$ 为 I 的累加多项式, 且 $xi(s) = \sum_{xi} x(s+xi)$ 。目标是计算多项式 $Y(s) = \sum_{xi} x(s+xi)$, 该多项式排除了所有对应于集合 I 中的单项式。使用 PFD [53]:

$$Y(s) = X(s) \cdot \frac{1}{I(s)} = x(s) \sum_{xi \in I} \frac{1}{I'(-xi) \cdot (s+xi)} = \sum_{xi \in I} \frac{1}{I'(-xi)} \cdot \frac{x(s)}{s+xi}$$

因此, 聚合证明, $\pi_I = g^{1^{Y(s)}} = \sum_{xi} I'(-xi) \cdot \hat{e}_i$, 其中 $ci = \frac{1}{I'(-xi)}$ I' 是 I 关于 s 的一阶导数, 而 $\hat{e}_i$ 是元素 i 的个体成员证明。

正确性与健全性。 可以看出, 只要每个个体 $\hat{e}_i$ 的结果正确, $\hat{e}$ 就满足聚合正确性。注意定理 4.1 已足以论证聚合的健全性 (定义 3.2), 因为聚合后的证明结果是一个批量证明, 因此可以通过运行 MemVerify 进行验证。

渐近性。 在具有足够多单位根的域中, 可以使用基于 FFT 的多项式插值和多点评估技术来计算多项式 $I(s)$, 并在所有 xi 处评估 I/s , 所需操作量为 $O(|I| \log^2 |I|)$ 。渐近性地, 计算所有 ci 需要 $O(|I| \log^2 |I|)$ 次操作, 而在 G_1 中计算聚合证明则需要 $|I|$ 次多重指数运算。

The complexity of verifying an aggregated proof is the same as verifying a batch proof. Note that verification requires a single multi-exponentiation of size $|I|$ in $\mathbb{G}_2$ to compute $g_2^{I(s)}$. However, using the PoE protocol, we can outsource the exponentiation cost to an untrusted prover. We discuss this optimization in §8.

5.2 Non-membership

In this section, we give an algorithm to combine multiple individual non-membership proofs into a single non-membership proof.

$\bar{\pi}_I \leftarrow \text{Acc.AggregNonMem}_{\text{pp}}(A_X, I, \{\bar{\pi}_1, \dots, \bar{\pi}_{|I|}\})$: Let I be a set of

elements disjoint from the accumulated set X , $\bar{\pi}_i = (\alpha_i, g_1^{\beta_i(s)}) \in \mathbb{Z}_p \times \mathbb{G}_1$ be the non-membership proof of the element $y_i \in I$, $X(s) = \prod_{x \in X} (s + x)$ be the accumulator polynomial, $I(s) = \prod_{y_i \in I} (s + y_i)$ be the accumulator polynomial of I , and $Y_i(s) = \frac{I(s)}{(s + y_i)}$.

Observe that $\gcd(Y_1(s), Y_2(s), \dots, Y_{|I|}(s)) = 1$. By generalization of Bézout's identity for polynomials, $\exists$ polynomials $c_i(s)$, s.t.:

$$\sum_{i=1}^{|I|} c_i(s) \cdot Y_i(s) = \gcd(Y_1(s), Y_2(s), \dots, Y_{|I|}(s)) = 1 \quad (1)$$

LEMMA 5.1. *The Bézout coefficient polynomials, $c_i(s)$, of Eq. 1 are non-zero constants.*

PROOF. Deferred to the extended version of our paper. $\square$

To compute the non-membership proof of I , we need to compute polynomials $\alpha(s)$ and $\beta(s)$ such that:

$$\alpha(s) \cdot X(s) + \beta(s) \cdot \prod_{y_i \in I} (s + y_i) = \gcd(X(s), \prod_{y_i \in I} (s + y_i)) = 1$$

Recall that each individual non-membership proof satisfies:

$$\alpha_i \cdot X(s) + \beta_i(s) \cdot (s + y_i) = 1, \text{ where } \deg(\alpha_i) = 0$$

Multiplying by $c_i \cdot Y_i(s)$: $\alpha_i c_i Y_i(s) X(s) + c_i \beta_i(s) (s + y_i) Y_i(s) = c_i Y_i(s)$
 $\Rightarrow \alpha_i c_i Y_i(s) X(s) + c_i \beta_i(s) I(s) = c_i Y_i(s)$

Summing over $i = 1$ to $|I|$: $\sum_{i=1}^{|I|} \alpha_i c_i Y_i(s) X(s) + \sum_{i=1}^{|I|} c_i \beta_i(s) I(s) = \sum_{i=1}^{|I|} c_i \cdot Y_i(s)$

$$\text{Using Eq. 1: } \left(\sum_{i=1}^{|I|} \alpha_i c_i Y_i(s) \right) \cdot X(s) + \left(\sum_{i=1}^{|I|} c_i \beta_i(s) \right) \cdot I(s) = 1$$

Therefore,

$$\alpha(s) = \sum_{i=1}^{|I|} \alpha_i c_i Y_i(s) \quad \beta(s) = \sum_{i=1}^{|I|} c_i \beta_i(s) \quad (2)$$

Thus the non-membership of set I is $\bar{\pi}_I = (g_2^{\alpha(s)}, g_1^{\beta(s)})$

Correctness and soundness. We can see that as long as each individual $\bar{\pi}_i$'s are correct, $\bar{\pi}_I$ satisfies *aggregation correctness*. The algorithm AggNonMem always outputs $\bar{\pi}_I$ because terms c_i do not depend on trapdoor s (Lemma 5.1), therefore exponentiation by c_i is feasible. We remark that Thm. 4.1, is sufficient to argue the soundness of aggregation (Definition 3.2) since the resulting proof after aggregation is a batch proof and thus can be verified by running NonMemVerify .

Asymptotics. The task of computing the aggregated proof can be divided into: (1) task of computing the Bézout coefficients c_i 's,

Scheme	RSA	BP
Domain	Prime	$\mathbb{Z}_p$
Setup(1^λ)	$O(1)$	$O(t) \cdot (\mathbb{G}_1 + \mathbb{G}_2)$
Commit	$O(X) \cdot \mathbb{G}$	$O(X) \cdot \mathbb{G}_1 + O(X \log^2 X) \cdot \mathbb{Z}_p$
AggMem	$O(I \log I) \cdot \mathbb{G} + O(I \log I) \cdot \mathbb{F}$	$O(X) \cdot \mathbb{G}_1 + O(X \log^2 I) \cdot \mathbb{Z}_p$
MemVerify	$O(I) \cdot \mathbb{G}$	$2\mathbb{P} + O(I) \cdot \mathbb{G}_2 + O(I \log^2 I) \cdot \mathbb{Z}_p$
MemVerifyPoE	$O(1) \cdot \mathbb{G} + O(I) \cdot \mathbb{F}$	$3\mathbb{P} + 1\mathbb{G}_1 + 1\mathbb{G}_2 + O(I \log^2 I) \cdot \mathbb{Z}_p$
AggNonMem	$O(I \log I) \cdot \mathbb{G} + O(I \log I) \cdot \mathbb{F}$	$O(I) \cdot (\mathbb{G}_1 + \mathbb{G}_2) + O(I ^2 \log I) \cdot \mathbb{Z}_p$
NonMemVerify	$O(I) \cdot \mathbb{G}$	$2\mathbb{P} + O(I) \cdot \mathbb{G}_2 + O(I \log^2 I) \cdot \mathbb{Z}_p$
NonMemVerifyPoE	$O(1) \cdot \mathbb{G} + O(I) \cdot \mathbb{F}$	$5\mathbb{P} + 1\mathbb{G}_1 + 1\mathbb{G}_2 + O(I \log^2 I) \cdot \mathbb{Z}_p$

Table 1: Set X denotes the entire accumulated set and $I \subseteq X$. Let $O(Y) \cdot \mathbb{G}$ denotes one large exponentiation to the product of Y elements or Y exponentiations. All exponentiations in BP can be sped up by a logarithmic factor using multi-exponentiations.

(2) task of computing the coefficients of each $Y_i(s)$, and (3) task of computing $(g_2^{\alpha(s)}, g_1^{\beta(s)})$.

First, the task of computing the Bézout coefficients in Eq. 1 takes $O(|I| \log^2 |I|)$ field operations. Second, the task of computing all Y_i 's take $O(|I|^2 \log |I|)$ field operations. This is because computing each $Y_i(s)$ costs $O(|I| \log |I|)$ field operations. Third, the task of computing $g_2^{\sum_{i=1}^{|I|} \alpha_i c_i Y_i(s)}$ and $g_1^{\sum_{i=1}^{|I|} c_i \beta_i(s)}$ requires a single multi-exponentiation of size $|I|$ in $\mathbb{G}_2$ and $\mathbb{G}_1$, respectively. Thus, in total, it takes $O(|I|^2 \log |I|)$ field operations and multi-exponentiations of size $O(|I|)$ in $\mathbb{G}_1, \mathbb{G}_2$ to compute the aggregated non-membership proof from individual proofs.

The complexity of verifying an aggregated proof is the same as verifying a batch proof. Note that verification requires a single multi-exponentiation of size $|I|$ in $\mathbb{G}_2$ to compute $g_2^{I(s)}$. However, using the PoE protocol, we can outsource the exponentiation cost to an untrusted prover. We discuss this optimization in §8.

6 NON-MEMBERSHIP PROOF UPDATES

In this section, we describe a new protocol that allows to efficiently update a GCD-based non-membership proof for the BP accumulator after changes (additions/deletions) to the accumulated set. Let $y \notin X$ and $\bar{\pi}_y = (\alpha, g_1^{\beta(s)}) \in \mathbb{Z}_p \times \mathbb{G}_1$ s.t. $\alpha X(s) + \beta(s)(s + y) = 1$. $\bar{\pi}'_y \leftarrow \text{Acc.NonMemProofUpdOnAdd}_{\text{pp}}(A_X, X, y, \bar{\pi}_y, I = \{z\})$: Recall that,

$$\alpha X(s) + \beta(s)(s + y) = 1 \quad (3)$$

Since $y \neq z$, by Bézout's Identity,

$$u(s + z) + v(s + y) = 1, \text{ where } u, v \in \mathbb{Z}_p \quad (4)$$

Goal is to find α' and $\beta'(s)$ s.t.,

$$\alpha' X'(s) + \beta'(s)(s + y) = 1, \text{ where } X'(s) = X(s)(s + z) \quad (5)$$

验证聚合证明的复杂度与验证批量证明相同。需要注意的是, 验证需要在 G_2 中进行一次大小为 $|I|$ 的单次多重指数运算来计算 $g^{2 \sum_{i=1}^{|I|} \alpha_i(s)}$ 。然而, 通过使用PoE协议, 我们可以将指数运算成本外包给不可信的证明者。我们将在§8中讨论这一优化。

5.2 非会员

本节提出一种算法, 可将多个独立的非成员证明整合为单一的非成员证明。

$\pi \leftarrow \text{Acc. AggNonMem}_{pp}(Ax, I, \{\pi_1, \dots, \pi_{|I|}\})$: 设 I 为一个集合与累积集 X 不相交的元素 $\pi_i = (\alpha_i, g^{\beta_i}(s)) \in Z_p \times G_1$ 为元素 $y_i \in I, X$ 的非隶属证明, $s = \prod_{x \in X} (s+x)$ 为累加多项式, $I(s) = \prod_{y_i \in I} (s+y_i)$ 为 I 的累加多项式, 且 $y_i(s) = \frac{I(s)}{(s+y_i)}$ 。

观察到 $\gcd(y_1(s), y_2(s), \dots, y_{|I|}(s)) = 1$ 。通过多项式贝祖恒等式的推广, 存在多项式 $c_i(s)$, 满足:

$$\sum_{i=1}^{|I|} c_i(s) \cdot y_i(s) = \text{最大公约数}(y_1(s), y_2(s), \dots, y_{|I|}(s)) = 1 \quad (1)$$

引理 5.1. 等式 1 的贝祖系数多项式 $c_i(s)$ 是非零常数。

证明。延后至本文扩展版本。 $\square$

为计算 I 的非成员证明, 我们需要计算多项式 $\alpha(s)$ 和 $\beta(s)$, 使得:

$$\alpha(s) \cdot X(s) + \beta(s) \cdot \prod_{y_i \in I} (s+y_i) = \gcd(X(s), \prod_{y_i \in I} (s+y_i)) = 1$$

需注意, 每个独立的非成员证明均满足:

$$\begin{aligned} \alpha_i X(s) + \beta_i(s) \cdot (s+y_i) &= 1, \text{ 其中 } \deg(\alpha_i) = 0 \text{ 乘以 } c_i \cdot y_i(s) : \alpha_i c_i y_i(s) X(s) + c_i \beta_i(s) (s+y_i) y_i(s) = c_i y_i(s) \\ \alpha_i c_i y_i(s) X(s) + c_i \beta_i(s) I(s) &= c_i y_i(s) \\ \text{对 } i=1 \text{ 到 } |I| \text{ 求和: } \sum_{i=1}^{|I|} \alpha_i c_i y_i(s) X(s) + \sum_{i=1}^{|I|} c_i \beta_i(s) I(s) &= \sum_{i=1}^{|I|} c_i \cdot Y_i(s) \end{aligned}$$

使用等式 1: $(\sum_{i=1}^{|I|} \alpha_i c_i(s) \cdot X(s) + (\sum_{i=1}^{|I|} c_i \beta_i(s)) \cdot I(s)) = 1$ 因此,

$$\alpha(s) = \sum_{i=1}^{|I|} \alpha_i c_i Y_i(s) \quad \beta(s) = \sum_{i=1}^{|I|} c_i \beta_i(s) \quad (2) \text{ 因此集合 } I \text{ 的非}$$

成员性为 $\pi_I = (g^{2 \sum_{i=1}^{|I|} \alpha_i(s)}, g^{\sum_{i=1}^{|I|} \beta_i(s)})$ **正确性与健全性**。可以看出, 只要每个个体 π_i 都正确, π_I 就满足聚合正确性。

算法AggNonMem始终输出 π_I , 因为项 c_i 不依赖于陷阱门 s (引理5.1), 因此通过 c_i 进行指数运算是可行的。我们注意到定理4.1足以论证聚合的正确性 (定义3.2), 因为聚合后的证明结果是一个批量证明, 因此可以通过运行NonMemVerify进行验证。

渐近性。计算聚合证明的任务可划分为: (1) 计算贝祖系数 c_i 的任务,

方案	RSA	BP
领域	主要的	Z_p
设置 (1.2)	$O(1)$	$O(t) \cdot (G_1 + G_2)$
提交	$O(X) \cdot G$	$O(x) \cdot G_1 + O(x \log^2 x) \cdot Z_p$
聚集膜	$O(I \log I) \cdot G + O(I \log I) \cdot F$	$O(I) \cdot G_1 + O(I \log^2 I) \cdot Z_p$
内存验证	$O(I) \cdot G$	$2P + O(I) \cdot G_2 + O(I \log^2 I) \cdot Z_p$
存储器验证PoE	$O(1) \cdot G + O(I) \cdot F$	$3P + 1G_1 + 1G_2 + O(I \log^2 I) \cdot Z_p$
非膜性聚集	$O(I \log I) \cdot G + O(I \log I) \cdot F$	$O(I) \cdot (G_1 + G_2) + O(I ^2 \log I) \cdot Z_p$
非内存验证	$O(I) \cdot G$	$2P + O(I) \cdot G_2 + O(I \log^2 I) \cdot Z_p$
非内存验证PoE	$O(1) \cdot G + O(I) \cdot F$	$5P + 1G_1 + 1G_2 + O(I \log^2 I) \cdot Z_p$

表1: 集合 x 表示整个累积集合, 且 $I \cap x = \emptyset$ 。令 $O(y) \cdot G$ 表示对 Y 个元素乘积或 Y 次幂的单次大指数运算。BP中的所有指数运算均可通过多指数运算以对数因子加速。

(2) 计算每个 $y_i(s)$ 的系数的任务, 以及(3)计算 $(g^{2 \sum_{i=1}^{|I|} \alpha_i(s)}, g^{\sum_{i=1}^{|I|} \beta_i(s)})$ 的任务。

首先, 计算等式1中Bezout系数的任务需要 $O(|I| \log^2 |I|)$ 次域运算。其次, 计算所有 y_i 的任务需要 $O(|I|^2 \log |I|)$ 次域运算。这是因为计算每个 $y_i(s)$ 需要 $O(|I| \log |I|)$ 次域运算。第三, 计算 $g^{2 \sum_{i=1}^{|I|} \alpha_i(s)}$ 和 $g^{\sum_{i=1}^{|I|} \beta_i(s)}$ 分别需要在 G_2 和 G_1 中进行一次大小为 $|I|$ 的多重指数运算。因此, 总共需要 $O(|I|^2 \log |I|)$ 次域运算和大小为 $O(|I|)$ 的多重指数运算, 分别在 G_1, G_2 中完成, 以从单个证明中计算出聚合的非成员证明。

验证聚合证明的复杂度与验证批量证明相同。需要注意的是, 验证需要在 G_2 中进行一次大小为 $|I|$ 的单次多重指数运算来计算 $g^{2 \sum_{i=1}^{|I|} \alpha_i(s)}$ 。然而, 通过使用PoE协议, 我们可以将指数运算成本外包给不可信的证明者。我们将在§8中讨论这一优化。

6 非会员资格证明更新

在本节中, 我们描述了一种新协议, 该协议允许在对累积集进行更改 (添加/删除) 后, 高效地更新基于GCD的非成员证明。设 $y \in X$ 且 $\pi_y = (\alpha, g^{\beta}(s)) \in Z_p \times G_1$ 且满足 $\alpha X(s) + \beta(s) \cdot (s+y) = 1$ 。 $\pi / y \leftarrow \text{Acc. NonMemProofUpdOnAdd}_{pp}(Ax, X, y, \pi_y, I = \{z\})$: Re-

称其为

$$\alpha X(s) + \beta(s)(s+y) = 1 \quad (3)$$

由于 $y=z$, 根据贝祖恒等式,

$$u(s+z) + U(s+y) = 1, \text{ 其中 } u, U \in Z_p \quad (4)$$

目标是找到 α' 和 $\beta'(s)$ s.t.,

$$\alpha' X(s) + \beta'(s) \cdot (s+y) = 1, \text{ 其中 } X(s) = X(s) \cdot (s+z) \quad (5)$$

Multiplying Eq. 3 by $u(s+z)$: $u\alpha X(s)(s+z) + u\beta(s)(s+z)(s+y) = u(s+z)$.

Using Eq. 4:

$$\begin{aligned} u\alpha X(s)(s+z) + u\beta(s)(s+z)(s+y) &= 1 - v(s+y) \\ (u\alpha)X'(s) + (v + u\beta(s)(s+z))(s+y) &= 1 \end{aligned}$$

From Eq. 5 we have: $\alpha' = u\alpha$ and $\beta'(s) = v + u\beta(s)(s+z)$

However, it is not possible to compute $g_1^{\beta'(s)}$ without the coefficients of $\beta(s)$ because individual proofs only contain $g^{\beta(s)}$. Thus, we simplify $\beta'(s)' = v + u\beta(s)(s+z)$, replace $u(s+z)$ as $1 - v(s+y)$ from Eq. 4: $\beta'(s) = v + \beta(s)(1 - v(s+y)) = v + \beta(s) - v\beta(s)(s+y) = v(1 - \beta(s)(s+y)) + \beta(s)$. Replace $1 - \beta(s)(s+y)$ as $\alpha X(s)$ from Eq. 3: $\beta'(s) = v\alpha X(s) + \beta(s)$.

Thus, $\alpha' = u\alpha$, $\beta'(s) = v\alpha X(s) + \beta(s)$ and $\bar{\pi}'_y = (\alpha', g_1^{\beta'(s)}) = (u\alpha, A_X^{v\alpha} \cdot g_1^{\beta(s)})$.

To compute the updated non-membership proof, $\bar{\pi}'_y = (\alpha', g_1^{\beta'(s)})$, we calculate constants u, v in Eq. 4 by running the extended Euclidean algorithm for degree one polynomials, which takes only $O(1)$ operations. We can compute the constant α' with one multiplication operation in $\mathbb{Z}_p$. Similarly, we can compute $g_1^{\beta'(s)} = ((A_X)^\alpha)^v \cdot g_1^{\beta(s)}$ with one multiplication operation in $\mathbb{Z}_p$, one exponentiation operation in $\mathbb{G}_1$ by $\mathbb{Z}_p$, and one addition in $\mathbb{G}_1$.

$\bar{\pi}'_y \leftarrow \text{Acc.NonMemProofUpdOnDel}_{\text{pp}}(A_X, A'_X, X, I = \{z\}, y, \bar{\pi}_y)$:

Recall that, $\alpha X(s) + \beta(s)(s+y) = 1$. Since $y \neq z$, by Bézout's Identity,

$$u(s+z) + v(s+y) = 1, \text{ where } u, v \in \mathbb{Z}_p \quad (6)$$

Goal is to find α' and $\beta'(s)$ s.t.,

$$\alpha'X'(s) + \beta'(s)(s+y) = 1, \text{ where } X'(s) = \frac{X(s)}{(s+z)} \quad (7)$$

$$\alpha X(s) + \beta(s)(s+y) = 1 \Rightarrow \alpha X'(s)(s+z) + \beta(s)(s+y) = 1$$

Replace $(s+z)$ as $\frac{1-v(s+y)}{u}$ from Eq. 6: $\alpha X'(s) \left(\frac{1-v(s+y)}{u} \right) + \beta(s)(s+y) = 1$

$$y) = 1 \Rightarrow \frac{\alpha}{u}X'(s) + \left(\beta(s) - \frac{v\alpha}{u}X'(s) \right)(s+y) = 1$$

Thus, from Eq. 7, $\alpha' = \frac{\alpha}{u}$, $\beta'(s) = \beta(s) - v\alpha'X'(s)$ and $\bar{\pi}'_y = (\alpha', g_1^{\beta'(s)}) = \left(\frac{\alpha}{u}, \frac{g_1^{\beta(s)}}{A_X^{v\alpha'}} \right)$.

To compute the updated non-membership proof, $\bar{\pi}'_y = (\alpha', g_1^{\beta'(s)})$, we calculate constants u, v in Eq. 6 by running the extended Euclidean algorithm for degree one polynomials, which takes only $O(1)$ operations. We can compute the constant α' with one inversion operation and one multiplication operation in $\mathbb{Z}_p$. Similarly, we can compute $g_1^{\beta'(s)} = \frac{g_1^{\beta(s)}}{A_X^{v\alpha'}}$ with one multiplication operation in $\mathbb{Z}_p$, one inversion operation, one exponentiation operation in $\mathbb{G}_1$ by $\mathbb{Z}_p$, and one addition operation in $\mathbb{G}_1$.

7 ZK BATCH PROOFS

We now show how our batch (non-)membership proofs in BP accumulators can be made zero-knowledge. We consider the following setting: a set $X = \{x_1, \dots, x_n\}$ of elements is accumulated, and a prover holds witnesses for a subset $I \subseteq X$, where $|I| = d$. We want to prove that $I \subseteq X$ (or $I \cap X = \emptyset$) while *hiding the set I itself*. Additionally, we want to be able to reveal just the size of I or in

other words that it includes “at least d elements”. This is important since typically a batch proof does not hide the number of batched elements and this is useful in identity systems, sanctions/embargoed lists, e-cash, etc. At the same time, we want to maintain the benefits of batch proofs: (1) the verifier cost should be constant for both membership and non-membership proofs, and (2) the size of the ZK batch proof should remain sublinear.

In the ZK setting, the verifier does not hold the set I or the proof π_I (or $\bar{\pi}_I$) in order to run the verification algorithm. Instead, the prover has to prove in ZK that the pairing equations in the verification algorithm hold. The prover's witness is the set I , the randomness r used in the commitment to I , a proof of membership π_I or a proof of non-membership $\bar{\pi}_I$. The inputs known to the verifier are the public parameters pp , a commitment to the subset C_I , and the accumulator value A_X .

More specifically, the prover (that knows the polynomial $I(x)$ and the commitment randomness r) has to compute ZK proofs for the following relations:

- $\mathcal{R}_{\text{mem}}(\text{pp}, C_I, A_X; \pi_I, r)$ knowledge of π_I such that membership verification holds and knowledge of randomness r
- $\mathcal{R}_{\text{nonmem}}(\text{pp}, C_I, A_X; \bar{\pi}_I, r)$ knowledge of $\bar{\pi}_I$ such that non-membership verification holds and knowledge of randomness r
- If revealing the size, $\mathcal{R}_{\text{degcheck}}(\text{pp}, C_I, d; I, r)$ proves that the set size $|I| = d$ and the set corresponds to C_I . A proof for this relation is analyzed in the following tasks:
 - Proving that $I(x) = \prod_{i=1}^d (x+x_i) = x^d + f(x)$, where $\deg(f) \leq d-1$ using a hiding commitment C_f .
 - Proving well-formedness of C_f in relation to C_I .

The commitment C_I allows the prover to later open all elements in I (if needed) by revealing r . In the following paragraphs, we explain in details how these proofs are constructed.

Notation. Let pp be the following public parameters:

- $g_1, h_1, g, g_1, g_2 \in \mathbb{G}_1$ and $g_2, h_2, h, h_1, h_2 \in \mathbb{G}_2$
- $\mathbf{g}_1^s = [g_1, g_1^s, \dots, g_1^{s^t}] \in \mathbb{G}_1^{t+1}$, $\mathbf{g}_2^s = [g_2, g_2^s, \dots, g_2^{s^t}] \in \mathbb{G}_2^{t+1}$
- $\mathbf{a} = [g_2^a, g_2^{as}, \dots, g_2^{as^t}] \in \mathbb{G}_2^{t+1}$

We view a polynomial $I(x)$ as equivalent to its coefficients that form a vector, therefore for the rest of the paper, we overload the Pedersen VC input with both vectors and polynomials.

7.1 Proving membership ($\mathcal{R}_{\text{mem}}$)

The high level idea of the initial proof is the following: $\mathcal{R}_{\text{mem.init}}$ essentially proves that the verification equation holds for $C_I h_2^{-r}$ and proves knowledge of r .

$$\mathcal{R}_{\text{mem.init}} = \left\{ \begin{array}{l} (\text{pp}, C_I \in \mathbb{G}_2, A_X \in \mathbb{G}_1; \pi_I \in \mathbb{G}_1, r \in \mathbb{Z}_p) : \\ C_I = h_2^r g_2^{I(s)} \wedge e(\pi_I, C_I) \cdot e(\pi_I, h_2)^{-r} = e(A_X, g_2) \end{array} \right\}$$

However, it still does not hide the witness π_I . Using techniques from RingCT [43], $\mathcal{R}_{\text{mem.init}}$ can be transformed into $\mathcal{R}_{\text{mem}}$, which proves the above statement in ZK. Specifically, it adds a proof that $\mathcal{R}_{\text{mem.init}}$ verifies for $\pi_{I,2} g^{-\tau_1}$ and proves knowledge of τ_1 , where $\pi_{I,2}$ is the blinded version of the batch proof π_I .

乘以等式3乘以 $u(s+z): u\zeta X(s)(s+z) + u\zeta'(s)(s+y) = u(s+z)$ 。

使用等式4:

$$\begin{aligned} u\alpha X(s)(s+z) + u\beta(s)(s+z)(s+y) &= 1 - v(s+y) \\ (u\alpha)X'(s) + (v + u\beta(s)(s+z))(s+y) &= 1 \end{aligned}$$

从等式5我们得到: $\zeta' = u\zeta$ 且 $\zeta'(s) = U + u\zeta(s)$ 。然而, 若没有 $\zeta(s)$ 的系数, 就无法计算 $g_1\zeta'(s)$ 。因为单独的证明仅包含 $g\zeta(s)$ 。因此, 我们简化 $\zeta'(s) = U + u\zeta(s)$, 并将 $u(s+z)$ 替换为 $1 - U(s+y)$, 来自 term_0 4: $\zeta'(s) = U + \zeta(s)(1 - U(s+y)) = U + \zeta(s) - U\zeta(s)(s+y) = U(1 - \zeta(s)(s+y)) + \zeta(s)$ 。将 $1 - \zeta(s)(s+y)$ 替换为 $\zeta X(s)$ 从

Eq. 3: $\beta'(s) = v\alpha X(s) + \beta(s)$ 。

因此, $\zeta' = u\zeta$, $\zeta'(s) = U\zeta X(s) + \zeta(s)$ 且 $\beta' = (\alpha', g_1\beta'(s)) = (u\zeta, A_X^{v\alpha} \cdot g_1\zeta(s))$ 。

为了计算更新的非成员证明, $\beta' = (\zeta', g_1\zeta'(s))$, 我们通过运行针对一次多项式的扩展欧几里得算法来计算等式4中的常数 u, U , 该算法仅需 $O(1)$ 次操作。我们可以在 Z_p 中用一次乘法运算计算常数 ζ' 。类似地, 我们可以在 Z_p 中用一次乘法运算、在 G_1 中用一次指数运算、在 G_p 中用一次加法运算, 计算 $g_1\zeta'(s) = (Ax)\zeta' \cdot g_1\zeta(s)$ 。

$\beta' \leftarrow \text{Acc. NonMemProofUpdOnDel}_{pp}(Ax, A'x, X, I = \{z\}, y, \beta_y) :$

回忆一下, $\zeta X(s) + \zeta'(s)(s+y) = 1$ 。由于 $y=z$, 根据贝祖恒等式,

$$u(s+z) + U(s+y) = 1, \text{ 其中 } u, U \in Z_p \quad (6)$$

目标是找到 ζ' 和 $\zeta'(s)$ 使得,

$$\zeta'X'(s) + \zeta'(s)(s+y) = 1, \text{ 其中 } X'(s) = \frac{X(s)}{(s+z)} \quad (7)$$

$$\alpha X(s) + \beta(s)(s+y) = 1 \Rightarrow \alpha X'(s)(s+z) + \beta(s)(s+y) = 1$$

将 $(s+z)$ 替换为 $\frac{1-v(s+y)}{u}$ 来自等式6: $\zeta X'(s)\left(\frac{1-v(s+y)}{u}\right) + \beta(s)(s+y) = 1 \Rightarrow \frac{X(s)}{u}(s+y) + \beta(s) = \frac{1-v(s+y)}{u}$

$(s) - \frac{v}{u}X'(s)(s+y) = 1$ 因此, 从等式7, $\zeta' = \frac{\alpha}{u}$, $\zeta'(s) = \zeta(s) - U\zeta'X'(s)$

$$(s) \text{ 且 } \beta' = (\zeta', g_1\zeta'(s)) = \left(\frac{\alpha}{u}, g_1\frac{\beta(s)}{A_X'}\right).$$

为了计算更新的非成员证明, $\beta' = (\zeta', g_1\zeta'(s))$, 我们通过运行针对一次多项式的扩展欧几里得算法来计算等式6中的常数 u, U , 该算法仅需 $O(1)$ 次操作。我们可以在 Z_p 中通过一次求逆操作和一次乘法操作计算常数 ζ' 。类似地, 我们可以计算 $g_1\zeta'(s) = g_1\frac{\beta(s)}{A_X'}(s)$ 在 Z_p 中包含一个乘法运算, 在 G_1 中包含一个由 Z_p 对应的逆运算和一个指数运算, 以及在 G_1 中包含一个加法运算。

7 ZK 批量证明

我们现在展示如何使BP累加器中的批处理(非)成员证明成为零知识证明。我们考虑以下场景: 一个元素集合 $X = \{x_1, \dots, x_n\}$ 被累加, 且证明者持有子集 $I \subseteq X$ 的见证, 其中 $|I| = d$ 。我们希望证明 $I \subseteq X$ (或 $I \cap X = \emptyset$), 同时隐藏集合 I 本身。此外, 我们希望能够仅揭示 I 的大小或在

换句话说, 它包含“至少 d 个元素”。这一点很重要, 因为通常批量证明不会隐藏批量元素的数量, 这在身份系统、制裁/禁运名单、电子现金等领域非常有用。同时, 我们希望保持批量证明的优势: (1)验证者成本对于成员和非成员证明都应保持恒定; (2)ZK 批量证明的规模应保持亚线性。

在ZK场景中, 验证者并不持有集合 I 或证明 $\hat{e}_I$ (或 $\hat{e}$)来运行验证算法。相反, 证明者必须在ZK中证明验证算法中的配对方程成立。证明者的见证包括集合 I 、用于向 I 提交承诺的随机性 r 、成员证明 $\hat{e}_I$ 或非成员证明 $\hat{e}_I$ 。验证者已知的输入包括公共参数 pp 、对子集 CI 的承诺以及累加器值 Ax 。

更具体地说, 证明者(已知多项式 $I(x)$ 和承诺随机性 r)

必须计算以下关系的ZK证明:

- $R_{\text{mem}}(pp, CI, Ax, \hat{e}_I, r)$ 对 $\hat{e}_I$ 的知识, 使得成员验证成立且对随机性 r 的知识
 - $R_{\text{nonmem}}(pp, CI, Ax; \hat{e}_I, r)$ 对 $\hat{e}_I$ 的知识, 使得非成员验证成立且对随机性 r 的知识
 - 若揭示集合大小, $R_{\text{degcheck}}(pp, CI, d; I, r)$ 证明集合大小 $|I| = d$ 且该集合对应于 CI 。关于此关系的证明将在以下任务中进行分析:
 - 证明 $I(x) = \prod_{i=1}^d (x + x_i) = xd + f(x)$, 其中 $\deg(f) \leq d-1$, 使用隐藏承诺 Cf ,
 - 证明 Cf 相对于 CI 的正确性。承诺 CI 允许证明者后续开放所有元素
- 在 I (如有需要) 中通过揭示 r 。在接下来的段落中, 我们将详细解释这些证明是如何构建的。

符号说明。 设 pp 为以下公共参数:

- $g_1, h_1, g, g_1, g_2 \in G_1$ 且 $g_2, h_2, 5, 5_1, 5_2 \in G_2$
- $g_1^s = [g_1, g_1^s, \dots, g_1^{s^t}] \in \mathbb{G}_1^{t+1}$, $g_2^s = [g_2, g_2^s, \dots, g_2^{s^t}] \in \mathbb{G}_2^{t+1}$
- $a = [g_2, g \text{ 作为 } 2, \dots, g \text{ 作为 } 2^t] \in \mathbb{G}_2^{t+1}$

我们将多项式 $I(x)$ 视为与其系数构成的向量等价, 因此在本文后续部分, 我们将Pedersen VC输入同时用向量和多项式进行重载。

7.1 证明成员资格 (R_{mem})

初始证明的高层次思想如下: $R_{\text{mem.init}}$ 本质上证明了验证方程对 $CI \cap I = \emptyset$ 成立, 并证明了对 r 的知晓。

$$R_{\text{mem.init}} = \left\{ \begin{array}{l} (pp, CI \in G_2, Ax \in G_1; \hat{e}_I \in G_1, r \in Z_p) : \\ CI \cap I = \emptyset \text{ 且 } e(\hat{e}_I, CI) \cdot e(\hat{e}_I, h_2)^{-r} = e(Ax, g_2) \end{array} \right\}$$

然而, 它仍然无法隐藏见证者 $\hat{e}_I$ 。利用RingCT[43]的技术, $R_{\text{mem.init}}$ 可以转换为 R_{mem} , 这证明了ZK中的上述陈述。具体而言, 它增加了 $R_{\text{mem.init}}$ 验证 $\hat{e}_I, 2g^{-\hat{e}_I}$ 的证明, 并证明了对 g_1 的知识, 其中 $\hat{e}_1, 2$ 是批处理证明 $\hat{e}_I$ 的盲化版本。

$$\mathcal{R}_{\text{mem}} = \left\{ \begin{array}{l} (\text{pp}, C_I \in \mathbb{G}_2, A_X \in \mathbb{G}_1; \pi_I \in \mathbb{G}_1, r, \tau_1, \tau_2 \in \mathbb{Z}_p) : \\ C_I = h_2^r g_2^{I(s)} \wedge \delta_1 = \tau_1 r \wedge \delta_2 = \tau_2 r \wedge \\ \pi_{I,1} = g_1^{\tau_1} g^{\tau_2} \wedge \pi_{I,2} = \pi_I g^{\tau_1} \wedge \\ \frac{e(\pi_{I,2}, C_I)}{e(A_X, g_2)} = e(g, C_I)^{\tau_1} \cdot e(g, h_2)^{-\delta_1} \cdot e(\pi_{I,2}, h_2)^r \end{array} \right\}$$

$\mathcal{R}_{\text{mem}}$ is instantiated with a generalized version of Schnorr's protocol for knowledge of DL as r, τ_1 are scalars.

Protocol for relation $\mathcal{R}_{\text{mem}}$. The interactive version of the protocol is as follows:

- Prover
 - Picks $\tau_1, \tau_2 \leftarrow \mathbb{Z}_p$
 - Computes $\pi_{I,1} = g_1^{\tau_1} g^{\tau_2}$ and $\pi_{I,2} = \pi_I g^{\tau_1}$
 - Picks $r_r, r_{\tau_1}, r_{\tau_2}, r_{\delta_1}, r_{\delta_2} \in \mathbb{Z}_p$
 - Sends:
 - * $\pi_{I,1}, \pi_{I,2}$
 - * $R_1 = g_1^{r_{\tau_1}} g^{r_{\tau_2}}, R_2 = \pi_{I,1}^{r_r} g_1^{-r_{\delta_1}} g^{-r_{\delta_2}}$
 - * $R_3 = e(g, C_I)^{r_{\tau_1}} e(g, h_2)^{-r_{\delta_1}} e(\pi_{I,2}, h_2)^{r_r}$
- Verifier sends $c \leftarrow \mathbb{Z}_p$
- Prover sends:
 - $s_r = r_r + cr$
 - $s_{\tau_1} = r_{\tau_1} + c\tau_1, s_{\tau_2} = r_{\tau_2} + c\tau_2$
 - $s_{\delta_1} = r_{\delta_1} + c\delta_1, s_{\delta_2} = r_{\delta_2} + c\delta_2$
- Verifier checks:
 - $R_1 = \pi_{I,1}^{-c} g_1^{s_{\tau_1}} g^{s_{\tau_2}}$
 - $R_2 = \pi_{I,1}^{s_r} g_1^{-s_{\delta_1}} g^{-s_{\delta_2}}$
 - $R_3 \cdot \left(\frac{e(\pi_{I,2}, C_I)}{e(A_X, g_2)} \right)^c = e(g, C_I)^{s_{\tau_1}} e(g, h_2)^{-s_{\delta_1}} e(\pi_{I,2}, h_2)^{s_r}$

Correctness, soundness, and ZK. We defer the security arguments to the extended version of our paper.

Asymptotics. The instantiation of $\mathcal{R}_{\text{mem}}$ consists of 5 group elements and 5 field elements, and has constant prover and verifier.

7.2 Proving non-membership ($\mathcal{R}_{\text{nonmem}}$)

For non-membership, $\mathcal{R}_{\text{mem}, \text{init}}$ is replaced by

$$\mathcal{R}_{\text{nonmem}, \text{init}} = \left\{ \begin{array}{l} (\text{pp}, C_I \in \mathbb{G}_2, A_X \in \mathbb{G}_1; \\ (\bar{A}, \bar{B}) \in \mathbb{G}_2 \times \mathbb{G}_1, r \in \mathbb{Z}_p) : \\ C_I = h_2^r g_2^{I(s)} \wedge \\ e(A_X, \bar{A}) \cdot e(\bar{B}, C_I) \cdot e(\bar{B}, h)^{-r} = e(g_1, g_2) \end{array} \right\}$$

Similarly, $\mathcal{R}_{\text{nonmem}, \text{init}}$ still does not hide the witness π_I . Thus, using techniques from RingCT [43], $\mathcal{R}_{\text{nonmem}, \text{init}}$ can be transformed into $\mathcal{R}_{\text{nonmem}}$, which proves the above statement in ZK.

$$\mathcal{R}_{\text{nonmem}} = \left\{ \begin{array}{l} (\text{pp}, C_I \in \mathbb{G}_2, A_X \in \mathbb{G}_1; (\bar{A}, \bar{B}) \in \mathbb{G}_2 \times \mathbb{G}_1, \\ r, \tau_1, \tau_3, \tau_4 \in \mathbb{Z}_p) : \\ C_I = h_2^r g_2^{I(s)} \wedge \delta_3 = \tau_3 r \wedge \delta_4 = \tau_4 r \wedge \\ \bar{A}_2 = \bar{A} g^{\tau_1} \wedge \\ \bar{B}_1 = g_1^{\tau_3} g^{\tau_4} \wedge \bar{B}_2 = \bar{B} g^{\tau_3} \wedge \\ \frac{e(A_X, \bar{A}_2) \cdot e(\bar{B}_2, C_I)}{e(g_1, g_2)} = \frac{e(A_X, h)^{\tau_1} \cdot e(g, C_I)^{\tau_3}}{e(g, h_2)^{-\delta_3} \cdot e(\bar{B}_2, h_2)^r} \end{array} \right\}$$

The rest of the protocol remains the same. We defer the security arguments to the extended version of our paper.

Protocol for relation $\mathcal{R}_{\text{nonmem}}$. The interactive version of the protocol is as follows:

- Prover
 - Picks $\tau_1, \tau_3, \tau_4 \leftarrow \mathbb{Z}_p$
 - Computes
 - * $\bar{A}_2 = \bar{A} g^{\tau_1}$
 - * $\bar{B}_1 = g_1^{\tau_3} g^{\tau_4}, \bar{B}_2 = \bar{B} g^{\tau_3}$
 - Picks $r_r, r_{\tau_1}, r_{\tau_3}, r_{\tau_4}, r_{\delta_3}, r_{\delta_4} \leftarrow \mathbb{Z}_p$
 - Sends:
 - * $\bar{A}_2, \bar{B}_1, \bar{B}_2$
 - * $R_{2,1} = g_1^{r_{\tau_3}} g^{r_{\tau_4}}, R_{2,2} = (\bar{B}_1)^{r_r} g_1^{-r_{\delta_3}} g^{-r_{\delta_4}}$
 - * $R_3 = e(A_X, h)^{r_{\tau_1}} \cdot e(g, C_I)^{r_{\tau_3}} \cdot e(g, h_2)^{-r_{\delta_3}} \cdot e(\bar{B}_2, h_2)^{r_r}$
- Verifier sends $c \leftarrow \mathbb{Z}_p$
- Prover sends:
 - $s_r = r_r + cr, s_{\tau_1} = r_{\tau_1} + c\tau_1$
 - $s_{\tau_3} = r_{\tau_3} + c\tau_3, s_{\tau_4} = r_{\tau_4} + c\tau_4$
 - $s_{\delta_3} = r_{\delta_3} + c\delta_3, s_{\delta_4} = r_{\delta_4} + c\delta_4$
- Verifier checks:
 - $R_{2,1} = (\bar{B}_1)^{-c} g_1^{s_{\tau_3}} g^{s_{\tau_4}}$
 - $R_{2,2} = (\bar{B}_1)^{s_r} g_1^{-s_{\delta_3}} g^{-s_{\delta_4}}$
 - $R_3 \cdot \left(\frac{e(A_X, \bar{A}_2) \cdot e(\bar{B}_2, C_I)}{e(g_1, g_2)} \right)^c = e(A_X, h)^{s_{\tau_1}} \cdot e(g, C_I)^{s_{\tau_3}} \cdot e(g, h_2)^{-s_{\delta_3}} \cdot e(\bar{B}_2, h_2)^{s_r}$

7.3 Proving degree bound ($\mathcal{R}_{\text{degcheck}}$)

We now want to prove in ZK the following statement: *The prover knows at least d elements.* So far, the verifier only knows that the verification algorithm holds for some commitment C_I . In order to be convinced that C_I is a commitment to a set $I = \{x_1, \dots, x_d\}$, the verifier has to prove knowledge of a polynomial $I(x)$ of degree d . It is implied that as long as $I(x)$ is a polynomial, it is well-formed as a product of d monomials $(x + x_i)$ ($I(x)$ is part of the accumulator exponent (membership) or coprime to the accumulator exponent (non-membership)). We define the following relation:

$$\mathcal{R}_{\text{degcheck}} = \left\{ \begin{array}{l} (\text{pp}, C_I \in \mathbb{G}_2, d \in \mathbb{Z}_p; r \in \mathbb{Z}_p, I \subset \mathbb{Z}_p) : \\ C_I = h_2^r g_2^{I(s)} \wedge |I| = d \end{array} \right\}$$

For proving the above statement with existing protocols (that prove maximum instead of minimum polynomial degree), we use the following idea: a correct computation of the polynomial $I(x)$ that corresponds to elements in the set I with $|I| = d$, results to the following polynomial:

$$I(x) = \prod_{i=1}^d (x + x_i) = x^d + a_{d-1}x^{d-1} + \dots + a_1x + a_0$$

Let $f(x) = a_{d-1}x^{d-1} + \dots + a_1x + a_0$. Thus, $I(x) = x^d + f(x)$.

In order to prove that the degree of $I(x)$ is at least d it suffices to show that $\deg(f(x)) \leq d-1$. This implies that there is no term of f that eliminates x^d . Therefore, the degree of polynomial I is at least d .

Proving that $\deg(f) \leq d-1$. In order to prove a polynomial's maximum degree, we follow the technique used in Marlin [22]. Instead of publishing parameters of degree-specific size, we transform the proof to use the accumulator's parameters of size t and we shift the polynomial such that it has degree t instead of d .

Informally: polynomial $f(x)$ gets multiplied with a random polynomial of degree $t - (d-1)$ (the sparse polynomial $c \cdot x^{t-(d-1)}$ where c is a random scalar would suffice). This is what we call a shift and can be proven to be computed correctly with the use of pairings.

$$\mathcal{R}_{\text{mem}} = \left\{ \begin{array}{l} (\text{pp}, C I \in \mathbb{G}_2, A X \in \mathbb{G}_1; \hat{e} I \in \mathbb{G}_1, r, \ell_1, \ell_2 \in \mathbb{Z}_p) : C I = h r_2 g I_2^{(s)} \wedge \hat{d}_1 = \ell_1 r^{-1} \hat{d}_2 = \ell_2 r^{-1} \hat{e} I, \\ 1 = g \ell_1^{r_1} g^{\tau_2} \hat{e} I, \ell_2 = \hat{e} I g \ell_1^{-1} \\ \frac{e(\pi_{12}, C I)}{e(A X, g_2)} = e(g, C I) \ell_1 \cdot e(g, h_2)^{-\hat{d}_1} \cdot e(\hat{e} I, \ell_2, h_2)^{r_1} \end{array} \right\}$$

欠 mem 通过广义的 Schnorr 协议实例化, 该协议用于 DL 知识, 其中 r, T_1 为标量。

关系 R 的协议 mem。 该协议的交互版本如下:

- 普罗弗
 - 选择 $\ell_1, \ell_2 \leftarrow \mathbb{Z}_p$
 - 计算 $\hat{d}_1 = g \ell_1^{r_1} g^{\tau_2}$ 和 $\hat{d}_2 = \hat{e} I g \ell_1^{-1}$
 - 选择 $r, r, r_{\ell_1}, r_{\ell_2}, r_{\hat{d}_1}, r_{\hat{d}_2} \leftarrow \mathbb{Z}_p$
 - 发送:
 - * $\hat{e} I, \ell_1, \ell_2$
 - * $R_1 = g r_1^{\tau_1} g^{\tau_2}, R_2 = \hat{e} I, r r g^{-1} r_{\hat{d}_1} g^{-r_{\hat{d}_2}}$
 - * $R_3 = e(g, C I) r_{\ell_1} e(g, h_2)^{-r_{\hat{d}_1}} e(\hat{e} I, \ell_2, h_2)^{r r}$
- 验证器发送 $c \leftarrow \mathbb{Z}_p$
- 证明发送者:
 - $s_r = r_r + c r$
 - $s_{\tau_1} = r_{\tau_1} + c \tau_1, s_{\tau_2} = r_{\tau_2} + c \tau_2$
 - $s_{\delta_1} = r_{\delta_1} + c \delta_1, s_{\delta_2} = r_{\delta_2} + c \delta_2$
- 验证器检查:
 - $R_1 = \pi_{I,1}^{-c} g_1^{s_{\tau_1}} g^{s_{\tau_2}}$
 - $R_2 = \pi_{I,1}^{s_r} g_1^{-s_{\delta_1}} g^{-s_{\delta_2}}$
 - $R_3 \cdot \left(\frac{e(\pi_{12}, C I)}{e(A X, g_2)} \right)^c = e(g, C I) s_{\ell_1} e(g, h_2)^{-s_{\delta_1}} e(\hat{e} I, \ell_2, h_2)^{s_r}$

正确性、健全性与 ZK 性。 我们把安全性论证留待论文扩展版中讨论。

渐近性。 欠 mem 的实例化由 5 个群元素和 5 个域元素组成, 且具有常数证明者和验证者。

7.2 证明非成员身份 (欠 nonmem)

对于非会员, 欠 mem.init 被替换为 $(\text{pp}, C I \in \mathbb{G}_2, A X \in \mathbb{G}_1; \hat{e} I \in \mathbb{G}_1, r, \ell_1, \ell_2 \in \mathbb{Z}_p) : C I = h r_2 g I_2^{(s)} \wedge \hat{d}_1 = \ell_1 r^{-1} \hat{d}_2 = \ell_2 r^{-1} \hat{e} I, 1 = g \ell_1^{r_1} g^{\tau_2} \hat{e} I, \ell_2 = \hat{e} I g \ell_1^{-1} \wedge \frac{e(\pi_{12}, C I)}{e(A X, g_2)} = e(g, C I) \ell_1 \cdot e(g, h_2)^{-\hat{d}_1} \cdot e(\hat{e} I, \ell_2, h_2)^{r_1}$

同样地, 欠非记忆。初始化仍然无法隐藏见证 $\hat{e} I$ 。因此, 利用 RingCT[43] 中的技术, 欠非记忆。初始化可以转换为欠非记忆, 这证明了 ZK 中的上述陈述。

$$\mathcal{R}_{\text{nonmem}} = \left\{ \begin{array}{l} (\text{pp}, C I \in \mathbb{G}_2, A X \in \mathbb{G}_1; (A, B) \in \mathbb{G}_2 \times \mathbb{G}_1, \hat{e} I \in \mathbb{G}_1, r, \tau_1, \tau_3, \tau_4 \in \mathbb{Z}_p) : \\ C I = h r_2 g I_2^{(s)} \wedge \hat{d}_3 = \ell_3 r \wedge \hat{d}_4 = \ell_4 r \wedge \\ A_2 = A h_1^{\tau_1} \frac{B_1 = g_1^{\tau_3} g^{\tau_4} \wedge B_2 = B g^{\tau_3} \wedge}{e(A X, A_2) \cdot e(B_2, C I)} = e(A X, 5) \ell_1 \cdot e(g, C I) \ell_3 \cdot e(g, h_2)^{-\delta_3} \cdot e(B_2, h_2)^{r_1} \end{array} \right\}$$

其余方案保持不变。安全论证部分将推迟至本文扩展版本中讨论。

关系 R 的协议非记忆。 该协议的交互版本如下:

- 普罗弗
 - 选择 $\ell_1, \ell_3, \ell_4 \leftarrow \mathbb{Z}_p$
 - 计算
 - * $A_2 = A h_1^{\tau_1}$
 - * $B_1 = g_1^{\tau_3} g^{\tau_4}, B_2 = B g^{\tau_3}$
 - 选择 $r, r, r_{\ell_1}, r_{\ell_3}, r_{\ell_4}, r_{\hat{d}_3}, r_{\hat{d}_4} \leftarrow \mathbb{Z}_p$
 - 发送者:
 - * A_2, B_1, B_2
 - * $R_{2,1} = g_1^{r_{\tau_3}} g^{r_{\tau_4}}, R_{2,2} = (B_1)^{r r} g_1^{-r_{\delta_3}} g^{-r_{\delta_4}}$
 - * $R_3 = e(A X, 5) r_{\ell_1} \cdot e(g, C I) r_{\ell_3} \cdot e(g, h_2)^{-r_{\delta_3}} \cdot e(B_2, h_2)^{r r}$
- 验证器发送 $c \leftarrow \mathbb{Z}_p$
- 证明发送者:
 - $s_r = r_r + c r, s_{\tau_1} = r_{\tau_1} + c \tau_1$
 - $s_{\tau_3} = r_{\tau_3} + c \tau_3, s_{\tau_4} = r_{\tau_4} + c \tau_4$
 - $s_{\delta_3} = r_{\delta_3} + c \delta_3, s_{\delta_4} = r_{\delta_4} + c \delta_4$
- 验证器检查:
 - $R_{2,1} = (B_1)^{-c} g_1^{s_{\tau_3}} g^{s_{\tau_4}}$
 - $R_{2,2} = (B_1)^{s_r} g_1^{-s_{\delta_3}} g^{-s_{\delta_4}}$
 - $R_3 \cdot \left(\frac{e(A X, A_2) \cdot e(B_2, C I)}{e(g_1, g_2)} \right)^c = e(A X, 5) s_{\ell_1} \cdot e(g, C I) s_{\ell_3} \cdot e(g, h_2)^{-s_{\delta_3}} \cdot e(B_2, h_2)^{s_r}$

7.3 证明度界 (欠 degcheck)

我们现在要 ZK 证明以下陈述: *证明者至少知道 d 个元素*。到目前为止, 验证者只知道验证算法适用于某个承诺 $C I$ 。为了确信 $C I$ 是对集合 $I = \{x_1, \dots, x_d\}$ 的承诺, 验证者必须证明其对于一个 $I(x)$ 的多项式具有知识, 其次数为 d 。这意味着只要 $I(x)$ 是一个多项式, 它就作为 d 个单项式 $(x + x_i)$ 的乘积是良好形成的 ($I(x)$ 是累加器指数 (成员) 或与累加器指数互质 (非成员))。我们定义以下关系:

$$\mathcal{R}_{\text{degcheck}} = \{ (\text{pp}, C I \in \mathbb{G}_2, d h r_2 g I_2^{(s)} \in \mathbb{Z}_p) : r^{-1} \mathbb{Z}_p, I \subset \mathbb{Z}_p : \frac{C I}{|I|} = d \}$$

为用现有协议 (证明最大而非最小多项式次数) 验证上述陈述, 我们采用以下思路: 正确计算对应于集合 I 中元素的多项式 $I(x)$, 其中 $|I| = d$, 最终得到如下多项式:

$$I(x) = \prod_{i=1}^d (x + x_i) = x^d + a_{d-1} x^{d-1} + \dots + a_1 x + a_0 \quad i=1$$

设 $f(x) = a_{d-1} x^{d-1} + \dots + a_1 x + a_0$, 因此, $I(x) = x^d + f(x)$ 。

为了证明 $I(x)$ 的次数至少为 d , 只需证明 $\deg(f(x)) \leq d-1$ 。这意味着 f 中不存在能消除 x^d 的项。因此, 多项式 I 的次数至少为 d 。

证明 $\deg(f) \leq d-1$ 。 为了证明多项式的最大次数, 我们采用 Marlin[22] 中使用的技术。我们不公布特定次数的参数大小, 而是将证明转换为使用大小为 t 的累加器参数, 并将多项式移位使其次数为 t 而不是 d 。

非正式地说: 多项式 $f(x)$ 会与一个随机的次数为 $t - (d-1)$ 的多项式 (即稀疏多项式 $c \cdot x^{t-(d-1)}$, 其中 c 是一个随机标量) 相乘即可。这就是我们所说的移位操作, 通过配对可以证明其计算正确。

Using knowledge assumptions [28] (and commitment to the same polynomial multiplied with a) the prover shows knowledge of f . In other words, the prover was able to construct the result polynomial using public parameters (that consist of generators raised to powers of s up to t), therefore the degree of f does not exceed $d - 1$.

Protocol for $\mathcal{R}_{\text{degcheck}}$. The interactive version of the protocol is as follows:

- Prover
 - Computes: $f(x)$ as $I(x) = \prod_{i=1}^d (x + x_i) = x^d + f(x)$, where $\deg(f) \leq d - 1$
 - Sends $C_f = \frac{C_I}{g_2^{s^d}}$
- Verifier
 - Sends: $c \in \mathbb{Z}_p$
- Prover
 - Computes: $f(x) \cdot cx^{t-d+1}$ and $r \cdot cx^{t-d+1}$
 - Sends: $C = g_2^{f(s)cs^{t-d+1}} h_2^{r \cdot cs^{t-d+1}} \in \mathbb{G}_2, C^a \in \mathbb{G}_2$
- Verifier checks:
 - $e(g_1, C_I) = e(g_1, C_f) \cdot e(g_1, h^{s^d})$
 - $e(g_1, C) = e(g_1^{cs^{t-d+1}}, C_f) \cdot e(g_1, C^a) = e(g_1^a, C)$

Correctness, soundness, and ZK. We defer the security arguments to the extended version of our paper.

Asymptotics. The protocol has $O(d)$ prover cost that comes from multiplying $f(x)$ with the polynomial of degree $t - (d - 1)$ and computing its commitment C . It can be made non-interactive using the Fiat-Shamir transformation. The proof size is constant (the prover has to send over constant sized commitments C_I, C_f, C, C^a : a commitment to polynomial $I(x)$, to polynomial $f(x)$, to polynomial $f(x) \cdot c \cdot x^{t-(d-1)}$ and to polynomial $a \cdot f(x) \cdot c \cdot x^{t-(d-1)}$ respectively) and verification cost is also constant (7 pairings).

8 PROOF OF EXPONENTIATION

We present the Proof-of-Exponentiation (PoE) protocol in the known-order group setting. Informally, the prover can convince the verifier that the exponentiation of a group element by the evaluation of a known polynomial at a specific point is correct. That is, given a tuple, $(A_U, A_W, V(x)) \in (\mathbb{G}, \mathbb{G}, \mathbb{Z}_p[x])$, the prover can convince the verifier that $A_W = A_U^{V(x)}$, with a constant sized proof requiring constant number of pairing checks. Given the coefficients of polynomial $V(s)$, naively, computing $g^{V(s)}$ would require as many exponentiations as $\deg(V(s))$. However, with PoE, the verifier has to just perform cheaper polynomial division and *constant* pairing computation instead of performing linear number of group exponentiations.

Since naively verifying batch (non-)membership proofs require a multi-exponentiation of size $|I|$, we can delegate the expensive exponentiations to the prover using the PoE protocol. Note that the PoE protocol is of independent interest and can be used as a building block in other constructions.

For ease of exposition, we present the protocol (Fig. 1) using symmetric pairings. However, the protocol can be instantiated using asymmetric pairings.

$$\mathcal{R}_{\text{PoE}} = \left\{ ((A_U, A_W \in \mathbb{G}, V(x) \in \mathbb{Z}_p[x]); \perp) : \begin{array}{l} A_W = A_U^{V(x)} \in \mathbb{G} \end{array} \right\}$$

$\text{pp} \leftarrow \text{PoE.Setup}(1^\lambda)$:

1. $(p, \mathbb{G}, \mathbb{G}_T, e, g) \leftarrow \text{BilGen}(1^\lambda)$
2. $s \leftarrow \mathbb{Z}_p^*$
3. $\text{pp} := ((p, g, \mathbb{G}, \mathbb{G}_T, e), \{g^{s^i} \mid 0 \leq i \leq t\})$

Protocol PoE for $\mathcal{R}_{\text{PoE}}$:

Params: $\text{pp} \leftarrow \text{PoE.Setup}(1^\lambda)$

Inputs: $(A_U, A_W \in \mathbb{G}, V(x) \in \mathbb{Z}_p[x])$

Claim: $A_W = A_U^{V(x)} \in \mathbb{G}$

1. Verifier sends $\ell \leftarrow \mathbb{Z}_p$
2. Prover computes:
 - $q(x), r$ s.t. $V(x) = q(x) \cdot (x + \ell) + r$
 - $Q_1 = g^{q(s)}$ using pp
 - $Q_2 = g^{q(s) \cdot (\ell + s)}$ using pp
3. Prover sends Q_1, Q_2 to Verifier.
4. Verifier computes: r s.t. $r \equiv V(x) \pmod{(x + \ell)}$
 Accepts if: $e(Q_1, g^{(s+\ell)}) \stackrel{?}{=} e(Q_2, g) \wedge$
 $e(A_U, Q_2) \cdot e(A_U, g^r) \stackrel{?}{=} e(A_W, g)$

Figure 1: PoE protocol. We use Fiat-Shamir transformation to make this protocol into non-interactive. For the ease of exposition we present the construction in the symmetric pairing setting. However, we remark that our implementation uses asymmetric pairing.

We present the interactive version of the protocol for the symmetric pairing in Fig. 1, which can be made non-interactive using Fiat-Shamir transformation. The soundness of $\mathcal{R}_{\text{PoE}}$ is defined similar to its RSA counterparts as defined in [51] and [10]. We defer the proof of soundness of our PoE protocol to the extended version of our paper.

9 EVALUATION

In this section, we experimentally compare the aggregation operations in the RSA [10] and BP setting. We implement RSA accumulator using C++17, GNU Multiple Precision Arithmetic library 6.2.1 [52], and OpenSSL 3.0.2 [45]. We choose two 1024-bits prime numbers at random and compute the product to obtain a 2048 RSA modulus (using OpenSSL [45]). We implement ² the BP accumulator using Golang bindings of the mcl library [36, 46]. Specifically, we use BLS12-381, a pairing-friendly elliptic curve. A single group element $\mathbb{G}$ and a field element $\mathbb{F}$ in the RSA setting, by the virtue of the choice of parameters, are 256 and 32 Bytes, respectively. In the elliptic curve group, a single compressed $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ group element requires 48, 96, 576 Bytes, respectively. Moreover, an element in $\mathbb{Z}_p$ requires 32 Bytes. A single exponentiation in the RSA group $\mathbb{G}$ by an exponent at most 256-bits takes 449 μs on an average. However, a single exponentiation in the elliptic curve source groups take $\mathbb{G}_1$ and $\mathbb{G}_2$ takes 106 μs and 250 μs , respectively. As BLS12-381 curve contains numerous roots of unity, we implement FFT based polynomial algorithms to support fast polynomial operations in go-mcl [46].

Our implementation is single threaded and all our experiments were performed on an Intel Core i7-4770 CPU @ 3.40GHz with 8 cores and 32 GiB of RAM. Unless stated otherwise, we perform 3 runs of each experiment and report the average.

²Our code is available at: <https://github.com/accumulators-agg/accumulators>

利用知识假设[28] (以及对同一多项式乘以 a 的承诺), 证明者展示了对 f 的知识。换言之, 证明者能够使用公共参数 (由生成元按 s 的幂次提升至 t 的次数构成) 构造结果多项式, 因此 f 的次数不超过 $d-1$ 。

Rdegcheck 协议。 该协议的交互式版本如下:

- 普罗弗
 - 计算: $f(x)$ 为 $I(x) = \prod_{i=1}^d (x + x_i) = x^d + f(x)$, 其中 $\deg(f) \leq d-1$
 - 发送 $cf = \frac{Cf}{g_2^s}$
- 验证程序
 - 发送: $c \in \mathbb{Z}_p$
- 普罗弗
 - 计算: $f(x) \cdot cx^{t-d+1}$ 和 $r \cdot cx^{t-d+1}$
 - 发送: $c = g^{f/2(s)} \cdot c s^{t-d+1} h r^{2 \cdot c s^{t-d+1}} \in \mathbb{G}_2$, $ca \in \mathbb{G}_2$
- 验证器检查:
 - $e(g_1, cI) = e(g_1, cf) \cdot e(g_1, h s^d)$
 - $e(g_1, c) = e(g_1 s^{t-d+1}, cf)$, $e(g_1, ca) = e(ga_1, c)$

正确性、健全性与 ZK 性。 我们把安全性论证留待论文扩展版中讨论。

渐近性。 该协议具有 $O(d)$ 的证明者成本, 其来源于将 $f(x)$ 与次数为 $t - (d-1)$ 的多项式相乘并计算其承诺 C 。通过 Fiat-Shamir 变换可实现非交互性。证明规模为常数 (证明者需发送固定大小的承诺 CI 、 Cf 、 C 、 Ca : 分别是对多项式 $I(x)$ 、多项式 $f(x)$ 、多项式 $f(x) \cdot c \cdot x^{t-(d-1)}$ 以及多项式 $a \cdot f(x) \cdot c \cdot x^{t-(d-1)}$ 的承诺), 验证成本同样为常数 (7 个配对)。

8 指数运算的证明

我们在已知阶群设置中提出指数证明 (PoE) 协议。通俗地说, 证明者可以通过在特定点评估已知多项式来确信群元素的指数运算是正确的。即给定一个元组 $(Au, Aw, V(x))$

$(G, G, \mathbb{Z}_p[x])$, 证明者可以确信 $Aw = Av_u(s)$, 且只需常数规模的证明和常数次配对检查。给定多项式 $V(s)$ 的系数, 直观上计算 $g^{V(s)}$ 需要与 $V(s)$ 的次数相同次数的指数运算。然而通过 PoE, 验证者只需执行更廉价的多项式除法和常数配对计算, 而非执行线性数量的群指数运算。

由于简单验证批处理 (非) 成员证明需要对规模 $|I|$ 进行多次指数运算, 我们可以使用 PoE 协议将这些昂贵的指数运算委托给证明者。需要注意的是, PoE 协议本身具有独立的研究价值, 可作为其他构造中的基础模块使用。

为便于阐述, 我们采用对称配对方式呈现该协议 (图1)。但该协议亦可基于非对称配对方式实现。

$$\mathcal{R}_{\text{PoE}} = \left\{ ((Au, Aw, wG, V(x) \in \mathbb{Z}_p[x]); \perp) : \begin{matrix} A \\ = A_U^{V(s)} \in \mathbb{G} \end{matrix} \right\}$$

$pp \leftarrow \text{PoE. 设置}(1\lambda)$:

1. $(p, G, GT, e, g) \leftarrow \text{BilGen}(1\lambda)$
2. $s \leftarrow \mathbb{Z}_p^*$
3. $pp := ((p, g, G, GT, e), \{g^{s^i} | 0 \leq i \leq t\})$

R PoE 的 PoE 协议:

参数: $pp \leftarrow \text{PoE. 设置}(1\lambda)$

输入: $(Au, Aw, G, V(x) \in \mathbb{Z}_p[x])$

声明: $Aw = Av_u(s) \in \mathbb{G}$

1. 验证器发送 $r \leftarrow \mathbb{Z}_p$
2. Prover 计算:
 - $q(x), r$ 使得 $V(x) = q(x) \cdot (x+r)$
 - $Q_1 = g^{q(s)} \cdot g^{r \cdot Q_1}$ 使用 pp
 - $Q_2 = g^{q(s)} \cdot (r+s)$ 使用 pp
3. 证明者将 Q_1, Q_2 发送给验证者。
4. 验证器计算: r 使得 $r \equiv V(x) \pmod{(x+r)}$; 接受条件为: $e(Q_1, g^{(s+r)}) \stackrel{?}{=} e(Q_2, g)$
 $e(Au, Q_2) \cdot e(Au, gr) \stackrel{?}{=} e(Aw, g)$

图1: PoE 协议。 我们采用 Fiat-Shamir 变换使该协议变为非交互式。为便于阐述, 本文在对称配对场景下展示其构造过程。但需说明的是, 我们的实现采用了非对称配对。

我们在图1中展示了对称配对协议的交互式版本, 该版本可通过 Fiat-Shamir 变换实现非交互式。R PoE 的可靠性定义与文献 [51] 和 [10] 中定义的 RSA 协议类似。我们将在论文扩展版本中证明 PoE 协议的可靠性。

9 评估

在本节中, 我们通过实验比较了 RSA [10] 和 BP 设置中的聚合操作。我们使用 C++17、GNU 多精度算术库 6.2.1 [52] 和 OpenSSL 3.0.2 [45] 实现 RSA 累加器。我们随机选择两个 1024 位素数并计算乘积以获得 2048 RSA 模数 (使用 OpenSSL [45])。我们使用 mcl 库的 Golang 绑定 [36, 46] 实现² BP 累加器。具体而言, 我们使用 BLS12-381 这种配对友好的椭圆曲线。在 RSA 设置中, 由于参数选择, 单个群元素 G 和域元素 F 分别为 256 和 32 字节。在椭圆曲线群中, 单个压缩的 G_1 、 G_2 、 G_T 群元素分别需要 48、96、576 字节。此外, $\mathbb{Z}_p$ 中的元素需要 32 字节。在 RSA 群 G 中, 一次最多 256 位的指数运算平均需要 449 μ 。然而, 椭圆曲线源群中的单次指数运算分别需要 G_1 和 G_2 分别花费 106 μ 秒和 250 μ 秒。由于 BLS12-381 曲线包含大量单位根, 我们在 go-mcl [46] 中实现了基于 FFT 的多项式算法以支持快速多项式运算。

本实现为单线程程序, 所有实验均在配备 8 核 32GB 内存的 Intel Core i7-4770 CPU (主频 3.40GHz) 上完成。除非另有说明, 每个实验均进行 3 次重复并报告平均值。

² 我们的代码可以在以下网址获取: <https://github.com/accumulators-agg/accumulators>

Operation	Sch.	Batch size				
		2 ⁹	2 ¹¹	2 ¹³	2 ¹⁵	2 ¹⁷
Domain mapping (s)	RSA	0.33	1.31	5.25	20.99	83.95
	BP	0	0	0	0	0
Commit (s)	RSA	0.52	2.09	8.38	33.55	134.37
	BP	0.05	0.24	1.12	5.17	24.28
AggMem (min)	RSA	0.04	0.17	0.8	3.66	16.49
	BP	0	0.01	0.18	2.51	38.35
MemVerify (s)	RSA	0.52	2.1	8.38	33.54	134.37
	BP	0.11	0.46	2.0	8.65	38.14
AggMemPoE (min)	RSA	0.04	0.19	0.86	3.87	17.33
	BP	0	0.02	0.2	2.58	39.12
MemVerifyPoE (s)	RSA	0.33	1.32	5.32	21.4	86.16
	BP	0.03	0.16	0.77	3.8	18.62
AggNonMem (min)	RSA	0.05	0.25	1.16	5.3	23.9
	BP	0.1	1.54	24.54	N/A	N/A
NonMemVerify (s)	RSA	0.72	2.87	11.49	45.98	184.12
	BP	0.11	0.46	2.0	8.65	38.14
AggNonMemPoE (min)	RSA	0.07	0.3	1.37	6.14	27.24
	BP	0.1	1.55	24.58	N/A	N/A
NonMemVerifyPoE (s)	RSA	0.34	1.34	5.33	21.41	86.17
	BP	0.03	0.16	0.77	3.8	18.62

Table 2: Accumulator batching operation costs for different batch sizes. In the first column, (s) denotes seconds and (min) minutes. The costs for RSA operations include the computations required to map to the prime domain. N/A stands for very large costs which are not interesting to compute.

9.1 Aggregation

In Table 1, we present the asymptotic costs for various operations in the BP setting, and in Table 2 we present our corresponding evaluation results.

Public parameters. Both the RSA and the BP accumulators require a trusted setup phase to generate the public parameters. Classgroups based accumulator constructions [13] do not require trusted setup in the unknown-order group setting, but they are too slow in practice. The public parameters in the RSA setting is just the RSA modulus and the group generator. However, in the BP setting, the public parameter consists of $n \cdot \mathbb{G}_1 + n \cdot \mathbb{G}_2$ elements, where n is the maximum size of the accumulated set³. For $n = 2^{17}$, the public parameters occupies 18 MiB. With PoE, it is sufficient to store *only* constant number of values (g_1, g_1^s, g_2, g_2^s) from the public parameter by the verifier. Recall that g_1, g_2 are the group generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively.

Domain mapping. To add values from an arbitrary domain $\mathcal{D}$ to the accumulator set, each element has to be mapped to a value in the specific accumulator domain $\mathcal{D}'$. Since the RSA accumulator requires a prime domain, we implement the standard “hash to prime” algorithm [9, 10], where the hash operation is successively applied on the input until the hash function returns a prime value. We use Blake2s hash implementation in OpenSSL [45] to convert a value from arbitrary domain to Prime domain and GMP’s Miller-Rabin’s

primality testing (15 rounds). For the BP setting, as discussed in §3.3, the specific domain is $\mathbb{Z}_p$. In our implementation we use the BLS12-381 curve for which the group order is around 256-bits [11]. Thus, we can accumulate any arbitrary string of size up to 256-bits without the need of any mapping.

In our experiments, we consider the accumulation of arbitrary 256-bit strings. In Table 2, we report the cost mapping these arbitrary strings to the accumulator domain. Mapping an arbitrary string to the prime domain takes 640 to 894 μ s for 15 to 50 rounds of Miller-Rabin primality testing. Before performing an accumulator operation, all elements have to be converted to the accumulator domain. Thus, we include the cost of mapping to the accumulator domain for all operations in Table 1.

Discussion. We now briefly discuss what could go wrong in a universal accumulator if we don’t include the mapping process during verification.

Consider the two most popular H_{prime} approaches [9, 10]⁴ for mapping a non-prime element x to a prime y : (1) perform repetitive hashing $H(H(\dots H(x))) = y$ until a primality test indicates that the output is a prime, then outputs x and r where r is the number of hashing rounds required, (2) perform repetitive hashing of the value $(x||r)$, where r is a random nonce, until $H(x||r) = y$ is a prime. In both cases, when a prover wishes to add element x to the RSA accumulator, it first calculates the mapping y and stores r . The element y is accumulated. When they wish to prove (non-)membership, they provide r as a witness along with the proof to decrease verification costs. However, if a malicious prover can find two numbers of hashing rounds r_1, r_2 for the same element x , that correspond to two elements y_1, y_2 in $\mathcal{D}'$, then if say y_1 was the accumulated value (for x), the malicious prover could use y_2 to argue non-membership for x .

In order to avoid such attacks in universal accumulators⁵, before verifying any (non-)membership proof, *all* verifiers need to check that the given prime mapping y is the first one that corresponds to the arbitrary element x and thus need to run all the repetitive steps the prover does. If the proof is a batch proof for $|I|$ elements, the verifiers need to repeat the process $|I|$ times individually for each element. The same holds for updates (addition or deletions) to ensure primality since they might be initiated by an untrusted entity.

Commit. To commit to a set I in the RSA setting, we first compute the product of the elements in the set. Then, we perform modular exponentiation of this large product of size $O(\lambda \cdot |I|)$ -bits. However, for BP accumulators, we first compute the coefficient of the accumulator polynomial using the subproduct algorithm and fast polynomial multiplication. Then, we perform a single multi-exponentiation of size $|I|$. Observe that from Table 2, even after subtracting the domain mapping costs from the Commit, BP accumulators are faster! For a set size of 2^{15} , it takes around 12.56 seconds to perform Commit, in the RSA setting, whereas it takes just 5.17 seconds in the BP setting.

³Rather than relying on a trusted entity, it is possible to use an MPC based setup ceremony to generate public parameters. We can adopt approaches from real-world MPC ceremonies of Zcash and AZTEC protocol which have successfully generated SNARK parameters for circuit sizes 2^{21} and 2^{27} , respectively [12][50].

⁴A recent work [39], attempts to optimize the “hash to prime” approach, by using Pocklington primality certificates in order to reduce the cost of primality testing on the side of the verifier. However, it still does not guarantee a deterministic mapping.

⁵If the accumulator does not support non-membership, then this attack does not apply.

操作	学校	批量大小				
		2 ⁹	2 ¹¹	2 ¹³	2 ¹⁵	2 ¹⁷
域映射	RSA	0.33	1.31	5.25	20.99	83.95
	BP	0	0	0	0	0
提交	RSA	0.52	2.09	8.38	33.55	134.37
	BP	0.05	0.24	1.12	5.17	24.28
AggMem (min)	RSA	0.04	0.17	0.8	3.66	16.49
	BP	0	0.01	0.18	2.51	38.35
MemVerify (s)	RSA	0.52	2.1	8.38	33.54	134.37
	BP	0.11	0.46	2.0	8.65	38.14
AggMemPoE (min)	RSA	0.04	0.19	0.86	3.87	17.33
	BP	0	0.02	0.2	2.58	39.12
MemVerifyPoE (s)	RSA	0.33	1.32	5.32	21.4	86.16
	BP	0.03	0.16	0.77	3.8	18.62
AggNonMem (min)	RSA	0.05	0.25	1.16	5.3	23.9
	BP	0.1	1.54	24.54	N/A	N/A
NonMemVerify (s)	RSA	0.72	2.87	11.49	45.98	184.12
	BP	0.11	0.46	2.0	8.65	38.14
AggNonMemPoE (min)	RSA	0.07	0.3	1.37	6.14	27.24
	BP	0.1	1.55	24.58	N/A	N/A
NonMemVerifyPoE (s)	RSA	0.34	1.34	5.33	21.41	86.17
	BP	0.03	0.16	0.77	3.8	18.62

表2: 不同批量大小的累加器批量操作成本。第一列中, (s)表示秒, (min) 表示分钟。RSA操作的成本包括映射到素数域所需的计算。N/A表示非常大的成本, 这些成本不值得计算。

9.1 聚集

在表1中, 我们展示了BP设置下各种操作的渐近成本, 而在表2中, 我们展示了相应的评估结果。

公共参数。 RSA和BP累加器都需要可信设置阶段来生成公共参数。基于类群的累加器构造[13]在未知阶群设置中不需要可信设置, 但实际运行速度太慢。RSA设置中的公共参数仅包含RSA模数和群生成器。然而, 在BP设置中, 公共参数由 $n \cdot G_1 + n \cdot G_2$ 个元素组成, 其中 n 是累加集的最大规模³。当 $n=2^{17}$ 时, 公共参数占用18MiB。通过PoE, 验证器只需存储来自公共参数的仅固定数量的值 (g_1, g_{s1}, g_2, g_{s2})。需注意 g_1, g_2 分别是 G_1 和 G_2 的群生成器。

域映射。 要将任意域 D 中的值添加到累加器集合, 每个元素都必须映射到特定累加器域 D 中的某个值。由于RSA累加器需要素数域, 我们采用标准的“哈希到素数”算法[9, 10], 即对输入连续执行哈希运算, 直到哈希函数返回素数值。我们使用OpenSSL中的Blake2哈希实现[45]将任意域值转换为素数域, 并采用GMP的米勒-拉宾算法

³ 而不是依赖可信实体, 可以使用基于MPC的设置仪式来生成公共参数。我们可以采用Zcash和aztec协议等真实世界MPC仪式中的方法, 这些方法已成功为电路规模221 和227 分别生成了snark参数[12] [50]。

素性测试(15轮)。对于BP设置, 如第3.3节所述, 其特定域为 Z_p 。在我们的实现中, 我们使用BLS12-381曲线, 其群阶约为256位[11]。因此, 我们可以累积任何长度不超过256位的任意字符串, 而无需任何映射。

在实验中, 我们研究了任意256位字符串的累加过程。表2中展示了这些字符串映射到累加器域的计算成本。将任意字符串映射到素数域需要640至894 μ 秒, 对应15到50轮Miller-Rabin素性测试。由于执行累加器操作前需将所有元素转换至累加器域, 因此表1中已包含所有操作的累加器映射成本。

讨论。 现在我们简要讨论一下, 如果在验证过程中不包含映射过程, 通用累加器可能会出现哪些问题。

考虑两种最流行的H素数方法[9, 10]⁴将非素数元素 x 映射到素数 y : (1) 进行重复哈希 $H(H(\dots H(x)))=y$, 直到素性测试表明输出为素数, 然后输出 x 和 r , 其中 r 是所需的哈希轮数; (2) 对值 $(x||r)$ 进行重复哈希, 其中 r 是随机随机数, 直到 $H(x||r)=y$ 为素数。在这两种情况下, 当证明者希望将元素 x 添加到RSA累加器时, 首先计算映射 y 并存储 r 。元素 y 被累加。当他们希望证明(非)成员身份时, 他们提供 r 作为见证与证明一起, 以降低验证成本。然而, 如果恶意证明者能够为同一元素 x 找到两个哈希轮数 r_1, r_2 , 它们对应于 D 中的两个元素 y_1, y_2 , 那么假设 y_1 是累积值(针对 x), 恶意证明者就可以利用 y_2 来论证 x 的非成员身份。

为避免通用累加器遭受此类攻击⁵, 在验证任何(非)成员证明前, 所有验证者都需要检查给定的素数映射 y 是否为对应任意元素 x 的第一个映射, 因此需要执行证明者进行的所有重复步骤。若证明是针对 $|I|$ 个元素的批量证明, 则验证者需针对每个元素单独重复该过程 $|I|$ 次。更新操作(添加或删除)同样需要进行素性验证, 因为这些操作可能由不可信实体发起。

Commit. 在RSA环境下对集合 I 进行Commit操作时, 我们首先计算该集合中元素的乘积。随后, 对这个规模为 $O(2 \cdot |I|)$ -比特的大型乘积执行模指数运算。然而对于BP累加器, 我们首先使用子乘积算法和快速多项式乘法计算累加多项式的系数, 然后对规模为 $|I|$ 的乘积执行单次多指数运算。从表2可以看出, 即使扣除域映射成本后, BP累加器的Commit速度仍更快! 当集合规模为 2^{15} 时, 在RSA环境下完成Commit需要约12.56秒, 而在BP环境下仅需5.17秒。

⁴ 最近的一项研究[39]尝试通过使用Pocklington素性证书来优化“哈希到素数”的方法, 以降低验证方的素性测试成本。然而, 该方法仍无法保证确定性映射。

⁵ 若累加器不支持非成员身份, 则该攻击无效。

Operation	RSA (Bytes)	BP (Bytes)
Digest	256	48
Mem. proof	256	48
Non-mem. proof	288	80
Agg. mem. proof (Naive)	256	48
Agg. non-mem. proof (Naive)	3.85* MiB	144* Bytes
Agg. mem. proof (PoE)	512	192
Agg. non-mem. proof (PoE)	1312	336
PoE	256	144 or 96
PoKE	544	×

Table 3: Sizes of accumulator digest and proofs in bytes. Asterisk(*) denotes a batch size of 2^{17} .

Membership aggregation. We implement the membership proof aggregation algorithm from Boneh et al. to aggregate a set of membership proofs in RSA accumulators [10]. Aggregating a pair of membership proofs involves computing Shamir’s trick, which requires computing the Bézout coefficients and performing two exponentiations. However, aggregating membership proofs in BP accumulators, involves $O(|I| \log^2 |I|)$ field operations and one $|I|$ -sized multi-exponentiation (Table 1). Thus, we observe that the prover’s cost to aggregate is lower for the BP accumulator for set sizes up to $|I| = 2^{15}$. Beyond these set sizes, the field operations in BP dominates aggregation cost. It is not very common for a prover to hold (or wish to aggregate) more than 2^{15} proofs. Thus, for most applications BP should be preferable.

We also implement PoE from Boneh et al. [10] and from §8 to optimize the verification of the aggregated proof in RSA and BP accumulator, respectively. Since the prover overhead in computing the aggregated proof is dominated by field operations in both RSA and BP accumulators, the additional exponentiations overhead incurred by a PoE enabled prover is limited. We observe this in our experiments as the prover engaging PoE additional incurs only around 12.7 and 4.2 seconds for RSA and BP accumulators, respectively, for 2^{15} values (Table 2).

We observe that verifying batch proofs (without PoE) in BP accumulator is $3.5\times$ to $4.7\times$ faster than RSA accumulators. This is because the multi-exponentiations in elliptic curve group $\mathbb{G}_1$ is faster than a single large exponentiation in the RSA group $\mathbb{G}$. In the PoE enabled setting, we observe that BP verification is $4.6\times$ to $11\times$ faster than RSA. In addition to the domain mapping costs, we also include the overhead to compute the Fiat-Shamir coins in our experiments.

Non-membership aggregation. The prover’s cost to aggregate non-membership proofs is better for RSA regardless of the use of PoE. This is due to BP’s $O(|I|^2 \log |I|)$ field operations that comes from constructing the $Y_i(s)$ terms (§5.2).

In comparison with RSA, verifying batch non-membership proofs in BP is at least $4.6\times$ faster in any case. Observe that verifying a batch non-membership is computationally similar to verifying a

Operation	2^9	Batch size		
		2^{11}	2^{13}	2^{15}
Pedersen Commitment (s)	0.08	0.31	1.24	4.98
Prover $\mathcal{R}_{\text{mem}}$		2.97 ms		
Verifier $\mathcal{R}_{\text{mem}}$		3.66 ms		
Proof size $\mathcal{R}_{\text{mem}}$		0.91 KiB		
Prover $\mathcal{R}_{\text{nonmem}}$		4.65 ms		
Verifier $\mathcal{R}_{\text{nonmem}}$		5.18 ms		
Proof size $\mathcal{R}_{\text{nonmem}}$		1.03 KiB		
Prover $\mathcal{R}_{\text{degcheck}}$ (s)	0.17	0.64	2.57	11.29
Verifier $\mathcal{R}_{\text{degcheck}}$		4.49 ms		
Proof size $\mathcal{R}_{\text{degcheck}}$		0.29 KiB		

Table 4: Single-threaded microbenchmarks for our ZK constructions.

batch membership proof in the BP setting, regardless of the usage of PoE (Table 1). Thus, we observe similar performance numbers for NonMemVerify and NonMemVerifyPoE when compared to MemVerify and MemVerifyPoE, respectively, in the BP setting.

Storage and proof sizes. In Table 3, we present the storage overhead of proofs in both RSA and BP setting. For a similar level of security, an RSA group element $\mathbb{G}$ is of size 256 bytes as opposed to an elliptic curve element that is of size 48 Bytes for $\mathbb{G}_1$ and 96 Bytes for $\mathbb{G}_2$. The accumulator value and the batch membership proof consist of one group element in both constructions. Non-membership consists of one group element and one integer for RSA and two group elements in the BP setting. The integer in RSA batch non-membership proof grows linear in the batch size. The PoE proof adds to the proof size one group element in the RSA. However, in the BP setting PoE adds an overhead of either $(\mathbb{G}_1, \mathbb{G}_1)$ or $(\mathbb{G}_1, \mathbb{G}_2)$ depending on whether prover computes a proof for $g_2^{I(s)}$ or $g_1^{I(s)}$, respectively. The RSA non-membership proof can be made succinct using PoE and PoKE [10]. We note that in BP, non-membership proofs do not need PoKE as the proofs are already constant sized. Thus, we observe that membership and non-membership proofs in BP accumulators are $2.5\times$ to $5\times$ smaller and $3.5\times$ smaller than the RSA accumulators, respectively.

9.2 Zero-knowledge batch proofs

We microbenchmark our proposed ZK batch proofs in Table 4.

Public parameters. The constructions for $\mathcal{R}_{\text{mem}}$ and $\mathcal{R}_{\text{nonmem}}$, require the prover and the verifier to store additional generators for Pedersen commitment. However for $\mathcal{R}_{\text{degcheck}}$, since we rely on t -PKE assumption, the prover needs to additionally store $3n \cdot \mathbb{G}_2$ elements (36 MiB). Whereas, the verifier needs to store only $2n \cdot \mathbb{G}$ elements (24 MiB). When $n = 2^{17}$, generating additional $3n \cdot \mathbb{G}_2$ parameter takes around 98 seconds using a single thread.

Prover overhead. To commit to coefficients of a polynomial using Pedersen commitment, we use the values $g_2^{s^i}$ and an independent group generator. Using multi-exponentiation, it takes 4.98 seconds to commit to a batch of size 2^{15} . Recall that, given a commitment to the subset I , prover incurs constant overhead to generate a ZK proof for $\mathcal{R}_{\text{mem}}$ and $\mathcal{R}_{\text{nonmem}}$ regardless of the batch size. Thus,

操作	RSA 字节	BP 字节
消化	256	48
校样	256	48
非记忆证明	288	80
总备忘录证明（朴素）	256	48
非记忆性证明（朴素）	3.85* MiB	144* 字节
聚合存储器证明（PoE）	512	192
非存储器聚合证明（PoE）	1312	336
PoE	256	144或96
POKE	544	×

表3：累加器消化和证明的大小（以字节为单位）。星号（*）表示批量大小为2¹⁷。

成员聚合。我们实现了Boneh等人提出的成员证明聚合算法，将一组成员证明聚合到RSA累加器中[10]。聚合一对成员证明涉及计算Shamir技巧，这需要计算Bezout系数并执行两次指数运算。然而，在BP累加器中聚合成员证明涉及 $O(|I| \log^2 |I|)$ 次域运算和一次 $|I|$ 规模的多重指数运算（表1）。因此我们观察到，对于集合大小不超过 $|I|=2^{15}$ 的情况，证明者在BP累加器中的聚合成本较低。超过这些集合大小时，BP中的域运算会主导聚合成本。证明者通常不会持有（或希望聚合）超过2¹⁵个证明。因此对于大多数应用而言，BP应是更优选择。

我们还实现了Boneh等人提出的PoE方法[10]从第8节开始，我们分别对RSA和BP累加器中的聚合证明验证进行了优化。由于在RSA和BP累加器中，计算聚合证明时证明者的开销主要由场运算主导，因此启用PoE的证明者所增加的指数运算开销较为有限。实验结果表明，当使用2¹⁵个数值时，启用PoE的证明者在RSA和BP累加器中分别仅额外消耗约12.7秒和4.2秒的时间（表2）。

我们发现，在BP累加器中验证批证明（无需PoE）的速度比RSA累加器快3.5到4.7倍。这是因为椭圆曲线群G₁中的多次指数运算比RSA群G中的单次大指数运算更快。在启用PoE的场景下，BP验证速度比RSA快4.6到11倍。除了域映射成本外，我们的实验还包含了计算Fiat-Shamir代币的额外开销。

非成员聚合。证明者聚合非成员证明的成本在RSA中更优，无论是否使用PoE。这是由于BP的 $O(|I|^2 \log |I|)$ 字段运算，该运算源自构建 $y_i(s)$ 项（§5.2）。

与RSA相比，BP中批量非成员证明的验证速度在任何情况下至少快4.6倍。需注意的是，验证批量非成员证明在计算复杂度上与验证

操作	2 ⁹	2 ¹¹	2 ¹³	2 ¹⁵
佩德森 承诺	0.08	0.31	1.24	4.98
证明R mem			2.97 毫秒	
验证器R内存			3.66 毫秒	
证明大小 R mem			0.91 KiB	
Prover R 非记忆			4.65 毫秒	
验证器R非存储器			5.18 毫秒	
证明大小 R 非内存			1.03 KiB	
Prover R degcheck (s)	0.17	0.64	2.57	11.29
验证程序 R degcheck			4.49 毫秒	
证明大小 R degcheck			0.29 KiB	

表4：我们 ZK 构建的单线程微基准测试。

在BP设置中，无论是否使用PoE（表1），批量成员证明的性能表现相似。因此，在BP设置中，我们观察到NonMemVerify和NonMemVerifyPoE的性能数据分别与MemVerify和MemVerifyPoE相当。

存储与证明规模。在表3中，我们展示了RSA和BP场景下证明的存储开销。在相同安全级别下，RSA群元素G的大小为256字节，而椭圆曲线元素的大小分别为G₁的48字节和G₂的96字节。累加器值和批量成员证明在两种构造中均包含一个群元素。非成员证明在RSA场景中包含一个群元素和一个整数，在BP场景中则包含两个群元素。RSA批量非成员证明中的整数随批量规模线性增长。PoE证明在RSA场景中会增加一个群元素的证明开销，而在BP场景中PoE会增加（G₁，G₁）或（G₁，G₂）的开销，具体取决于证明者是为 $g^{I_2(s)}$ 还是 $g^{I_1(s)}$ 计算证明。通过PoE和PoKE[10]，RSA非成员证明可实现简化。我们注意到，在BP中，非成员证明不需要PoKE，因为证明的大小已经固定。因此，我们观察到BP累加器中的成员证明和非成员证明分别比RSA累加器小2.5倍和3.5倍。

9.2 零知识批量证明

我们在表4中对提出的 ZK 批证明进行了微基准测试。

公共参数。R mem 和 R nonmem 的构造需要证明者和验证者为佩德森承诺存储额外的生成器。然而对于 R degcheck，由于我们依赖于t-PKE假设，证明者需要额外存储 $3n \cdot G_2$ 个元素（36 MiB）。而验证者仅需存储 $2n \cdot G$ 个元素（24 MiB）。当 $n=2^{17}$ 时，使用单线程生成额外的 $3n \cdot G_2$ 个参数大约需要98秒。

证明者开销。使用，对多项式的系数进行承诺

Pedersen承诺中，我们使用 g_{s2} 值和一个独立的群生成器。通过多指数运算，对大小为2¹⁵的批次进行承诺需要4.98秒。需要指出的是，给定对子集I的承诺，证明者为R mem和R nonmem生成 ZK 证明时会产生常数开销，且与批次大小无关。因此，

to prove $\mathcal{R}_{\text{mem}}$ and $\mathcal{R}_{\text{nonmem}}$, it takes 2.97 and 4.65 milliseconds, respectively. To prove a lower bound on the degree of $I(s)$, the prover needs to compute Pedersen commitments on $I(s)$ with and without t -PKE.

Verification time and proof size. The proof of $\mathcal{R}_{\text{mem}}$ is $(4 \cdot \mathbb{G}_1 + \mathbb{G}_T + 5 \cdot \mathbb{Z}_p)$ and the proof of $\mathcal{R}_{\text{nonmem}}$ is $(3 \cdot \mathbb{G}_1 + \mathbb{G}_2 + \mathbb{G}_T + 6 \cdot \mathbb{Z}_p)$. With a 64-bit integer to denote the degree and three elements in $\mathbb{G}_2$, a prover can prove a lower bound on the degree of a polynomial. All the proofs in our scheme can be verified with a constant number of exponentiations and pairing operations.

9.3 Comparison with HARISA [18]

In this subsection, we argue that our approach to ZK batch proofs of membership can be at least $16\times$ faster than the current state-of-the-art approaches to ZK batch membership proofs in the RSA setting for a reasonable choice of batch size. We also report the performance of our ZK batch proof of non-membership in Fig. 2. HARISA does not support non-membership.

Experimental setup. We fix the maximum size of the set to 2^{17} elements and measure the performance of computing the zero-knowledge proof of batch (non-)membership while revealing the size of the batched subset. Moreover, we consider an experimental setup where the prover must do maximal work. That is, we assume that the prover: (1) has access only to the individual (non-)membership witness but not the batch membership witness, (2) does not have access to the commitment to the batched subset, and (3) has access to the accumulator digest and public parameters. Thus, the prover incurs the cost of: (1) computing the commitment to set I , (2) aggregating the individual (non-)membership witnesses to obtain batch witness, (3) proving the relation $\mathcal{R}_{\text{mem}}$ or $\mathcal{R}_{\text{nonmem}}$, and (4) proving the relation $\mathcal{R}_{\text{degcheck}}$ for $d = |I|$.

Baseline measure. We compare the performance of our scheme against results of HARISA [18] by Campanelli et al., which builds a succinct batch proof of membership while preserving the privacy of the batched elements. Their work combines proof of knowledge of exponent (PoKE) along with CP-SNARK for integer arithmetic relations and bound checks to prove batch membership. They implement their construction using LegoGroth16 in C++. Similar to our experiments, they require the prover to compute the batch witness using individual witnesses and their experiments are single

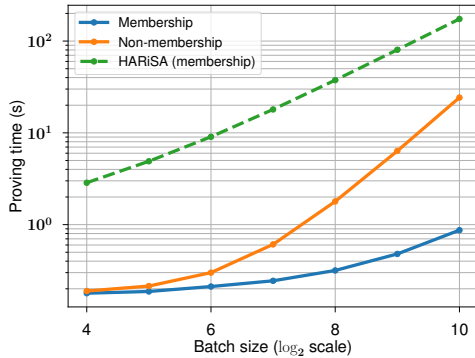


Figure 2: We extrapolate the proving costs using the numbers reported in HARISA [18]. Note that the results in the RSA setting does include the Hash-to-prime costs.

Scheme	Setup	Verifier	Proof size
HARISA [18]	Mem.	Trusted	63 ms
This work	Mem.	Trusted	7.94 ms
	Non-Mem.	Trusted	9.41 ms

Table 5: Verification overhead and proof size.

threaded. Also, recall that in our experiments we reveal the size of the subset I , which is currently not implemented in HARISA [18].

Proving time. HARISA reports a prover time of 2.86 and 9.02 seconds for a batch of size 16 and 64, respectively. Recall that their implementation uses LegoGroth16 proof system, thus the prover time is dominated by large FFTs and exponentiations. The performance numbers reported by HARISA uses Amazon EC2 r5.8xlarge [18, Figure 4], which we extrapolate for various batch sizes in Fig. 2. However, in Fig. 2, the performance of our scheme is measured on an Intel Core i7-4770 CPU@ 3.40GHz. We observe that for a batch size of 16, our approach takes merely 0.18 seconds, whereas HARISA takes 2.86 seconds, thus resulting in $16\times$ speed up. Clearly, our performance is still an order of magnitude faster even when benchmarked on a much slower machine. Thus, we argue that our order of magnitude performance gain will carry over even when we benchmark our scheme and HARISA on the same machine.

To the best of our knowledge, there are no known sub-linear proof sized privacy preserving batch non-membership proofs in the RSA setting without generic arguments such as SNARKs. However, SNARKs based approaches are too slow in practice. Thus, we report the performance of our non-membership scheme in the context of membership scheme due to lack of appropriate baseline.

Verification time and proof size. Our approach has a comparable proof size and superior verification speed. We report this in Table 5.

ACKNOWLEDGMENTS

We thank Yupeng Zhang for pointing us to prior work on proving the degree of a polynomial in ZK and Alin Tomescu for productive discussions. Foteini Baldimtsi and Ioanna Karantaidou were partially supported by NSF #171767, NSF #2143287 and awards from Facebook and Google Research. Charalampos Papamanthou is supported in part by awards from Protocol Labs, VMware, JPMorgan, the ACE-PAVE grant from the Algorand Foundation as well as NSF awards CNS-1652259 and CNS-2154732.

REFERENCES

- [1] 2015. Hyperledger Indy website. <https://www.hyperledger.org/> [Online; accessed 1-May-2022].
- [2] Tolga Acar and Lan Nguyen. 2011. Revocation for delegatable anonymous credentials. In *International Workshop on Public Key Cryptography*. Springer, 423–440.
- [3] Shashank Agrawal, Chaya Ganesh, and Payman Mohassel. 2018. Non-Interactive Zero-Knowledge Proofs for Composite Statements. In *Advances in Cryptology - CRYPTO (Lecture Notes in Computer Science, Vol. 10993)*, Hovav Shacham and Alexandra Boldyreva (Eds.). Springer, 643–673.
- [4] Man Ho Au, Patrick P Tsang, Willy Susilo, and Yi Mu. 2009. Dynamic universal accumulators for DDH groups and their application to attribute-based anonymous credential systems. In *Topics in Cryptology - CT-RSA 2009*, Marc Fischlin (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 295–308.
- [5] F. Baldimtsi, J. Camenisch, M. Dubovitskaya, A. Lysyanskaya, L. Reyzin, K. Samelin, and S. Yakubov. 2017. Accumulators with Applications to Anonymity-Preserving Revocation. In *2017 IEEE European Symposium on Security and Privacy (EuroS P)*. 301–315. <https://doi.org/10.1109/EuroSP.2017.13> <https://eprint.iacr.org/2017/043>.

证明 R_{mem} 和 R_{nonmem} 分别需要2.97毫秒和4.65毫秒。为了证明 $I(s)$ 的度数下界, 证明者需要在有和无 t -PKE 的情况下对 $I(s)$ 计算Pedersen承诺。

验证时间和证明规模。 R_{mem} 的证明为 $(4 \cdot G_1 + G_T + 5 \cdot Z_p)$, 而 R_{nonmem} 的证明为 $(3 \cdot G_1 + G_2 + G_T + 6 \cdot Z_p)$ 。通过用 64 位整数表示次数, 并在 G_2 中使用三个元素, 证明者可以证明多项式的次数下界。我们方案中的所有证明都可以通过常数次的指数运算和配对操作进行验证。

9.3 与HARiSA的比较[18]

在本小节中, 我们论证了在RSA设置下, 对于合理的批量大小选择, 我们的 ZK 批量成员证明方法至少比当前最先进的 ZK 批量成员证明方法快16倍。我们还在图2中报告了 ZK 批量非成员证明的性能。HARiSA不支持非成员证明。

实验设置。 我们将集合的最大规模固定为 2^{17} 个元素, 并测量在揭示批量子集规模的同时计算批量(非)成员关系零知识证明的性能。此外, 我们考虑一个需要证明者完成最大工作量的实验设置。即假设证明者: (1)仅能访问个体(非)成员关系证明, 无法访问批量成员关系证明; (2)无法访问批量子集的承诺; (3)可访问累加器摘要和公共参数。因此, 证明者需承担以下成本: (1)计算集合 I 的承诺; (2)聚合个体(非)成员关系证明以获得批量证明; (3)证明关系 R_{mem} 或 R_{nonmem} 以及(4)证明 $d=|I|$ 时的关系 R_{degcheck} 。

基线基准测试。 我们将本方案的性能与Campanelli等人提出的HARiSA[18]进行对比。该方案通过构建简洁的批量成员证明, 在保持批量元素隐私性的同时实现批量成员验证。其工作结合了指数知识证明(PoKE)与CP-snark算法处理整数算术关系, 并通过边界检查来验证批量成员资格。他们使用C++语言中的LegoGroth16库实现了该构造方案。与我们的实验类似, 该方案要求证明者通过独立见证计算批量见证, 且实验仅针对单个

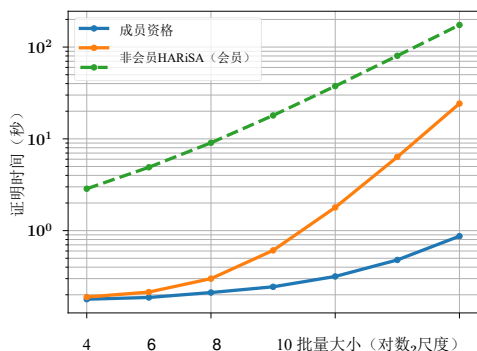


图2: 我们根据HARiSA[18]中报告的数据推算证明成本。需注意, RSA场景下的结果确实包含哈希到素数的计算成本。

方案		设置验证器证明大小		
HARiSA [18]	命令	可信的	63毫秒	1.14 KiB
本工作	命令	可信的	7.94毫秒	1.2 KiB
	非记忆性	可信的	9.41毫秒	1.32 KiB

表5: 验证开销与证明规模。

螺旋化。此外, 需要注意的是, 在我们的实验中揭示了子集 I 的大小, 该功能目前尚未在HARiSA[18]中实现。

证明时间。 HARiSA报告的证明时间分别为16批次和64批次时的2.86秒和9.02秒。需要说明的是, 其实现采用LegoGroth16证明系统, 因此证明时间主要受大规模快速傅里叶变换和指数运算的限制。HARiSA报告的性能数据基于Amazon EC2 r5.8xlarge服务器[18, 图4], 我们通过图2对不同批次规模进行了外推。然而在图2中, 我们的方案是在Intel Core i7-4770 CPU@3.40GHz上进行测试的。我们观察到当批次规模为16时, 我们的方法仅需0.18秒, 而HARiSA需要2.86秒, 实现了16倍的加速。显然, 即使在性能测试所用机器速度较慢的情况下, 我们的性能仍比HARiSA快一个数量级。因此我们认为, 这种数量级的性能提升即使在相同机器上进行方案与HARiSA的基准测试时, 依然能够保持。

据我们所知, 在不依赖SNARKs等通用论证的RSA场景中, 目前尚未发现已知的次线性证明规模的隐私保护批量非成员证明。然而, 基于SNARKs的方法在实际应用中速度过慢。因此, 由于缺乏合适的基准, 我们在成员证明方案的背景下报告了非成员证明方案的性能。

验证时间和证明规模。 我们的方法具有可比的证明规模和更优的验证速度。我们在表5中报告了这些结果。

致谢

我们感谢张玉鹏(YupengZhang)指引我们参考 ZK 多项式次数证明的前期研究, 感谢阿林·托梅斯库(AlinTomescu)的富有成效的讨论。福泰尼·巴尔迪姆齐(FoteiniBaldimtsi)和伊奥安娜·卡拉泰杜(IoannaKarantaidou)的研究工作部分获得美国国家科学基金会(NSF)项目#171767、#2143287以及Facebook和GoogleResearch的资助。查拉拉姆波斯·帕帕曼图(Charalampos-Papamantou)的研究工作部分获得ProtocolLabs、VMware、摩根大通(JPMorgan)、Algorand基金会ACE-PAVE项目资助, 以及美国国家科学基金会(NSF)项目CNS-1652259和CNS-2154732的资助。

参考文献

- [1] 2015年。Hyperledger Indy网站。https://www.hyperledger.org/ [在线; 访问日期: 2022年5月1日]。
- [2] Tolga Acar Lan Nguyen. 2011. 可委托的匿名凭证撤销机制. 在 公钥密码学国际研讨会. 施普林格出版社, 423–440.
- [3] Shashank Agrawal, Chaya Ganesh 和 Payman Mohassel. 2018. 复合语句的非交互式零知识证明. 收录于 密码学进展——密码学(计算机科学讲义, 第10993卷), Hovav Shacham 和 Alexandra Boldyreva (编). 施普林格出版社, 643–673页。
- [4] Man Ho Au, Patrick P Tsang, Willy Susilo 和 Yi Mu. 2009. 用于DDH群的动态通用累加器及其在基于属性的匿名凭证系统中的应用. 收录于 密码学专题——CT-RSA 2009, Marc Fischlin (编). 施普林格出版社, 柏林, 海德堡, 295–308页。
- [5] F. Baldimtsi, J. Camenisch, M. Dubovitskaya, A. Lysyanskaya, L. Reyzin, K. Samelin, 和 S. Yakubov. 2017. 用于匿名保护撤销的累加器. 在 2017 IEEE 欧洲安全与隐私研讨会 (EuroSP). 301–315. https://doi.org/10.1109/EuroSP.2017.13 https://eprint.iacr.org/2017/043.

- [6] Niko Barić and Birgit Pfizmann. 1997. Collision-Free Accumulators and Fail-Stop Signature Schemes Without Trees. In *Advances in Cryptology – EUROCRYPT '97*, Walter Fumy (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 480–494.
- [7] Balthazar Bauer, Georg Fuchsbauer, and Julian Loss. 2020. A classification of computational assumptions in the algebraic group model. In *Annual International Cryptology Conference*. Springer, 121–151.
- [8] Josh Benaloh and Michael de Mare. 1993. One-Way Accumulators: A Decentralized Alternative to Digital Signatures. In *Advances in Cryptology – EUROCRYPT '93*, Tor Helleseth (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 274–285. <https://www.microsoft.com/en-us/research/publication/one-way-accumulators-a-decentralized-alternative-to-digital-signatures/>
- [9] Daniel Benarroch, Matteo Campanelli, Dario Fiore, Kobi Gurkan, and Dimitris Kolonelos. 2021. Zero-Knowledge Proofs for Set Membership: Efficient, Succinct, Modular. In *Financial Cryptography and Data Security*. Springer Berlin Heidelberg, 393–414. https://doi.org/10.1007/978-3-662-64322-8_19
- [10] Dan Boneh, Benedikt Bünz, and Ben Fisch. 2019. Batching Techniques for Accumulators with Applications to IOPs and Stateless Blockchains. In *Advances in Cryptology – CRYPTO 2019*, Alexandra Boldyreva and Daniele Micciancio (Eds.). Springer, Springer International Publishing, Cham, 561–586. <https://eprint.iacr.org/2018/1188>.
- [11] Sean Bowe. 2017. Fast amortized Kate proofs. <https://electriccoin.co/blog/new-snark-curve/>.
- [12] Sean Bowe, Ariel Gabizon, and Ian Miers. 2017. Scalable Multi-party Computation for zk-SNARK Parameters in the Random Beacon Model. Cryptology ePrint Archive, Report 2017/1050. <https://ia.cr/2017/1050>.
- [13] Johannes Buchmann and Safuat Hamdy. 2011. A survey on IQ cryptography. In *Public-Key Cryptography and Computational Number Theory*, 1–15.
- [14] Vitalik Buterin. 2020. Using polynomial commitments to replace state roots. <https://ethresear.ch/t/using-polynomial-commitments-to-replace-state-roots/7095>.
- [15] Jan Camenisch, Markulf Kohlweiss, and Claudio Soriente. 2009. An Accumulator Based on Bilinear Maps and Efficient Revocation for Anonymous Credentials. In *Public Key Cryptography – PKC 2009*, Stanisław Jarecki and Gene Tsudik (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 481–500. <https://eprint.iacr.org/2008/539>.
- [16] Jan Camenisch and Anna Lysyanskaya. 2002. Dynamic Accumulators and Application to Efficient Revocation of Anonymous Credentials. In *Advances in Cryptology – CRYPTO 2002*, Moti Yung (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 61–76.
- [17] Matteo Campanelli, Dario Fiore, Nicola Greco, Dimitris Kolonelos, and Luca Nizardo. 2020. Incrementally Aggregatable Vector Commitments and Applications to Verifiable Decentralized Storage. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 3–35.
- [18] Matteo Campanelli, Dario Fiore, Semin Han, Jihye Kim, Dimitris Kolonelos, and Hyunok Oh. 2021. Succinct Zero-Knowledge Batch Proofs for Set Accumulators. Cryptology ePrint Archive, Paper 2021/1672. <https://eprint.iacr.org/2021/1672> <https://eprint.iacr.org/archive/2021/1672/20220503:104027>.
- [19] Ran Canetti, Omer Paneth, Dimitrios Papadopoulos, and Nikos Triandopoulos. 2014. Verifiable Set Operations over Outsourced Databases. In *Public-Key Cryptography – PKC 2014*, Hugo Krawczyk (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 113–130.
- [20] Dario Catalano and Dario Fiore. 2013. Vector Commitments and Their Applications. In *International Workshop on Public Key Cryptography*, Kaoru Kurosawa and Goichiro Hanaoka (Eds.). Springer, Springer Berlin Heidelberg, Berlin, Heidelberg, 55–72. <https://eprint.iacr.org/2011/495>.
- [21] Alexander Chepur, Charalampos Papamanthou, Shravan Srinivasan, and Yupeng Zhang. 2018. Edrax: A Cryptocurrency with Stateless Transaction Validation. Cryptology ePrint Archive, Report 2018/968. <https://ia.cr/2018/968>.
- [22] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. 2020. Marlin: Preprocessing zkSNARKs with universal and updatable srs. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 738–768.
- [23] Aisling Connolly, Pascal Lafourcade, and Octavio Perez Kempner. 2022. Improved Constructions of Anonymous Credentials from Structure-Preserving Signatures on Equivalence Classes. In *Public-Key Cryptography – PKC 2022*, Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe (Eds.). Springer International Publishing, Cham, 409–438.
- [24] Ivan Damgård and Nikos Triandopoulos. 2008. Supporting Non-membership Proofs with Bilinear-map Accumulators. Cryptology ePrint Archive, Report 2008/538. <https://eprint.iacr.org/2008/538>.
- [25] Dankrad Feist and Dmitry Khovratovich. 2020. Fast amortized Kate proofs. <https://github.com/khovratovich/Kate>.
- [26] Esha Ghosh, Olga Ohrimenko, Dimitrios Papadopoulos, Roberto Tamassia, and Nikos Triandopoulos. 2016. Zero-Knowledge Accumulators and Set Algebra. In *Advances in Cryptology – ASIACRYPT 2016*. Springer Berlin Heidelberg, 67–100. https://doi.org/10.1007/978-3-662-53890-6_3
- [27] Sergey Gorbunov, Leonid Reyzin, Hoeteck Wee, and Zhenfei Zhang. 2020. Pointproofs: Aggregating Proofs for Multiple Vector Commitments. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, USA) (CCS '20). Association for Computing Machinery, New York, NY, USA, 2007–2023. <https://doi.org/10.1145/3372297.3417244> <https://eprint.iacr.org/2020/419>.
- [28] Jens Groth. 2010. Short pairing-based non-interactive zero-knowledge arguments. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 321–340.
- [29] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. 2010. Constant-Size Commitments to Polynomials and Their Applications. In *Advances in Cryptology – ASIACRYPT 2010*, Masayuki Abe (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 177–194. <https://www.iacr.org/archive/asiacrypt2010/6477178/6477178.pdf>.
- [30] Dmitry Khovratovich and Jason Law. 2017. Sovrin: digital identities in the blockchain era. *Github Commit by jasonlaw October 17 (2017)*, 38–99.
- [31] Russell W.F. Lai and Giulio Malavolta. 2019. Subvector Commitments with Application to Succinct Arguments. In *Advances in Cryptology – CRYPTO 2019*, Alexandra Boldyreva and Daniele Micciancio (Eds.). Springer, Springer International Publishing, Cham, 530–560. <https://eprint.iacr.org/2018/705>.
- [32] Jiangtao Li, Ninghui Li, and Rui Xue. 2007. Universal Accumulators with Efficient Nonmembership Proofs. In *Applied Cryptography and Network Security*, Jonathan Katz and Moti Yung (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 253–269. https://www.cs.purdue.edu/homes/ninghui/papers/accumulator_acns07.pdf.
- [33] Helger Lipmaa. 2012. Secure Accumulators from Euclidean Rings without Trusted Setup. In *Applied Cryptography and Network Security – 10th International Conference, ACNS 2012, Singapore, June 26–29, 2012. Proceedings*, Feng Bao, Pierangela Samarati, and Jianying Zhou (Eds.), Vol. 7341. Springer, 224–240.
- [34] Ralph C. Merkle. 1988. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology – CRYPTO '87*, Carl Pomerance (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 369–378.
- [35] Ian Miers, Christina Garman, Matthew Green, and Aviel D. Rubin. 2013. Zerocoin: Anonymous Distributed E-Cash from Bitcoin. In *2013 IEEE Symposium on Security and Privacy*. 397–411. <https://doi.org/10.1109/SP.2013.34>
- [36] Mitsunari Shigeo. 2020. mcl: a portable and fast pairing-based cryptography library. <https://github.com/herumi/mcl/> Accessed: 2020-10-14.
- [37] Kartik Nayak, Ling Ren, Elaine Shi, Nitin H. Vaidya, and Zhuolun Xiang. 2020. Improved Extension Protocols for Byzantine Broadcast and Agreement. In *34th International Symposium on Distributed Computing (DISC 2020) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 179)*, Hagit Attiya (Ed.), Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 28:1–28:17.
- [38] Lan Nguyen. 2005. Accumulators from Bilinear Pairings and Applications. In *Topics in Cryptology – CT-RSA 2005*, Alfred Menezes (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 275–292. <https://eprint.iacr.org/2005/123>.
- [39] Alex Ozdemir, Riad Wahby, Barry Whitehat, and Dan Boneh. 2020. Scaling Verifiable Computation Using Efficient Set Accumulators. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 2075–2092. <https://www.usenix.org/conference/usenixsecurity20/presentation/ozdemir>
- [40] Charalampos Papamanthou. 2011. *Cryptography for Efficiency: New Directions in Authenticated Data Structures*. Ph.D. Dissertation. Brown University, Providence, Rhode Island. <https://doi.org/10.7301/Z0Z60M99>.
- [41] Charalampos Papamanthou, Roberto Tamassia, and Nikos Triandopoulos. 2011. Optimal Verification of Operations on Dynamic Sets. In *Advances in Cryptology – CRYPTO 2011*, Phillip Rogaway (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 91–110.
- [42] Torben Pryds Pedersen. 1991. Non-interactive and information-theoretic secure verifiable secret sharing. In *Annual international cryptology conference*. Springer, 129–140.
- [43] Shi-Feng Sun, Man Ho Au, Joseph K Liu, and Tsz Hon Yuen. 2017. Ringct 2.0: A compact accumulator-based (linkable ring signature) protocol for blockchain cryptocurrency monero. In *European Symposium on Research in Computer Security*. Springer, 456–474.
- [44] Steve Thakur. 2019. Batching non-membership proofs with bilinear accumulators. Cryptology ePrint Archive, Report 2019/1147. <https://eprint.iacr.org/archive/2019/1147/20210929:175523>.
- [45] The OpenSSL Project. 2003. OpenSSL: The Open Source toolkit for SSL/TLS. (April 2003). www.openssl.org.
- [46] Alin Tomescu. 2022. go-mcl. <https://github.com/alinhush/go-mcl> <https://github.com/alinhush/go-mcl> [Online; accessed 1-May-2022].
- [47] Alin Tomescu, Ittai Abraham, Vitalik Buterin, Justin Drake, Dankrad Feist, and Dmitry Khovratovich. 2020. Aggregatable Subvector Commitments for Stateless Cryptocurrencies. In *Security and Cryptography for Networks*. Springer International Publishing, Cham, 45–64. <https://eprint.iacr.org/2020/527>.
- [48] Alin Tomescu, Vivek Bhupatiraju, Dimitrios Papadopoulos, Charalampos Papamanthou, Nikos Triandopoulos, and Srinivas Devadas. 2019. Transparency logs via append-only authenticated dictionaries. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 1299–1316.
- [49] Giuseppe Vito and Alex Biryukov. 2020. Dynamic Universal Accumulator with Batch Update over Bilinear Groups. Cryptology ePrint Archive, Report 2020/777. <https://eprint.iacr.org/2020/777>.

- [6] Niko Baric 和 Birgit Pfiztmann. 1997. 无碰撞累加器与无树故障停止签名方案. 收录于 *密码学进展——eurocrypt 97*, 沃尔特·富米(编). 施普林格出版社, 柏林, 海德堡, 480–494页。
- [7] Balthazar Bauer、Georg Fuchsbauer 和 Julian Loss. 2020. 代数群模型中计算假设的分类. 在 *国际密码学年会*. 施普林格出版社, 121–151.
- [8] Josh Benaloh 和 Michael de Mare. 1993. 单向累加器: 数字签名的去中心化替代方案. 收录于 *密码学进展——eurocrypt 93*, Tor Helleseth (编). 施普林格出版社, 柏林, 海德堡, 274–285 页. <https://www.microsoft.com/en-us/research/publication/one-way-accumulators-a-decentralized-alternative-to-digital-signatures/>
- [9] Daniel Benarroch、Matteo Campanelli、Dario Fiore、Kobi Gurkan 和 Dimitris Kolonelos. 2021年. 集合成员资格的零知识证明: 高效、简洁、模块化. 见 *《金融密码学与数据安全》*. 施普林格出版社, 柏林-海德堡, 393-414 页. https://doi.org/10.1007/978-3-662-64322-8_19
- [10] Dan Boneh、Benedikt Bunz 和 Ben Fisch. 2019年. 累加器的批处理技术及其在输入输出处理器和无状态区块链中的应用. 收录于 *密码学进展——crypto 2019*, Alexandra Boldyreva 和 Daniele Micciancio (编). Springer出版社, Springer国际出版集团, Cham, 561–586. <https://eprint.iacr.org/2018/1188>.
- [11] 肖恩·鲍. 2017年. 快速折旧的凯特证明. <https://electriccoin.co/blog/new-snark-curve->
- [12] Sean Bowe、Ariel Gabizon 和 Ian Miers. 2017年. 随机信标模型中zk-snark参数的可扩展多方计算. 密码学电子预印本档案, 报告2017/1050. <https://ia.cr/2017/1050>.
- [13] Johannes Buchmann 和 Safuat Hamdy. 2011. *《IQ密码学综述》*. 收录于 *《公钥密码学与计算数论》* 第1–15页。
- [14] 维塔利克·布特林. 2020年. 用多项式承诺替代状态根. <https://ethresear.ch/t/using-polynomial-commitments-to-replace-state-roots/7095>.
- [15] Jan Camenisch, Markulf Kohlweiss, and Claudio Soriente. 2009. 基于双线性映射的匿名凭证高效撤销机制的累加器. 收录于 *公钥密码学 - PKC 2009*, Stanislaw Jarecki 和 Gene Tsudik (编). 施普林格柏林海德堡出版社, 柏林, 海德堡, 481–500. <https://eprint.iacr.org/2008/539>.
- [16] Jan Camenisch Anna Lysyanskaya. 2002. 动态累加器及其在匿名凭证高效撤销中的应用. 收录于 *加密学进展——crypto 2002*, Moti Yung (编). 施普林格出版社, 柏林, 海德堡, 61–76.
- [17] Matteo Campanelli、Dario Fiore、Nicola Greco、Dimitris Kolonelos 和 Luca Nizzardo. 2020年. 增量可聚合向量承诺及其在可验证去中心化存储中的应用. 收录于 *国际密码学与信息安全理论及应用会议*. 施普林格出版社, 3–35页。
- [18] Matteo Campanelli、Dario Fiore、Semin Han、Jihye Kim、Dimitris Kolonelos 和 Hyunok Oh. 2021年. 集合累加器的简洁零知识批量证明. 密码学ePrint档案, 论文2021/1672. <https://eprint.iacr.org/2021/1672> <https://eprint.iacr.org/archive/2021/1672/20220503:104027>.
- [19] Ran Canetti、Omer Paneth、Dimitrios Papadopoulos 和 Nikos Triandopoulos. 2014年. *《外包数据库上的可验证集合操作》*. 收录于 *《公钥密码学——PKC 2014》*, Hugo Krawczyk (编). 施普林格出版社, 柏林, 海德堡, 113–130页。
- [20] Dario Catalano Dario Fiore. 2013. 向量承诺及其应用. 在 *公钥密码学国际研讨会*, 黑泽熏和花冈刚一郎(编). 施普林格出版社, 施普林格柏林海德堡, 柏林, 海德堡, 55–72. <https://eprint.iacr.org/2011/495>.
- [21] Alexander Chepurinov、Charalampos Papamanthou、Shrivani Srinivasan 和 Yupeng Zhang. 2018年. Edrax: 一种具有无状态交易验证功能的加密货币. 密码学电子预印本档案, 报告2018/968. <https://ia.cr/2018/968>.
- [22] Alessandro Chiesa、Yuncong Hu、Mary Maller、Pratyush Mishra、Noah Vesely 和 Nicholas Ward. 2020年. Marlin: 使用通用且可更新的srs对zk-snarks进行预处理. 收录于 *密码技术理论与应用国际年会*. Springer出版社, 738–768页。
- [23] Aisling Connolly、Pascal Lafourcade 和 Octavio Perez·肯普纳. 2022年. 基于等价类结构保持签名的匿名凭证构造证明. 收录于 *公钥密码学——PKC 2022*, 花冈刚一郎、志方淳二、渡边洋平(编). 施普林格国际出版, Cham, 409–438.
- [24] 伊万·达姆加德与尼科斯·特里安多普洛斯. 2008年. 基于双线性映射累加器的非成员证明支持. 密码学电子预印本档案, 报告2008/538. <https://eprint.iacr.org/2008/538>.
- [25] 丹克雷德·费斯特与德米特里·霍夫拉托维奇. 2020年. 快速归还的Kate证明. <https://github.com/khovratovich/Kate>.
- [26] Esha Ghosh、Olga Ohrimenko、Dimitrios Papadopoulos、Roberto Tamassia 和 Nikos Triandopoulos. 2016. 零知识累加器与集合代数. 收录于 *密码学进展——asiacrypt 2016*. 施普林格出版社, 柏林-海德堡, 67-100 页. https://doi.org/10.1007/978-3-662-53890-6_3
- [27] Sergey Gorbunov、Leonid Reyzin、Hoeteck Wee 和 Zhenfei Zhang. 2020年. Pointproofs: 多向量承诺的聚合证明. 收录于 *2020年ACM计算机与通信SIGSAC会议论文集* 安全(虚拟活动, 美国)(中国化学会 20). 美国纽约州纽约市计算机协会, 2007–2023年. <https://doi.org/10.1145/3372297.3417244> <https://eprint.iacr.org/2020/419>.
- [28] Jens Groth. 2010. 基于短配对的非交互式零知识论证. 在 *密码学与信息安全理论及应用国际会议*. 施普林格出版社, 321–340.
- [29] Aniket Kate、Gregory M. Zaverucha 和 Ian Goldberg. 2010年. *《多项式恒定规模承诺及其应用》*. 收录于 *《密码学进展——亚洲密码学2010》*, 由Masayuki Abe (编). 施普林格出版社, 柏林, 海德堡, 177–194. <https://www.iacr.org/archive/asiacrypt2010/6477178/6477178.pdf>.
- [30] 德米特里·霍夫拉托维奇与杰森·劳. 2017年. *《主权: 区块链时代的数字身份》*. GitHub 由 jasonlaw 于 2017 年 10 月 17 日提交, 第 38–99 页。
- [31] Russell W.F. Lai 和 Giulio Malavolta. 2019. 适用于简洁论证的向量承诺. 收录于 *密码学进展——crypto 2019*, Alexandra Boldyreva 和 Daniele Micciancio (编). 施普林格国际出版集团, Cham, 530–560. <https://eprint.iacr.org/2018/705>.
- [32] Jiangtao Li、Ninghui Li、and Rui Xue. 2007. 具有高效非成员证明的通用累加器. 收录于 *应用密码学与网络安全*, Jonathan Katz 和 Moti Yung (编). 施普林格柏林海德堡, 柏林, 海德堡, 253–269. https://www.cs.purdue.edu/homes/ninghui/papers/accumulator_acns07.pdf.
- [33] Helger Lipmaa. 2012. 无需可信设置的欧几里得环安全累加器. 在 *应用密码学与网络安全-第10届国际会议, ACNS 2012, 新加坡, 2012年6月26-29日. 会议论文集*, 冯宝, 皮兰杰拉·萨马拉蒂和周建英(编), 第7341卷. 施普林格出版社, 224-240 页。
- [34] Ralph C.默克尔. 1988. 基于传统加密函数的数字签名. 收录于 *《密码学进展——crypto 87》*, 卡尔·波默兰斯(编). 施普林格出版社, 柏林, 汉诺威, 369–378.
- [35] Ian Miers、Christina Garman、Matthew Green 和 Aviel D. Rubin. 2013年. Zerocoin: 一种基于比特币的匿名分布式电子现金. 在 *2013年IEEE安全与隐私研讨会*. 397–411. <https://doi.org/10.1109/SP.2013.34>
- [36] Mitsunari Shigeo. 2020. mcl: 一个便携且易于快速配对的密码库. <https://github.com/herumi/mcl> 访问日期: 2020-10-14.
- [37] Kartik Nayak、Ling Ren、Elaine Shi、Nitin H. Vaidya 和 Zhuolun Xiang. 2020年. 拜占庭广播与协议的改进扩展协议. 收录于 *第34届国际分布式计算研讨会(DISC 2020)* (莱布尼茨国际信息学会议论文集(LIPIcs), 第179卷), 哈吉特·阿提亚(编). 德国达格斯图尔城堡-莱布尼茨信息学中心, 达格斯图尔, 28: 1–28:17.
- [38] Lan Nguyen. 2005. 双线性配对中的累加器及其应用. 收录于 *密码学专题——CT-RSA 2005*, Alfred Menezes (编). 施普林格出版社, 柏林, 海德堡, 275–292页. <https://eprint.iacr.org/2005/123>.
- [39] Alex Ozdemir、Riad Wahby、Barry Whitehat、and Dan Boneh. 2020. 使用高效集合累加器扩展可验证计算. 在 *第29届USENIX安全研讨会(USENIX安全20)*上. USENIX 协会, 2075–2092. <https://www.usenix.org/conference/usenixsecurity20/presentation/ozdemir>
- [40] 查拉拉波斯·帕帕曼图. 2011年. *高效密码学: 认证数据结构的新方向*. 博士论文. 布朗大学, 罗德岛州普罗维登斯. <https://doi.org/10.7301/Z0Z60M99>.
- [41] Charalampos Papamanthou、Roberto Tamassia 和 Nikos Triandopoulos. 2011. 动态集合操作的最优验证. 收录于 *密码学进展——crypto 2011*, 菲利普·罗加韦(编). 施普林格出版社, 柏林, 海德堡, 91–110页。
- [42] 托本·普里兹·佩德森. 1991年. 非交互式与信息论安全可验证秘密共享. 载于 *国际密码学年会*. 施普林格出版社, 129–140页。
- [43] Shi-Feng Sun、Man Ho Au、Joseph K. Liu 和 Tsz Hon Yuen. 2017. Ringct 2.0: 一种基于紧凑型累加器(可链接环签名)的区块链加密货币门罗币协议. 收录于 *欧洲计算机安全研究研讨会*. 施普林格出版社, 456–474页。
- [44] 史蒂夫·塔库尔. 2019年. 基于双线性累加器的批处理非成员证明. 密码学电子预印本档案, 报告2019/1147. <https://eprint.iacr.org/archive/2019/1147/20210929:175523>.
- [45] OpenSSL项目. 2003年. OpenSSL: SSL/TLS的开源工具包。(2003年4月)。www.openssl.org。
- [46] Alin Tomescu. 2022. go-mcl. <https://github.com/alinush/go-mcl> <https://github.com/alinush/go-mcl> [在线; 访问日期: 2022年5月1日]。
- [47] Alin Tomescu、Ittai Abraham、Vitalik Buterin、Justin Drake、Dankrad Feist 和 Dmitry Khovratovich. 2020年. 无状态加密货币的可聚合向量承诺. 收录于 *《网络安全与密码学》*. 施普林格国际出版公司, Cham, 45-64 页. <https://eprint.iacr.org/2020/527>.
- [48] Alin Tomescu、Vivek Bhupatiraju、Dimitrios Papadopoulos、Charalampos Papamanthou、Nikos Triandopoulos 和 Srinivas Devadas. 2019年. 通过(仅可追加的)认证字典实现透明度日志. 收录于 *2019年ACM SIGSAC 计算机与通信安全会议论文集*, 第1299–1316页。
- [49] 朱塞佩·维托与亚历克斯·比留科夫. 2020年. 基于双线性群的动态通用累加器与批量更新. 密码学电子预印本档案, 报告2020/777. <https://eprint.iacr.org/2020/777>.

- [50] Thomas Walton-Pocock. 2020. AZTEC CRS: The Biggest MPC Setup in History has Successfully Finished. <https://medium.com/aztec-protocol/aztec-crs-the-biggest-mpc-setup-in-history-has-successfully-finished-74c6909cd0c4>.
- [51] Benjamin Wesolowski. 2019. Efficient Verifiable Delay Functions. In *Advances in Cryptology – EUROCRYPT 2019*, Yuval Ishai and Vincent Rijmen (Eds.). Springer International Publishing, Cham, 379–407. <https://eprint.iacr.org/2018/623>.
- [52] Wikipedia contributors. 2020. The GNU Multiple Precision Arithmetic Library — Wikipedia, The Free Encyclopedia. <https://gmplib.org/manual/Extended-GCD>
- [53] Wikipedia contributors. 2020. Partial fraction decomposition — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Partial_fraction_decomposition&oldid=931350868 [Online; accessed 11-April-2020].
- [54] Wikipedia contributors. 2020. Polynomial greatest common divisor — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Polynomial_greatest_common_divisor&oldid=993818674#B%C3%A9zout's_identity_and_extended_GCD_algorithm [Online; accessed 12-December-2020].
- [55] Wikipedia contributors. 2020. Polynomial remainder theorem — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Product_rule&oldid=992085655 [Online; accessed 03-December-2020].
- [56] Dae Hyun Yum, Jae Woo Seo, and Pil Joong Lee. 2008. Generalized Combinatoric Accumulator. *IEICE - Trans. Inf. Syst.* E91-D, 5 (May 2008), 1489–1491. <https://doi.org/10.1093/ietisy/e91-d.5.1489>
- [57] Y. Zhang, J. Katz, and C. Papamanthou. 2017. An Expressive (Zero-Knowledge) Set Accumulator. In *2017 IEEE European Symposium on Security and Privacy (EuroSP)*. 158–173. <https://doi.org/10.1109/EuroSP.2017.35>

- [50] 托马斯·沃尔顿-波科克. 2020年. 阿兹特克CRS: 史上最大MPC设置已成功完成。
<https://medium.com/aztec-protocol/aztec-crs-the-biggest-mpc-setup-in-history-has-successfully-finished-74c6909cd0e4>。
- [51] Benjamin Wesolowski . 2019. 高效可验证延迟函数. 在*密码学进展——eurocrypt 2019*中, Yuval Ishai 和 Vincent Rijmen (编). 施普林格国际出版, Cham, 379–407。
<https://eprint.iacr.org/2018/623>。
- [52] 维基百科贡献者. 2020年. GNU多精度算术库——维基百科, 自由百科全书。ht-
[tps://gmplib.org/manual/Extended-GCD](https://gmplib.org/manual/Extended-GCD)
- [53] 维基百科贡献者. 2020年. 部分分式分解——维基百科, 自由百科全书。
https://en.wikipedia.org/w/index.php?title=Partial_fraction_decomposition&oldid=931350868 [在线; 访问日期: 2020年4月11日]。
- [54] 维基百科贡献者. 2020年. 多项式最大公约数——维基百科, 自由百科全书。
https://en.wikipedia.org/w/index.php?title=Polynomial_greatest_common_divisor&oldid=993818674#B%C3%A9zout's_identity_and_extended_GCD_algorithm [在线; 访问日期: 2020年12月12日]。
- [55] 维基百科贡献者. 2020年. 多项式余数定理——维基百科, 自由百科全书。
https://en.wikipedia.org/w/index.php?title=Product_rule&oldid=992085655 [在线; 访问日期: 2020年12月3日]。
- [56] Dae Hyun Yum, Jae Woo Seo 和 Pil Joong Lee. 2008. 通用组合累积器. *IEICE - 信息统汇刊* E91-D, 5 (2008年5月), 1489–1491. <https://doi.org/10.1093/ietisy/e91-d.5.1489>
- [57] Y. Zhang, J. Katz 和 C. Papamanthou. 2017. 一种表达性 (零知识) 集合累加器. 在 *2017 IEEE 欧洲安全与隐私研讨会 (EuroSP)*. 158–173. <https://doi.org/10.1109/EuroSP.2017.35>